

itelescope_premium_the_hard_way

November 26, 2024

1 iTelescope premium image set processing, the hard way

Author: Dave Strickland

Version: 0.5.2-beta1

This notebook illustrates how to process a premium image set from iTelescope using the AstroPhotography package. The python environment used corresponds to a miniconda environment `ap-env.yml`.

The example dataset used is the iTelescope Plan-20 premium dataset of the M101 supernova [SN 2023 ixf](#). Note that these files are not provided with this package. However the notebook should work with your own files if you specify a valid fits-file containing directory at the prompt below.

1.1 Notebook environment setup

```
1]: import math
import os
import pathlib
import shlex
import subprocess
import sys
import time

import AstroPhotography as ap
import astropy
import matplotlib
import matplotlib.pyplot as plt
import numpy as np
from astropy import wcs
from astropy.io import fits
from astropy.table import Table
from ccdproc import ImageFileCollection

2]: def print_module_version(mod):
    """
    Convenience function to pretty-print Python module mod version.

    Parameters
    -----
    mod : {module}
        Imported Python module

    Returns
    -----
    str
        module name and version
    """
    print(f"Using module {mod.__name__:30s} version: {mod.__version__}")
    return

3]: # Get Version information
print(f"Python version: {sys.version}")
print_module_version(np)
print_module_version(matplotlib)
print_module_version(astropy)
print_module_version(ap)

Python version: 3.10.13 | packaged by conda-forge | (main, Dec 23 2023,
15:36:39) [GCC 12.3.0]
Using module numpy                version: 1.26.3
Using module matplotlib           version: 3.8.2
Using module astropy              version: 6.0.0
Using module AstroPhotography     version: 0.6.0

4]: # Enable inline plotting for graphics
# %matplotlib inline
# Set default figure size to be larger
```

```
# this may only work in matplotlib 2.0+!
from IPython.core.interactiveshell import InteractiveShell

matplotlib.rcParams["figure.figsize"] = [10.0, 6.0]
# Enable multiple outputs from jupyter cells
InteractiveShell.ast_node_interactivity = "all"
```

2 Doing it by hand

2.1 What files do we have to work with?

Let us change the working directory from the location of this notebook to a directory holding some iTelescope premium image set files. In my case this is the Plan-20 M101 dataset.

Note: The following processing is fairly low-level and requires a lot of human interaction. In a later section we'll process the same set of files in a more automated way.

```
5]: print("Enter the path to the directory containing the premium image set")
wdir = input("Image set path:").strip()
try:
    os.chdir(wdir)
    print("Switched directory to " + os.getcwd())
except:
    print(f"Error, os.chdir threw an exception changing to {wdir}")
    print("Check that the path you supplied is a valid filesystem path.")
    raise
```

Enter the path to the directory containing the premium image set

Image set path: /old_lnx/home/dks/Downloads/iTelescopeScratch/Plan-20-M101SN

Switched directory to

/old_lnx/home/dks/Downloads/iTelescopeScratch/Plan-20-M101SN

We now want a pattern that can be used to select the FITS files we want to process, while ignoring any other files or directories we have in that directory (e.g. processed files, configurations files, and so on).

Listing the fits files from the shell I get:

```
ls ../Plan-20-M101SN/*fits
../Plan-20-M101SN/Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-1.fits
../Plan-20-M101SN/Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-2.fits
../Plan-20-M101SN/Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-3.fits
../Plan-20-M101SN/Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Blue-1.fits
../Plan-20-M101SN/Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Blue-2.fits
../Plan-20-M101SN/Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Blue-3.fits
../Plan-20-M101SN/Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Green-1.fits
../Plan-20-M101SN/Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Green-2.fits
../Plan-20-M101SN/Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Green-3.fits
../Plan-20-M101SN/Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Red-1.fits
../Plan-20-M101SN/Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Red-2.fits
../Plan-20-M101SN/Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Red-3.fits
```

So a valid pattern that picks up the original files is Calibrated-iTelescope*.fits

```
6]: default_pattern = "Calibrated-iTelescope*.fits"
print(f"Default file pattern for input files: {default_pattern}")
msg = "Enter new input file pattern (or return to accept default pattern)"
file_pattern = input(msg).strip() or default_pattern
print(f'Using "{file_pattern}" as the input file pattern.')
```

Default file pattern for input files: Calibrated-iTelescope*.fits

Enter new input file pattern (or return to accept default pattern)

Using "Calibrated-iTelescope*.fits" as the input file pattern.

```
7]: p_dir = pathlib.Path(r"./")
flist = list(sorted(p_dir.glob(file_pattern)))
print(f'{len(flist)} files match pattern "{file_pattern}" in current directory.')
for fpath in flist:
    print(f" {fpath.name}")
```

12 files match pattern "Calibrated-iTelescope*.fits" in current directory.

```
Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-1.fits
Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-2.fits
Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-3.fits
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Blue-1.fits
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Blue-2.fits
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Blue-3.fits
```

Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Green-1.fits
 Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Green-2.fits
 Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Green-3.fits
 Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Red-1.fits
 Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Red-2.fits
 Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Red-3.fits

2.1.1 Do these have full headers?

Lets have a look at the FITS header for the first file in this list.

```
8]: hdr = None
with fits.open(flist[0], mode="readonly") as hdulist:
    print(f"Opened {flist[0].name}")
    hdulist.verify("fix")
    print(hdulist.info())
    hdr = hdulist[0].header
    print(f"Printing the FITS header, length={len(hdr)}")
    hdr
```

Opened Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-1.fits
 Filename: Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-1.fits

No.	Name	Ver	Type	Cards	Dimensions	Format
0	PRIMARY	1	PrimaryHDU	17	(3056, 3056)	float32

None
 Printing the FITS header, length=17

```
9]: SIMPLE = T / conforms to FITS standard
BITPIX = -32 / array data type
NAXIS = 2 / number of array dimensions
NAXIS1 = 3056
NAXIS2 = 3056
EXTEND = T
OWNER = 'iTelescope'
RA = '14 03 12.40' / [hms J2000] Target right ascension
OBJCTRA = '14 03 12.40' / [hms J2000] Target right ascension
DEC = '+54 20 58.0' / [dms +N J2000] Target declination
OBJCTDEC = '+54 20 58.0' / [dms +N J2000] Target declination
FOCALLEN = 3962.0 / Focal length of telescope in mm
APTDIA = 610.0 / Aperture diameter of telescope in mm
EXPTIME = 180.0 / Exposure time in seconds
FILTER = 'Lum' / Filter type used
HIERARCH OBSTARGET = 'M101 Supernova 2023ixf' / Object imaged
SRCLINK = '3jnv6zuw' / Original source file ID
```

```
9]: # What keywords do we have (non-comment, non-history)
sorted(list(hdr.keys()))
```

```
9]: ['APTDIA',
'BITPIX',
'DEC',
'EXPTIME',
'EXTEND',
'FILTER',
'FOCALLEN',
'NAXIS',
'NAXIS1',
'NAXIS2',
'OBJCTDEC',
'OBJCTRA',
'OBSTARGET',
'OWNER',
'RA',
'SIMPLE',
'SRCLINK']
```

Lets look at the keyword differences in more detail

```
def fkeywords(fname):
    with fits.open(fname) as hdulist:
        hdr=hdulist[0].header
        print(f'FITS header for primary HDU of {fname}')
        print( sorted( list( hdr.keys() ) ) )
    return
```

```
f1='Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-1.fits'
```

On the M101 files we've been looking at:

```
fkeywords(f1)
```

```
FITS header for primary HDU of Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-1.fits
```

```
['APTDIA', 'BITPIX', 'DEC', 'EXPTIME', 'EXTEND', 'FILTER', 'FOCALLEN', 'NAXIS', 'NAXIS1', 'NAXIS2', 'OBJCTDEC', 'OBJCTRA', 'OBSTARGET', 'OWNE
```

```
# iTelescope calibration system
```

```
In [17]: f2_itel='calibrated-T05-davestrickland-M82-20210219-002600-Red-BIN1-W-300-001.fit'
```

```
# iTelescope raw file
```

```
In [18]: f2_raw='raw-T05-davestrickland-M82-20210219-002600-Red-BIN1-W-300-001.fit'
```

```
# Calibrated file produced by AstroPhotography from the raw file above...
```

```
In [19]: f2_apcal='cal-T05-davestrickland-M82-20210219-002600-Red-BIN1-W-300-001.fits'
```

```
In [20]: fkeywords(f2_raw)
```

```
FITS header for primary HDU of raw-T05-davestrickland-M82-20210219-002600-Red-BIN1-W-300-001.fit
```

```
['APTAREA', 'APTDIA', 'BITPIX', 'BSCALE', 'BZERO', 'CBLACK', 'CCD-TEMP', 'CSTRETCH', 'CWHITE', 'DATE-OBS', 'EGAIN', 'EXPOSURE', 'EXPTIME', 'F
```

```
In [21]: fkeywords(f2_itel)
```

```
FITS header for primary HDU of calibrated-T05-davestrickland-M82-20210219-002600-Red-BIN1-W-300-001.fit
```

```
['AIRMASS', 'ALT-OBS', 'APTAREA', 'APTDIA', 'BITPIX', 'BSCALE', 'BZERO', 'CALSTAT', 'CBLACK', 'CCD-TEMP', 'CSTRETCH', 'CWHITE', 'DATE', 'DATE
```

```
In [22]: fkeywords(f2_apcal)
```

```
FITS header for primary HDU of cal-T05-davestrickland-M82-20210219-002600-Red-BIN1-W-300-001.fits
```

```
['AIRMASS', 'ALT-OBS', 'APTAREA', 'APTDIA', 'BIASCORR', 'BIASFILE', 'BITPIX', 'BPIXCORR', 'BPIXDPIX', 'BPIXFILE', 'BPIXNBAD', 'BPIXNFI
```

After some rearranging by hand we get the following table of header keywords. The premium file is clearly missing a large number of header keywords, although this doesn't tell us which ones are needed and which ones are superfluous.

iTelescope raw	iTelescope calibrated	AP calibrated	M101 premium
	AIRMASS	AIRMASS	
	ALT-OBS	ALT-OBS	
APTAREA	APTAREA	APTAREA	
APTDIA	APTDIA	APTDIA	APTDIA
		BIASCORR	
		BIASFILE	
BITPIX	BITPIX	BITPIX	BITPIX
		BPIXCORR	
		BPIXDPIX	
		BPIXFILE	
		BPIXNBAD	
		BPIXNFI	
		BPIXNREM	
		BPIX_MIN	
BSCALE	BSCALE		
BZERO	BZERO		
		BUNIT	
	CALSTAT		
CBLACK	CBLACK	CBLACK	
CCD-TEMP	CCD-TEMP	CCD-TEMP	
		CR_CLEAN	
		CR_NPIX	
CSTRETCH	CSTRETCH	CSTRETCH	
CWHITE	CWHITE	CWHITE	
		DARKCORR	
		DARKFILE	
	DATE		
	DATE-HP		
DATE-OBS	DATE-OBS	DATE-OBS	
	DEC		DEC
		DEC-OBJ	
EGAIN	EGAIN	EGAIN	
EXPOSURE	EXPOSURE	EXPOSURE	
EXPTIME	EXPTIME	EXPTIME	EXPTIME
			EXTEND
FILTER	FILTER	FILTER	FILTER
		FLATCORR	
		FLATFILE	
FLIPSTAT	FLIPSTAT	FLIPSTAT	
FOCALLEN	FOCALLEN	FOCALLEN	FOCALLEN
IMAGETYP	IMAGETYP	IMAGETYP	
INSTRUME	INSTRUME	INSTRUME	
JD	JD	JD	
JD-HELIO	JD-HELIO	JD-HELIO	
	LAT-OBS	LAT-OBS	

iTelescope raw	iTelescope calibrated	AP calibrated	M101 premium
	LONG-OBS	LON-OBS	
NAXIS	NAXIS	NAXIS	NAXIS
NAXIS1	NAXIS1	NAXIS1	NAXIS1
NAXIS2	NAXIS2	NAXIS2	NAXIS2
NOTES	NOTES	NOTES	
	OBJCTDEC		OBJCTDEC
	OBJCTRA		OBJCTRA
OBJECT	OBJECT	OBJECT	
		OBJNAME	
OBSERVER	OBSERVAT	OBSERVAT	
	OBSERVER	OBSERVER	
			OBSTARGET
			OWNER
PEDESTAL	PEDESTAL		
	PIERSIDE		
	RA		RA
	RADECSYS		
		RA-OBJ	
READOUTM	READOUTM	READOUTM	
ROWORDER	ROWORDER	ROWORDER	
	RollAngle		
SBSTDVER	SBSTDVER	SBSTDVER	
SET-TEMP	SET-TEMP	SET-TEMP	
SIMPLE	SIMPLE	SIMPLE	SIMPLE
SITELAT	SITELAT	SITELAT	
SITELONG	SITELONG	SITELONG	
			SRCLINK
	ST		
SWCREATE	SWCREATE	SWCREATE	
SWOWNER	SWMODIFY	SWOWNER	
SWSERIAL	SWOWNER	SWSERIAL	
	SWSERIAL		
TELESCOP	TELESCOP	TELESCOP	
	TIME-OBS		
	TIMESYS		
TRAKTIME	TRAKTIME	TRAKTIME	
	USERNAME		
	UT		
XBINNING	XBINNING	XBINNING	
XORGSUBF	XORGSUBF	XORGSUBF	
XPIXSZ	XPIXSZ	XPIXSZ	
YBINNING	YBINNING	YBINNING	
YORGSUBF	YORGSUBF	YORGSUBF	
YPIXSZ	YPIXSZ	YPIXSZ	
	iTelescope		
	iTelescopePlateScaleH		
	iTelescopePlateScaleV		

2.2 Astrometry (aka Navigation)

Combining multiple images of the same source in the same filter requires that each image have a valid World Coordinate System solution, based on an astrometric solution to the stars in the field of view. The **AstroPhotography** package also refers to this process as image navigation.

The process consists of finding the pixel locations brightest N stars in each image, then calling **Astrometry.net** to compute a WCS solution based on those star locations, and creating a new “navigated” fits file that combines the calibrated file and the WCS solution. By default 2-dimensional Gaussian profiles are also fitted to a representative number of stars per image to assess the Full Width at Half Maximum of the star images (in pixels), the results of which are shown graphically in a plot and numerically in a YAML file describing the stars found and fitted per image. At the end of the process a summary of the image (star) statistics is generated in CSV format that also incorporates the plate scale (arcseconds per pixel) found in the astrometric solution, which allows a human to look for outliers such as images where the seeing or tracking was especially bad.

In terms of pseudo-code

```
for cal-img in calibrated-images:
    ap_find_stars cal-img --> source-list source-plot fwhm-plot quality-report.yml ds9-sources.reg
    ap_astrometry cal-img source-list --> navigated-img
ap_quality_summary (all quality-report.yml files) --> quality-summary.csv
```

Using the (current but deprecated) bash script `navigate_all.sh` the output for a simple calibrated file is as follows:

```
Processing ./20210306/cal-T05-davestrickland-M81-20210306-235058-B-BIN1-W-300-001.fits at Sun Feb 11 04:17:23 PM EST 2024
Cleaning all files...
removed 'SourceLists/srclist_T05-davestrickland-M81-20210306-235058-B-BIN1-W-300-001.fits'
removed 'SourceLists/implot_T05-davestrickland-M81-20210306-235058-B-BIN1-W-300-001.png'
removed 'SourceLists/ds9_T05-davestrickland-M81-20210306-235058-B-BIN1-W-300-001.reg'
removed 'MetaData/qual_T05-davestrickland-M81-20210306-235058-B-BIN1-W-300-001.yaml'
removed 'MetaData/srclog_T05-davestrickland-M81-20210306-235058-B-BIN1-W-300-001.log'
```

```
removed 'NavigatedImages/nav_T05-davestrickland-M81-20210306-235058-B-BIN1-W-300-001.fits'
```

```
Performing star detection on ./20210306/cal-T05-davestrickland-M81-20210306-235058-B-BIN1-W-300-001.fits
```

```
About the run the following command python3 /home/dks/git/AstroPhotography/AstroPhotography/scripts/ap_find_stars.py -l DEBUG -m 200 ./20210306/cal-T05-davestrickland-M81-20210306-235058-B-BIN1-W-300-001.fits
```

```
Star detection completed successfully at Sun Feb 11 04:17:28 PM EST 2024
```

```
Log file written to MetaData/srclog_T05-davestrickland-M81-20210306-235058-B-BIN1-W-300-001.log
```

```
Performing astrometry on ./20210306/cal-T05-davestrickland-M81-20210306-235058-B-BIN1-W-300-001.fits
```

```
About the run the following command: python3 /home/dks/git/AstroPhotography/AstroPhotography/scripts/ap_astrometry.py -l DEBUG ./20210306/cal-T05-davestrickland-M81-20210306-235058-B-BIN1-W-300-001.fits
```

```
Astrometry completed successfully at Sun Feb 11 04:17:38 PM EST 2024
```

2.2.1 Navigating a single image

Let's try detecting the stars in a single premium image, the positions of which can then be used with Astrometry.net to get an astrometric solution.

```
0]: # parameters used with source detection
p_loglevel = "DEBUG" # Loglevel
p_fitsimg = str(flist[0]) # Input fits image
p_extnum = 0 # Extension number for data in input fits
p_fitstbl = p_fitsimg.replace(
    "Calibrated-iTelescope", "srclist"
) # Output fits table of srcs
p_search_fwhm = 3.0 # initial estimate at star fwhm in pixels
p_search_nsigma = 7.0 # min detection sigma above background
p_detector_bitdepth = (
    16 # detector bitdepth (16 for most CCDs, ?? for CMOS, ?? for camera)
)
p_sat_frac = 0.8 # Fraction of full well at which we assume star saturated
p_max_sources = 200 # Max number of sources to output or None
p_nosatmask = True # Keep possibly saturated stars.
p_regfile = p_fitsimg.replace("Calibrated-iTelescope", "ds9").replace(
    "fits", "reg"
) # Name of ds9 region file or None
p_plotfile = p_fitsimg.replace("Calibrated-iTelescope", "implot").replace(
    "fits", "png"
) # None or name of output png image w sources.
p_qual_rprt = p_fitsimg.replace("Calibrated-iTelescope", "qual").replace(
    "fits", "yaml"
) # None or name of output ascii report
p_quiet = False # Quiet mode suppresses STDOUT list printing
p_fwhm_plot = p_fitsimg.replace("Calibrated-iTelescope", "fwhmplot").replace(
    "fits", "png"
) # PNG/JPG plot of PSF fits

# parameters associated with astrometric solution, if not already specified above.
# Astrometry.Net API key will be dealt with elsewhere
p_inping = p_fitsimg
p_srclist = p_fitstbl
p_src_extname = "AP_XYPOS"
p_outimg = p_fitsimg.replace("Calibrated-iTelescope", "navigated")
p_use_sip = False
p_user_scale = None
p_scale_err_ratio = None

# Regenerate files even if they exist, set p_clean = True
p_clean = False
```

```
1]: def does_file_exists(filename, verbose=False):
    """
    Returns True if the file name or path exists, false otherwise
    """
    if verbose and not Path(filename).exists():
        print(f"Cannot find {filename}. Not a valid path or file.")
        return False
    else:
        print(f"Found {filename}.")
    return True
```

```
2]: def find_stars_wrapper(
    p_loglevel,
    p_fitsimg,
    p_fitstbl,
    p_extnum=0,
    p_search_fwhm=3.0,
    p_search_nsigma=7.0,
    p_detector_bitdepth=16,
    p_sat_frac=0.8,
```

```

p_max_sources=200,
p_nosatmask=True,
p_plotfile=None,
p_quiet=False,
p_fwhm_plot=None,
p_qual_rprt=None,
p_regfile=None,
):
    """
    Wrapper for finding stars using ApFindStars within a python session,
    instead of using the command line ap_find_stars.py script.
    """

    # Perform initial source detection using default parameters.
    find_stars = ap.ApFindStars(
        p_fitsimg,
        p_extnum,
        p_search_fwhm,
        p_search_nsigma,
        p_detector_bitdepth,
        p_max_sources,
        p_nosatmask,
        p_sat_frac,
        p_loglevel,
        p_plotfile,
        p_quiet,
    )

    # Measure 2-Gaussian FWHM for select stars, get average over x and y
    (p_new_fwhm, p_madstd_fwhm, p_npts) = find_stars.measure_fwhm(p_fwhm_plot, "both")

    # Refine source detection
    find_stars.source_search(p_new_fwhm, p_search_nsigma)

    # Re-run photometry
    find_stars.aperture_photometry()

    # As the source searching and photometry was redone, we should redo
    # the plotting.
    if p_plotfile is not None:
        find_stars.plot_image(p_plotfile)

    # Write optional quality report
    if p_qual_rprt is not None:
        find_stars.write_quality_report(p_qual_rprt)

    # Write optional ds9 format region file
    if p_regfile is not None:
        find_stars.write_ds9_region_file(p_regfile)

    # Write final sourcelist with photometry.
    find_stars.write_source_list(p_fitstbl)

    return

```

```

3]: find_stars_wrapper(
    p_loglevel,
    p_fitsimg,
    p_fitstbl,
    p_extnum,
    p_search_fwhm,
    p_search_nsigma,
    p_detector_bitdepth,
    p_sat_frac,
    p_max_sources,
    p_nosatmask,
    p_plotfile,
    p_quiet,
    p_fwhm_plot,
    p_qual_rprt,
    p_regfile,
)

```

2024-11-26 21:59:10,454 | ApFindStars | DEBUG | 2-D BITPIX=-32 image with 3056 columns, 3056 rows
2024-11-26 21:59:10,548 | ApFindStars | DEBUG | Raw data statistics are
min=0.00, max=67450.95, median=491.96
2024-11-26 21:59:11,399 | ApFindStars | DEBUG | Sigma clipped image stats:
mean=492.291, median=491.025, stddev=28.186
2024-11-26 21:59:13,964 | ApFindStars | DEBUG | Source-masked image stats:
mean=489.139, median=488.849, stddev=25.656
2024-11-26 21:59:13,965 | ApFindStars | DEBUG | Checking for possibly saturated stars or regions.
2024-11-26 21:59:13,965 | ApFindStars | DEBUG | Looking for possibly saturated regions above 52428 ADU separated by {boxsize} pixels.
2024-11-26 21:59:14,149 | ApFindStars | DEBUG | Retaining 17 possibly saturated stars in source searching and photometry.
2024-11-26 21:59:14,149 | ApFindStars | DEBUG | Running DAOStarFinder with FWHM=3.00 pixels, threshold=7.0 BG sigma.

Saturated position table:
x_peak y_peak peak_value

2168	134	58007.41
1861	192	67450.945
542	301	66776.38
1733	305	66585.7
124	785	66806.375
925	837	65296.09
978	1256	55520.92
107	1333	66617.82
116	1334	66440.46
3034	1826	65005.867
2118	2062	65066.12
244	2213	65891.75
3026	2245	66356.28
146	2641	67146.664
2034	2658	64963.445
2078	2846	65638.266
1860	2936	54830.844

2024-11-26 21:59:14,385 | ApFindStars | INFO | Initial source list using FWHM=3.000 pixels, threshold=7.00 x BG stddev, found 265 sources.
2024-11-26 21:59:14,386 | ApFindStars | DEBUG | Sources with pixels exceding 52428 ADU will be flagged as possibly saturated.
2024-11-26 21:59:14,386 | ApFindStars | DEBUG | There are 20 possibly saturated stars in the output source list.
2024-11-26 21:59:14,387 | ApFindStars | DEBUG | Radius of circular aperture photometry is 6 pixels.
2024-11-26 21:59:14,387 | ApFindStars | DEBUG | Local BG estimation in annulus outer radius 9 pixels, inner radius 6 pixels.
2024-11-26 21:59:14,431 | ApFindStars | DEBUG | Image exposure time [seconds]: 180.00
/home/dks/git/AstroPhotography/AstroPhotography/core/ApFindStars.py:527:
RuntimeWarning: invalid value encountered in log10
phot_table["magnitude"] = -2.5 * np.log10(phot_table["adu_per_sec"])
2024-11-26 21:59:14,432 | ApFindStars | DEBUG | Brightest source (idx=0) generates 27429.19 ADU/s.
2024-11-26 21:59:14,433 | ApFindStars | DEBUG | Median source (idx=132) generates 108.27 ADU/s.
2024-11-26 21:59:14,433 | ApFindStars | DEBUG | Faintest source (idx=264) generates -85.00 ADU/s.
2024-11-26 21:59:14,433 | ApFindStars | DEBUG | Selecting 200 sources out of 265 to write to the output source list.
2024-11-26 21:59:14,433 | ApFindStars | DEBUG | Radius of circular aperture photometry is 6 pixels.
2024-11-26 21:59:14,433 | ApFindStars | DEBUG | Local BG estimation in annulus outer radius 9 pixels, inner radius 6 pixels.

id	xcentroid	ycentroid	sharpness	roundness1	roundness2	npix	sky	peak	flux
mag	psbl_sat								
1	1667.02	19.15	0.3604	0.0048	0.0677	25	0	442.41	
1.7313	-0.5959	False							
2	2989.30	36.39	0.4067	-0.2695	0.1677	25	0	696.49	
2.5255	-1.0059	False							
3	1909.27	67.00	0.4246	-0.1126	0.1653	25	0	403.46	
1.4061	-0.3700	False							
4	1554.09	73.38	0.4523	0.0858	0.5413	25	0	1799.92	


```
3.0552 -1.2126 False
 5 153.73 77.81 0.4811 -0.1907 -0.1428 25 0 491.31
1.4794 -0.4252 False
 6 2385.73 102.48 0.4248 -0.4378 0.1723 25 0 409.73
1.4355 -0.3925 False
 7 816.19 111.03 0.9502 -0.0965 -0.0142 25 0 1002.37
4.0108 -1.5081 False
 8 2167.66 134.02 0.4556 -0.1268 0.1220 25 0 57518.56
198.8871 -5.7465 True
 9 2186.56 141.70 0.5121 0.1010 0.0511 25 0 470.86
1.6511 -0.5444 False
... ... ... ... ...
... ... ... ... ...
257 894.75 2827.24 0.4656 -0.4934 0.0031 25 0 2972.84
10.1728 -2.5186 False
258 2845.95 2838.94 0.4426 -0.8304 -0.3469 25 0 243.17
1.2947 -0.2804 False
259 845.68 2844.45 0.4612 -0.5107 0.0878 25 0 2697.90
9.2069 -2.4103 False
260 1860.15 2935.60 0.4588 -0.3651 0.1043 25 0 54342.00
193.8598 -5.7187 True
261 292.42 2947.25 0.4059 -0.4113 -0.0727 25 0 1297.22
4.5110 -1.6357 False
262 1073.03 2948.38 0.4814 -0.3479 0.1203 25 0 3171.04
11.0785 -2.6112 False
263 64.29 2973.19 0.4433 -0.5090 -0.1063 25 0 10470.56
34.1943 -3.8349 False
264 2570.11 2985.47 0.4674 -0.2362 0.1745 25 0 794.87
2.9769 -1.1844 False
265 317.29 3041.85 0.4893 -0.5151 -0.1560 25 0 323.19
1.0895 -0.0930 False
Length = 265 rows
id xcenter ycenter aperture_sum peak_adu psbl_sat bgmed_per_pix adu_per_sec
magnitude
pix pix
-----
12 1856.53 195.21 4937254.6531 66219.60 True 6182.96 27429.1925
-11.0955
14 1858.48 197.73 4664030.2303 65806.47 True 5124.15 25911.2791
-11.0337
13 1861.00 194.82 4594638.3099 62863.52 True 4832.10 25525.7684
-11.0174
111 116.29 1333.13 4108741.3952 65569.82 True 11373.42 22826.3411
-10.8961
245 145.75 2640.01 3613403.4601 65908.50 True 2780.66 20074.4637
-10.7566
244 144.61 2639.29 3609494.5672 66132.87 True 2626.85 20052.7476
-10.7554
213 245.40 2213.62 2453175.0246 64892.54 True 1583.71 13628.7501
-10.3361
212 246.18 2213.10 2447453.7011 65165.94 True 1587.34 13596.9650
-10.3336
... ... ... ... ...
...
54 860.43 727.51 1041.4839 224.51 False 486.81 5.7860
-1.9060
23 382.08 378.11 1022.4234 420.09 False 481.86 5.6801
-1.8859
87 1264.06 1095.35 941.1482 271.79 False 491.88 5.2286
-1.7960
82 2231.29 1053.18 684.5490 325.91 False 509.57 3.8030
-1.4503
33 469.76 509.77 556.6896 217.78 False 484.89 3.0927
-1.2259
215 397.07 2225.92 496.9866 323.37 False 483.75 2.7610
-1.1027
73 1015.94 965.08 220.4001 230.63 False 487.14 1.2244
-0.2198
116 2423.09 1366.05 -115.2431 245.92 False 512.31 -0.6402
nan
101 1.00 1191.17 -15299.1878 462.39 False 486.40 -84.9955
nan
Length = 265 rows
2024-11-26 21:59:15,410 | ApFindStars | DEBUG | Plotted asinh-stretched bitmap
```

of image and sources to implot-M101_Supernova_2023ixf-180s-Lum-1.png

2024-11-26 21:59:15,413 | ApMeasureStars | INFO | Measuring the source extent (Gaussian FWHM) from input list of 200 sources.

2024-11-26 21:59:15,413 | ApMeasureStars | DEBUG | Up to 5 stars per sub-region will be fitted, excluding the brightest 0 stars.

2024-11-26 21:59:15,413 | ApMeasureStars | DEBUG | Count based weighting factors will be used in fitting?: True

2024-11-26 21:59:15,414 | ApMeasureStars | DEBUG | Will the background level be fitted for? : True

2024-11-26 21:59:15,415 | ApMeasureStars | DEBUG | Size of input trimmed source list (filtered): 180

2024-11-26 21:59:15,415 | ApMeasureStars | DEBUG | Size of full source list used for neighbor removal: 265

2024-11-26 21:59:15,415 | ApMeasureStars | DEBUG | Adopting a fit box width of 18 pixels, edge exclusion of 10 pixels.

2024-11-26 21:59:15,415 | ApMeasureStars | DEBUG | Preparing to trim the input source list of stars with neighbors within 18 pixels.

2024-11-26 21:59:15,416 | ApMeasureStars | DEBUG | Constructed a kdtree with 265 data points.

2024-11-26 21:59:15,419 | ApMeasureStars | DEBUG | Nearest neighbor filtering removed 13 stars from consideration.

2024-11-26 21:59:15,419 | ApMeasureStars | DEBUG | Selecting candidate stars for fitting out of 167 stars in 3056 row x 3056 column image.

2024-11-26 21:59:15,419 | ApMeasureStars | DEBUG | Center has radius 764.0 pixels centered on x=1528.0, y=1528.0

2024-11-26 21:59:15,420 | ApMeasureStars | DEBUG | There are 166 stars more than 10 pixels away from the edge of the detector.

2024-11-26 21:59:15,420 | ApMeasureStars | DEBUG | In region CN found 55 candidates for fitting:

2024-11-26 21:59:15,422 | ApMeasureStars | DEBUG | In region TL found 25 candidates for fitting:

2024-11-26 21:59:15,425 | ApMeasureStars | DEBUG | In region TR found 29 candidates for fitting:

2024-11-26 21:59:15,426 | ApMeasureStars | DEBUG | In region BR found 31 candidates for fitting:

2024-11-26 21:59:15,428 | ApMeasureStars | DEBUG | In region BL found 26 candidates for fitting:

2024-11-26 21:59:15,430 | ApMeasureStars | DEBUG | In 25x18x18 pixel array min=417.57, max=42571.63

2024-11-26 21:59:15,431 | ApMeasureStars | INFO | Starting to fit Const2D+Gaussian2D model to 25 star cutouts.

2024-11-26 21:59:15,432 | ApMeasureStars | DEBUG | Fitting star #0 (id=161) at fixed xpos=8.66, ypos=9.07

2024-11-26 21:59:15,442 | ApMeasureStars | DEBUG | Fitting star #1 (id=122) at fixed xpos=8.86, ypos=9.08

2024-11-26 21:59:15,450 | ApMeasureStars | DEBUG | Fitting star #2 (id=109) at fixed xpos=8.95, ypos=8.87

2024-11-26 21:59:15,463 | ApMeasureStars | DEBUG | Fitting star #3 (id=186) at fixed xpos=8.85, ypos=8.62

2024-11-26 21:59:15,475 | ApMeasureStars | DEBUG | Fitting star #4 (id=176) at fixed xpos=8.84, ypos=8.93

2024-11-26 21:59:15,483 | ApMeasureStars | DEBUG | Fitting star #5 (id=263) at fixed xpos=9.29, ypos=9.19

2024-11-26 21:59:15,494 | ApMeasureStars | DEBUG | Fitting star #6 (id=239) at fixed xpos=8.55, ypos=9.42

2024-11-26 21:59:15,503 | ApMeasureStars | DEBUG | Fitting star #7 (id=216) at fixed xpos=8.91, ypos=8.90

2024-11-26 21:59:15,514 | ApMeasureStars | DEBUG | Fitting star #8 (id=250) at fixed xpos=9.31, ypos=9.28

2024-11-26 21:59:15,526 | ApMeasureStars | DEBUG | Fitting star #9 (id=246) at fixed xpos=8.70, ypos=8.73

2024-11-26 21:59:15,537 | ApMeasureStars | DEBUG | Fitting star #10 (id=229) at fixed xpos=8.91, ypos=8.92

2024-11-26 21:59:15,546 | ApMeasureStars | DEBUG | Fitting star #11 (id=227) at fixed xpos=9.05, ypos=9.49

2024-11-26 21:59:15,554 | ApMeasureStars | DEBUG | Fitting star #12 (id=163) at fixed xpos=8.92, ypos=8.56

2024-11-26 21:59:15,563 | ApMeasureStars | DEBUG | Fitting star #13 (id=243) at fixed xpos=8.99, ypos=9.44

2024-11-26 21:59:15,571 | ApMeasureStars | DEBUG | Fitting star #14 (id=207) at fixed xpos=8.83, ypos=9.32

2024-11-26 21:59:15,580 | ApMeasureStars | DEBUG | Fitting star #15 (id=49) at fixed xpos=9.45, ypos=8.67

2024-11-26 21:59:15,589 | ApMeasureStars | DEBUG | Fitting star #16 (id=43) at fixed xpos=9.49, ypos=8.89

2024-11-26 21:59:15,597 | ApMeasureStars | DEBUG | Fitting star #17 (id=53) at

fixed xpos=9.15, ypos=9.35
2024-11-26 21:59:15,606 | ApMeasureStars | DEBUG | Fitting star #18 (id=35) at
fixed xpos=9.09, ypos=8.99
2024-11-26 21:59:15,614 | ApMeasureStars | DEBUG | Fitting star #19 (id=127) at
fixed xpos=9.07, ypos=9.40

Input table supplied to ApMeasureStars after saturated star filtering:

id	xcenter	ycenter	peak_adu	bgmed_per_pix	magnitude
pix		pix			
36	988.34	568.25	42082.78	779.98	-9.1844
49	1717.45	696.67	34665.86	713.27	-8.8793
161	1790.66	1721.07	34437.17	815.72	-8.7501
43	2376.49	635.89	26131.05	649.73	-8.5449
53	1958.15	719.35	22915.31	638.96	-8.3771
229	2596.91	2435.92	24442.35	622.76	-8.3455
35	1742.09	544.99	19561.60	609.22	-8.1980
227	2144.05	2398.49	19322.73	608.14	-8.1614
...	...	...	...	...	...
69	467.93	915.06	2089.46	488.37	-4.1678
6	2385.73	102.48	409.73	488.93	-4.1673
220	777.05	2266.04	462.40	483.24	-4.1623
160	1262.14	1712.05	502.49	492.90	-4.1539
77	635.99	1000.31	411.78	491.44	-4.1469
247	1436.99	2644.52	475.95	495.08	-4.1415
9	2186.56	141.70	470.86	505.66	-4.1227
108	2805.60	1294.94	511.44	495.03	-4.1116
3	1909.27	67.00	403.46	506.01	-4.1100

Length = 180 rows

Initial sources with nearest neighbor distance

id	xcenter	ycenter	peak_adu	bgmed_per_pix	magnitude	nn_dist
pix		pix				
36	988.34	568.25	42082.78	779.98	-9.1844	71.98
49	1717.45	696.67	34665.86	713.27	-8.8793	87.15
161	1790.66	1721.07	34437.17	815.72	-8.7501	125.45
43	2376.49	635.89	26131.05	649.73	-8.5449	65.21
53	1958.15	719.35	22915.31	638.96	-8.3771	59.14
229	2596.91	2435.92	24442.35	622.76	-8.3455	115.43
35	1742.09	544.99	19561.60	609.22	-8.1980	153.67
227	2144.05	2398.49	19322.73	608.14	-8.1614	75.32
...	...	...	...	...	...	...
69	467.93	915.06	2089.46	488.37	-4.1678	147.91
6	2385.73	102.48	409.73	488.93	-4.1673	203.00
220	777.05	2266.04	462.40	483.24	-4.1623	29.49
160	1262.14	1712.05	502.49	492.90	-4.1539	31.49
77	635.99	1000.31	411.78	491.44	-4.1469	71.47
247	1436.99	2644.52	475.95	495.08	-4.1415	81.22
9	2186.56	141.70	470.86	505.66	-4.1227	20.40
108	2805.60	1294.94	511.44	495.03	-4.1116	70.90
3	1909.27	67.00	403.46	506.01	-4.1100	136.63

Length = 180 rows

id	xcenter	ycenter	peak_adu	bgmed_per_pix	magnitude	nn_dist	dx	dy
radius region								
pix		pix						
161	1790.66	1721.07	34437.17	815.72	-8.7501	125.45	262.66	193.07
325.98	CN							
122	1804.86	1387.08	17871.50	654.15	-8.0227	53.17	276.86	-140.92
310.66	CN							
109	1196.95	1314.87	11376.10	564.54	-7.6046	132.69	-331.05	-213.13
393.72	CN							
186	1683.85	1972.62	6178.29	579.09	-6.8751	100.60	155.85	444.62
471.14	CN							
176	868.84	1873.93	5657.47	529.22	-6.8357	196.77	-659.16	345.93
744.42	CN							
id	xcenter	ycenter	peak_adu	bgmed_per_pix	magnitude	nn_dist	dx	dy
radius region								
pix		pix						
pix								

263	64.29	2973.19	10470.56	564.72	-7.6378	226.73	-1463.71	1445.19
2056.95	TL							
239	1432.55	2563.42	8489.51	552.94	-7.3538	79.54	-95.45	1035.42
1039.81	TL							
216	308.91	2229.90	5232.64	530.88	-6.8197	64.94	-1219.09	701.90
1406.71	TL							
250	296.31	2694.28	4009.30	519.19	-6.5934	160.04	-1231.69	1166.28
1696.25	TL							
246	978.70	2641.73	4047.20	511.50	-6.4761	184.53	-549.30	1113.73
1241.82	TL							

id	xcenter	ycenter	peak_adu	bgmed_per_pix	magnitude	nn_dist	dx	dy
radius	region							
	pix	pix					pix	pix

229	2596.91	2435.92	24442.35	622.76	-8.3455	115.43	1068.91	907.92
1402.46	TR							
227	2144.05	2398.49	19322.73	608.14	-8.1614	75.32	616.05	870.49
1066.43	TR							
163	2408.92	1724.56	15033.04	617.66	-7.8377	29.87	880.92	196.56
902.59	TR							
243	1650.99	2628.44	14108.12	578.42	-7.8341	55.65	122.99	1100.44
1107.29	TR							
207	2344.83	2160.32	12815.53	574.06	-7.6723	140.08	816.83	632.32
1032.98	TR							

id	xcenter	ycenter	peak_adu	bgmed_per_pix	magnitude	nn_dist	dx	dy
radius	region							
	pix	pix					pix	pix

49	1717.45	696.67	34665.86	713.27	-8.8793	87.15	189.45	-831.33
852.64	BR							
43	2376.49	635.89	26131.05	649.73	-8.5449	65.21	848.49	-892.11
1231.18	BR							
53	1958.15	719.35	22915.31	638.96	-8.3771	59.14	430.15	-808.65
915.94	BR							
35	1742.09	544.99	19561.60	609.22	-8.1980	153.67	214.09	-983.01
1006.06	BR							
127	2343.07	1451.40	12210.71	608.41	-7.6697	45.28	815.07	-76.60
818.67	BR							

id	xcenter	ycenter	peak_adu	bgmed_per_pix	magnitude	nn_dist	dx	dy
radius	region							
	pix	pix					pix	pix

36	988.34	568.25	42082.78	779.98	-9.1844	71.98	-539.66	-959.75
1101.07	BL							
50	894.33	702.21	11603.38	577.52	-7.7666	42.30	-633.67	-825.79
1040.90	BL							
48	347.88	679.01	6457.10	546.23	-7.1341	110.89	-1180.12	-848.99
1453.78	BL							
22	761.35	365.20	5483.79	534.47	-6.9577	35.89	-766.65	-1162.80
1392.79	BL							
58	79.36	801.57	4874.97	536.57	-6.9187	47.73	-1448.64	-726.43
1620.58	BL							

Fit candidates with data extraction box indices:

id	xcenter	ycenter	peak_adu	bgmed_per_pix	magnitude	nn_dist	...	radius	region
xmin	xmax	ymin	ymax						
	pix	pix					...	pix	
pix	pix	pix	pix						

161	1790.66	1721.07	34437.17	815.72	-8.7501	125.45	...	325.98	CN
1782	1800	1712	1730						
122	1804.86	1387.08	17871.50	654.15	-8.0227	53.17	...	310.66	CN
1796	1814	1378	1396						
109	1196.95	1314.87	11376.10	564.54	-7.6046	132.69	...	393.72	CN
1188	1206	1306	1324						
186	1683.85	1972.62	6178.29	579.09	-6.8751	100.60	...	471.14	CN

1675	1693	1964	1982						
176	868.84	1873.93	5657.47	529.22	-6.8357	196.77	...	744.42	CN
860	878	1865	1883						
263	64.29	2973.19	10470.56	564.72	-7.6378	226.73	...	2056.95	TL
55	73	2964	2982						
239	1432.55	2563.42	8489.51	552.94	-7.3538	79.54	...	1039.81	TL
1424	1442	2554	2572						
216	308.91	2229.90	5232.64	530.88	-6.8197	64.94	...	1406.71	TL
300	318	2221	2239						
...	...	...	...	...	...	...	...	...	...
...	...	...	...						
43	2376.49	635.89	26131.05	649.73	-8.5449	65.21	...	1231.18	BR
2367	2385	627	645						
53	1958.15	719.35	22915.31	638.96	-8.3771	59.14	...	915.94	BR
1949	1967	710	728						
35	1742.09	544.99	19561.60	609.22	-8.1980	153.67	...	1006.06	BR
1733	1751	536	554						
127	2343.07	1451.40	12210.71	608.41	-7.6697	45.28	...	818.67	BR
2334	2352	1442	1460						
36	988.34	568.25	42082.78	779.98	-9.1844	71.98	...	1101.07	BL
979	997	559	577						
50	894.33	702.21	11603.38	577.52	-7.7666	42.30	...	1040.90	BL
885	903	693	711						
48	347.88	679.01	6457.10	546.23	-7.1341	110.89	...	1453.78	BL
339	357	670	688						
22	761.35	365.20	5483.79	534.47	-6.9577	35.89	...	1392.79	BL
752	770	356	374						
58	79.36	801.57	4874.97	536.57	-6.9187	47.73	...	1620.58	BL
70	88	793	811						

Length = 25 rows

```

2024-11-26 21:59:15,623 | ApMeasureStars | DEBUG | Fitting star #20 (id=36) at
fixed xpos=9.34, ypos=9.25
2024-11-26 21:59:15,634 | ApMeasureStars | DEBUG | Fitting star #21 (id=50) at
fixed xpos=9.33, ypos=9.21
2024-11-26 21:59:15,643 | ApMeasureStars | DEBUG | Fitting star #22 (id=48) at
fixed xpos=8.88, ypos=9.01
2024-11-26 21:59:15,652 | ApMeasureStars | DEBUG | Fitting star #23 (id=22) at
fixed xpos=9.35, ypos=9.20
2024-11-26 21:59:15,663 | ApMeasureStars | DEBUG | Fitting star #24 (id=58) at
fixed xpos=9.36, ypos=8.57
2024-11-26 21:59:15,675 | ApMeasureStars | DEBUG | Estimating median FWHM
(direction=x) using 25 FWHM measurements (25 OK fits before clipping).
2024-11-26 21:59:15,676 | ApMeasureStars | DEBUG | Measured FWHM=4.070 +/- 0.156
pixels in direction x
2024-11-26 21:59:15,676 | ApMeasureStars | DEBUG | Estimating median FWHM
(direction=y) using 24 FWHM measurements (25 OK fits before clipping).
2024-11-26 21:59:15,677 | ApMeasureStars | DEBUG | Measured FWHM=4.341 +/- 0.101
pixels in direction y
2024-11-26 21:59:15,677 | ApMeasureStars | INFO | Source average axis ratio =
1.066 +/- 0.048, overall sources are circular.
2024-11-26 21:59:15,677 | ApMeasureStars | DEBUG | Fitting completed in 0.246 s
(average 9.8 ms/star)
2024-11-26 21:59:15,766 | ApMeasureStars | DEBUG | Estimating median FWHM
(direction=both) using 49 FWHM measurements (50 OK fits before clipping).
2024-11-26 21:59:15,766 | ApMeasureStars | DEBUG | Measured FWHM=4.312 +/- 0.343
pixels in direction both
2024-11-26 21:59:15,766 | ApMeasureStars | DEBUG | Estimating median FWHM
(direction=x) using 25 FWHM measurements (25 OK fits before clipping).
2024-11-26 21:59:15,767 | ApMeasureStars | DEBUG | Measured FWHM=4.070 +/- 0.156
pixels in direction x
2024-11-26 21:59:15,768 | ApMeasureStars | DEBUG | Estimating median FWHM
(direction=y) using 24 FWHM measurements (25 OK fits before clipping).
2024-11-26 21:59:15,768 | ApMeasureStars | DEBUG | Measured FWHM=4.341 +/- 0.101
pixels in direction y

```

Fit results:

id	xcenter	ycenter	peak_adu	bgmed_per_pix	magnitude	nn_dist	...	theta_err	axrat
axrat_err	circular	fit_ok	rchisq						
	pix	pix					...		
161	1790.66	1721.07	34437.17	815.72	-8.7501	125.45	...	0.015	1.06
0.00	True	True	192.65						
122	1804.86	1387.08	17871.50	654.15	-8.0227	53.17	...	0.018	1.08
0.00	True	True	44.52						

109	1196.95	1314.87	11376.10	564.54	-7.6046	132.69	...	0.020	1.09
0.00	True	True	13.82						
186	1683.85	1972.62	6178.29	579.09	-6.8751	100.60	...	0.031	1.09
0.01	True	True	11.46						
176	868.84	1873.93	5657.47	529.22	-6.8357	196.77	...	0.025	1.11
0.01	True	True	6.81						
263	64.29	2973.19	10470.56	564.72	-7.6378	226.73	...	0.011	1.16
0.00	False	True	17.56						
239	1432.55	2563.42	8489.51	552.94	-7.3538	79.54	...	0.018	1.14
0.01	False	True	119.05						
216	308.91	2229.90	5232.64	530.88	-6.8197	64.94	...	0.019	1.15
0.01	False	True	5.49						
...	...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...	...
43	2376.49	635.89	26131.05	649.73	-8.5449	65.21	...	0.056	1.02
0.00	True	True	252.64						
53	1958.15	719.35	22915.31	638.96	-8.3771	59.14	...	0.017	1.07
0.00	True	True	61.57						
35	1742.09	544.99	19561.60	609.22	-8.1980	153.67	...	0.023	1.05
0.00	True	True	33.42						
127	2343.07	1451.40	12210.71	608.41	-7.6697	45.28	...	0.022	1.08
0.00	True	True	45.65						
36	988.34	568.25	42082.78	779.98	-9.1844	71.98	...	0.012	1.06
0.00	True	True	99.73						
50	894.33	702.21	11603.38	577.52	-7.7666	42.30	...	0.022	1.07
0.00	True	True	25.82						
48	347.88	679.01	6457.10	546.23	-7.1341	110.89	...	0.024	1.10
0.01	True	True	11.05						
22	761.35	365.20	5483.79	534.47	-6.9577	35.89	...	0.044	1.06
0.01	True	True	9.80						
58	79.36	801.57	4874.97	536.57	-6.9187	47.73	...	0.024	1.13
0.01	False	True	54.83						

Length = 25 rows

```

2024-11-26 21:59:16,989 | ApMeasureStars | INFO | Plotted star FWHM fit cut-outs
to fwhmplot-M101_Supernova_2023ixf-180s-Lum-1.png
2024-11-26 21:59:16,990 | ApMeasureStars | DEBUG | Estimating median FWHM
(direction=both) using 49 FWHM measurements (50 OK fits before clipping).
2024-11-26 21:59:16,990 | ApMeasureStars | DEBUG | Measured FWHM=4.312 +/- 0.343
pixels in direction both
2024-11-26 21:59:16,990 | ApMeasureStars | DEBUG | Estimating median FWHM
(direction=x) using 25 FWHM measurements (25 OK fits before clipping).
2024-11-26 21:59:16,991 | ApMeasureStars | DEBUG | Measured FWHM=4.070 +/- 0.156
pixels in direction x
2024-11-26 21:59:16,991 | ApMeasureStars | DEBUG | Estimating median FWHM
(direction=y) using 24 FWHM measurements (25 OK fits before clipping).
2024-11-26 21:59:16,992 | ApMeasureStars | DEBUG | Measured FWHM=4.341 +/- 0.101
pixels in direction y
2024-11-26 21:59:16,992 | ApFindStars | INFO | Median FWHM (over x and y) is
4.31 +/- 0.34 pixels using 49 data points
2024-11-26 21:59:16,992 | ApFindStars | DEBUG | Running DAOStarFinder with
FWHM=4.31 pixels, threshold=7.0 BG sigma.
2024-11-26 21:59:17,288 | ApFindStars | INFO | Initial source list using
FWHM=4.312 pixels, threshold=7.00 x BG stddev, found 306 sources.
2024-11-26 21:59:17,289 | ApFindStars | DEBUG | Sources with pixels exceding
52428 ADU will be flagged as possibly saturated.
2024-11-26 21:59:17,289 | ApFindStars | DEBUG | There are 13 possibly saturated
stars in the output source list.
2024-11-26 21:59:17,290 | ApFindStars | DEBUG | Radius of circular aperture
photometry is 6 pixels.
2024-11-26 21:59:17,290 | ApFindStars | DEBUG | Local BG estimation in annulus
outer radius 9 pixels, inner radius 6 pixels.
2024-11-26 21:59:17,341 | ApFindStars | DEBUG | Image exposure time [seconds]:
180.00
/home/dks/git/AstroPhotography/AstroPhotography/core/ApFindStars.py:527:
RuntimeWarning: invalid value encountered in log10
  phot_table["magnitude"] = -2.5 * np.log10(phot_table["adu_per_sec"])
2024-11-26 21:59:17,342 | ApFindStars | DEBUG | Brightest source (idx=0)
generates 23173.82 ADU/s.
2024-11-26 21:59:17,342 | ApFindStars | DEBUG | Median source (idx=153)
generates 74.44 ADU/s.
2024-11-26 21:59:17,342 | ApFindStars | DEBUG | Faintest source (idx=305)
generates -43.01 ADU/s.
2024-11-26 21:59:17,343 | ApFindStars | DEBUG | Selecting 200 sources out of 306
to write to the output source list.

```

2024-11-26 21:59:17,343 | ApFindStars | DEBUG | Radius of circular aperture photometry is 6 pixels.
2024-11-26 21:59:17,343 | ApFindStars | DEBUG | Local BG estimation in annulus outer radius 9 pixels, inner radius 6 pixels.

id	xcentroid	ycentroid	sharpness	roundness1	roundness2	npix	sky	peak	flux
mag	psbl_sat								

1	1667.04	19.13	0.3563	0.0236	0.0561	25	0	442.41	
2.3482	-0.9268	False							
2	2989.28	36.41	0.3944	-0.2044	0.1603	25	0	696.49	
3.3373	-1.3085	False							
3	908.27	65.77	0.2993	-0.0374	-0.0883	25	0	315.90	
1.5493	-0.4753	False							
4	1909.24	67.00	0.4013	-0.0568	0.1475	25	0	403.46	
1.9161	-0.7061	False							
5	1554.09	73.38	0.3961	0.0825	0.5344	25	0	1799.92	
4.6817	-1.6760	False							
6	153.72	77.83	0.4333	-0.0941	-0.1353	25	0	491.31	
2.0948	-0.8029	False							
7	2385.69	102.40	0.3458	-0.0067	0.0372	25	0	380.84	
1.9621	-0.7318	False							
8	816.16	111.02	0.9742	-0.0769	-0.0110	25	0	1002.37	
4.1598	-1.5477	False							
9	2167.68	134.01	0.4109	-0.0769	0.1167	25	0	57518.56	
287.5834	-6.1469	True							
...	...	...	...	...	...	...	...	...	
...	...	...							
298	1618.94	2924.55	0.2720	-0.0706	-0.2489	25	0	290.91	
1.0600	-0.0633	False							
299	1860.14	2935.61	0.4188	-0.2630	0.1029	25	0	54342.00	
273.4385	-6.0921	True							
300	292.39	2947.23	0.3786	-0.3089	-0.0726	25	0	1297.22	
6.4539	-2.0246	False							
301	1073.11	2948.45	0.4095	-0.2028	0.1153	25	0	3116.22	
15.3209	-2.9632	False							
302	1393.08	2963.16	0.2703	-0.4100	-0.1301	25	0	167.84	
1.0591	-0.0623	False							
303	64.27	2973.18	0.3972	-0.3965	-0.1055	25	0	10470.56	
50.6621	-4.2617	False							
304	2570.18	2985.55	0.4350	-0.0544	0.1287	25	0	785.11	
3.8947	-1.4762	False							
305	865.78	2990.44	0.8635	-0.2542	0.0521	25	0	245.39	
1.1538	-0.1553	False							
306	317.27	3041.85	0.4286	-0.4066	-0.1429	25	0	323.19	
1.6078	-0.5156	False							
Length = 306 rows									
id	xcenter	ycenter	aperture_sum	peak_adu	psbl_sat	bgmed_per_pix	adu_per_sec		
magnitude									
	pix								

124	116.23	1333.20	4171287.9500	65569.82	True	11336.23	23173.8219		
-10.9125									
280	144.84	2639.25	3627638.6756	65923.66	True	2733.48	20153.5482		
-10.7609									
278	146.95	2637.97	3606562.9534	66247.62	True	2553.70	20036.4609		
-10.7546									
237	245.46	2213.58	2454963.7331	64892.54	True	1580.07	13638.6874		
-10.3369									
236	246.23	2213.06	2445606.6724	64559.41	True	1587.34	13586.7037		
-10.3328									
219	2116.21	2061.74	2419072.3437	63780.82	True	1595.91	13439.2908		
-10.3209									
19	542.67	301.32	1996319.3592	65803.72	True	1278.48	11090.6631		
-10.1124									
66	924.95	838.16	1709664.2325	64458.88	True	1125.49	9498.1346		
-9.9441									
...	...	...	...	...	...	...	...	...	
...									
60	860.41	727.55	1037.1896	224.51	False	486.87	5.7622		
-1.9015									
165	787.73	1626.59	1018.3260	129.01	False	487.19	5.6574		
-1.8815									
96	1264.05	1095.30	953.7015	271.79	False	491.85	5.2983		

```

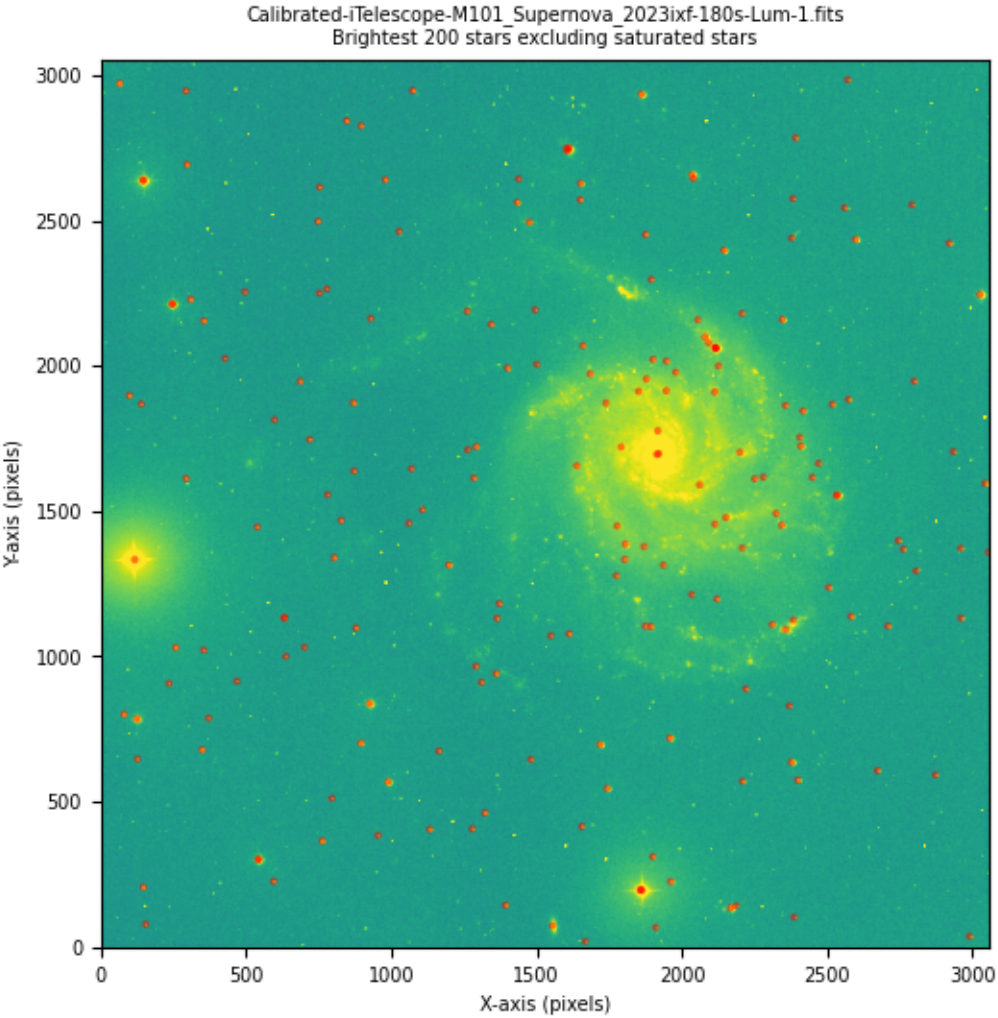
-1.8103
185 415.71 1755.45      893.1337   140.34   False      489.74      4.9619
-1.7391
249 1121.01 2317.36      707.3303   184.72   False      488.19      3.9296
-1.4859
 33 469.77  509.81      553.8340   217.78   False      484.89      3.0769
-1.2203
279 288.30 2637.83      32.4541   122.64   False      490.25      0.1803
1.8600
268 2649.61 2532.63     -407.4675    36.02   False      490.11     -2.2637
nan
234   2.22 2191.53     -7742.4820   210.22   False      459.38     -43.0138
nan
Length = 306 rows

2024-11-26 21:59:18,308 | ApFindStars | DEBUG | Plotted asinh-stretched bitmap
of image and sources to implot-M101_Supernova_2023ixf-180s-Lum-1.png
2024-11-26 21:59:18,309 | ApFindStars | INFO | Image is 3056 cols x 3056 rows
2024-11-26 21:59:18,309 | ApFindStars | DEBUG | FITS keywords found in image:
{'IMG_FILE': ('Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-1.fits',
'Name of image file searched for stars'), 'IMG_COLS': (3056, 'Number of columns
in input image'), 'IMG_ROWS': (3056, 'Number of rows in input image'),
'AP_NDET': (306, 'Number of sources detected in the image.'), 'AP_NPHOT': (200,
'Number of sources final photometry.'), 'AP_NFIT': (25, 'Number of sources used
in FWHM fitting.'), 'AP_NSIGM': (7.0, 'Source searching threshold (sigma above
background)'), 'APTDIA': (610.0, '[mm] Diameter of telescope aperture'), 'RA':
('14 03 12.40', 'Target right ascension'), 'DEC': ('+54 20 58.0', 'Target
declination'), 'FOCALLEN': (3962.0, '[mm] Stated telescope focal length'),
'FILTER': ('Lum', 'Filter used')}
2024-11-26 21:59:18,310 | ApFindStars | DEBUG | FITS keywords missing from
image: ['EXPOSURE', 'DATE-OBS', 'OBJECT', 'OBJNAME', 'TELESCOP', 'INSTRUME',
'CCD-TEMP', 'RA-OBJ', 'DEC-OBJ', 'XPIXSZ', 'YPIXSZ', 'EGAIN', 'LAT-OBS', 'LONG-
OBS', 'ALT-OBS', 'AIRMASS']
2024-11-26 21:59:18,317 | ApFindStars | INFO | Approximate coordinates:
ra=14.053444 hours, dec=54.349444 deg
2024-11-26 21:59:18,318 | ApFindStars | WARNING | Keyword OBJECT not found, not
written to quality report.
2024-11-26 21:59:18,318 | ApFindStars | WARNING | Keyword TELESCOP not found,
not written to quality report.
2024-11-26 21:59:18,318 | ApFindStars | WARNING | Keyword DATE-OBS not found,
not written to quality report.
2024-11-26 21:59:18,318 | ApFindStars | WARNING | Keyword EXPOSURE not found,
not written to quality report.
2024-11-26 21:59:18,319 | ApFindStars | WARNING | Keyword CCD-TEMP not found,
not written to quality report.
2024-11-26 21:59:18,319 | ApFindStars | WARNING | Keyword EGAIN not found, not
written to quality report.
2024-11-26 21:59:18,319 | ApFindStars | WARNING | Keyword AIRMASS not found, not
written to quality report.
2024-11-26 21:59:18,319 | ApFindStars | WARNING | Keyword APRX_XWD not found,
not written to quality report.
2024-11-26 21:59:18,319 | ApFindStars | WARNING | Keyword APRX_YHG not found,
not written to quality report.
2024-11-26 21:59:18,319 | ApFindStars | WARNING | Keyword APRX_XPS not found,
not written to quality report.
2024-11-26 21:59:18,319 | ApFindStars | WARNING | Keyword APRX_YPS not found,
not written to quality report.
2024-11-26 21:59:18,319 | ApFindStars | WARNING | Skipping some quality
reporting that requires an estimated platescale.
2024-11-26 21:59:18,321 | ApFindStars | INFO | Wrote image quality report to
qual-M101_Supernova_2023ixf-180s-Lum-1.yaml
2024-11-26 21:59:18,325 | ApFindStars | DEBUG | Wrote ds9-format region file to
ds9-M101_Supernova_2023ixf-180s-Lum-1.reg
2024-11-26 21:59:18,325 | ApFindStars | INFO | Image is 3056 cols x 3056 rows
2024-11-26 21:59:18,325 | ApFindStars | DEBUG | FITS keywords found in image:
{'IMG_FILE': ('Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-1.fits',
'Name of image file searched for stars'), 'IMG_COLS': (3056, 'Number of columns
in input image'), 'IMG_ROWS': (3056, 'Number of rows in input image'),
'AP_NDET': (306, 'Number of sources detected in the image.'), 'AP_NPHOT': (200,
'Number of sources final photometry.'), 'AP_NFIT': (25, 'Number of sources used
in FWHM fitting.'), 'AP_NSIGM': (7.0, 'Source searching threshold (sigma above
background)'), 'APTDIA': (610.0, '[mm] Diameter of telescope aperture'), 'RA':
('14 03 12.40', 'Target right ascension'), 'DEC': ('+54 20 58.0', 'Target
declination'), 'FOCALLEN': (3962.0, '[mm] Stated telescope focal length'),
'FILTER': ('Lum', 'Filter used')}
2024-11-26 21:59:18,326 | ApFindStars | DEBUG | FITS keywords missing from

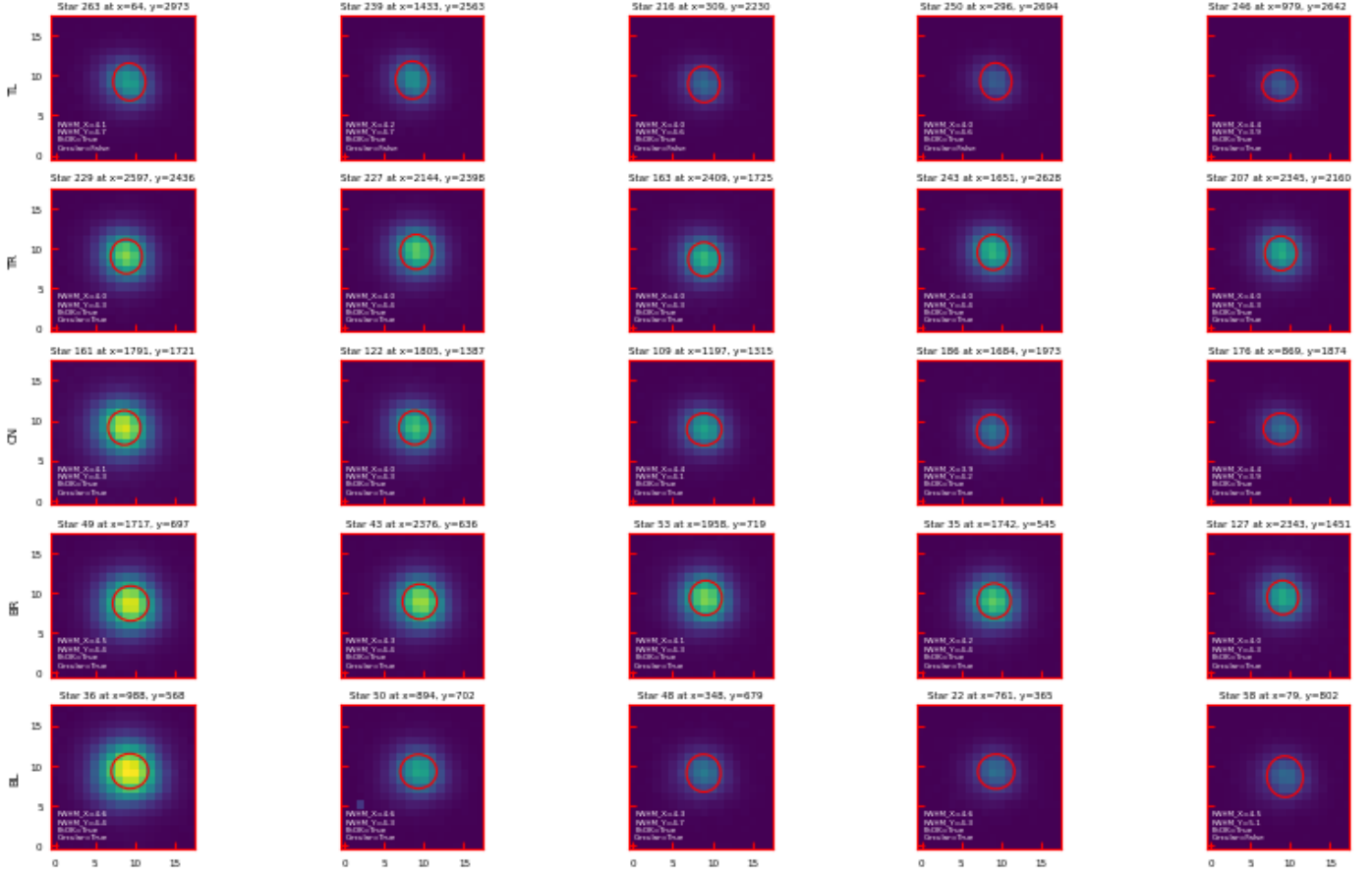
```

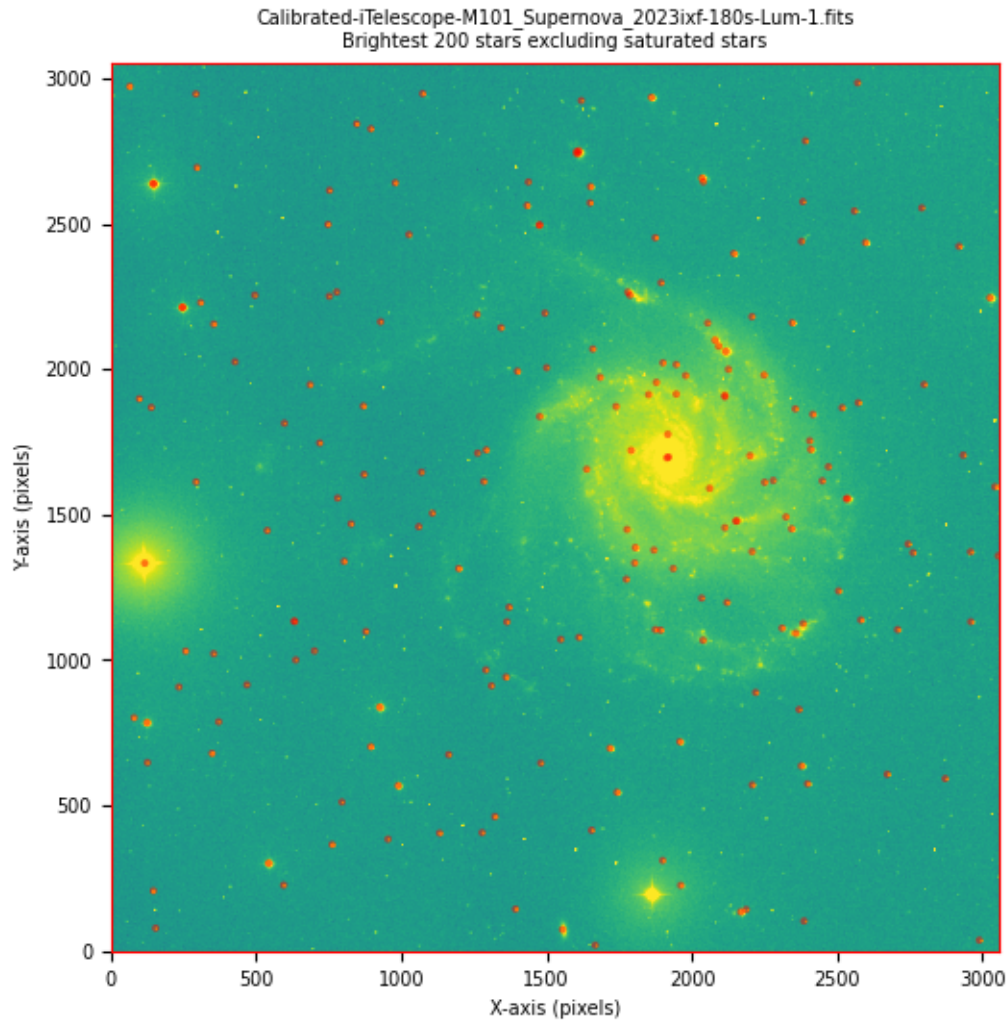


```
image: ['EXPOSURE', 'DATE-OBS', 'OBJECT', 'OBJNAME', 'TELESCOP', 'INSTRUME',  
'CCD-TEMP', 'RA-OBJ', 'DEC-OBJ', 'XPIXSZ', 'YPIXSZ', 'EGAIN', 'LAT-OBS', 'LONG-  
OBS', 'ALT-OBS', 'AIRMASS']  
2024-11-26 21:59:18,326 | ApFindStars | INFO | Approximate coordinates:  
ra=14.053444 hours, dec=54.349444 deg  
2024-11-26 21:59:18,326 | ApFindStars | DEBUG | Converting python 0-based  
coordinates to FITS 1-based pixel coordinates for XY table.  
2024-11-26 21:59:18,327 | ApFindStars | INFO | Writing source list to FITS  
binary table srclist-M101_Supernova_2023ixf-180s-Lum-1.fits  
WARNING: Attribute `version` of type <class 'dict'> cannot be added to FITS  
Header - skipping [astropy.io.fits.convenience]  
WARNING: VerifyWarning: Keyword name 'aperture_photometry_args' is greater than  
8 characters or contains characters not allowed by the FITS standard; a HIERARCH  
card will be created. [astropy.io.fits.card]  
WARNING: VerifyWarning: Card is too long, comment will be truncated.  
[astropy.io.fits.card]
```



Star FWHM measurements for:
 Calibrated-Telescope-M101_Supernova_2023ixf-180s-Lum-1.fits
 Median FWHM=4.31 +/- 0.34 (MAD stddev) pixels
 Along X FWHM=4.07 +/- 0.16 pixels, along Y FWHM=4.34 +/- 0.10 pixels





Rather surprisingly this ran without error, despite the limited set of header keywords. The logging did note that “FITS keywords missing from image: ['EXPOSURE', 'DATE-OBS', 'OBJECT', 'OBJNAME', 'TELESCOP', 'INSTRUME', 'CCD-TEMP', 'RA-OBJ', 'DEC-OBJ', 'XPIXSZ', 'YPIXSZ', 'EGAIN', 'LAT-OBS', 'LONG-OBS', 'ALT-OBS', 'AIRMASS']”

Missing Keywords Note that a warning is emitted stating several keywords, in particular `APRX_XWD` `APRX_YHG` `APRX_XPS` `APRX_YPS`, will not be written to the quality report. These represent the approximate angular width and height of the image in decimal degrees, and the approximate plate scale in X and Y. These could not be computed because some of the required telescope metadata was not present in the input image header. In this case, the CCD detector `XPIXSZ` and `YPIXSZ` were missing, so the approximate plate scale could not be computed. Without that information the approximate field of view (angular width & height) of the image could not be computed.

Is That Large Background Real? Another item to note is a message early in the output: `Source-masked image stats: mean=489.139, median=488.849, stddev=25.656`. The average signal per pixel after excluding the region around the initially detected point sources is 489 ± 26 per pixel. The units are not listed in the header, so they could be ADU, or could have been converted into electrons. One concern is that the numerical values iTelescope images often include an artificial positive `OFFSET` or `PEDESTAL`. This is listed in the raw FITS files, and if not removed acts as a large increase in sky brightness. It is not clear whether this is present in these premium images.

We’re not going to try to solve that question, or subtract the background, in this notebook.

Astrometry To attempt astrometry using `Astrometry.net` we need an **API key**. If you have set up the astroquery config file with one already (see `doc/iTelescope_processing.md`) you can hope that it is picked up, or you can enter one in the following prompt.

```
4]: msg = "Enter your Astrometry.net API key, or hit return to use one preconfigured with astroquery: "
p_astnetkey = input(msg).strip() or None
if p_astnetkey is None:
    print(
        "Warning: Assuming that 'api_key' is specified in your ~/.astropy/config/astroquery.cfg file."
    )
    print(" If this is not the case processing will fail.")
```

Enter your Astrometry.net API key, or hit return to use one preconfigured with astroquery:

Warning: Assuming that 'api_key' is specified in your
~/astropy/config/astroquery.cfg file.
If this is not the case processing will fail.

```
5]: ap_astrom = ap.ApAstrometry(  
    p_inping,  
    p_srclist,  
    p_outimg,  
    inp_img_extnum=p_extnum,  
    srclist_extname=p_src_extname,  
    astnet_key=p_astnetkey,  
    use_sip=p_use_sip,  
    user_scale=p_user_scale,  
    scale_err_ratio=p_scale_err_ratio,  
    loglevel=p_loglevel,  
)  
p_status = ap_astrom.status()  
print(f"ApAstrometry return status: {p_status}")  
if p_status == ap.ApAstrometry.NOMINAL:  
    print(" Astrometric solution succeeded.")  
elif p_status == ap.ApAstrometry.INPUT_ERROR:  
    print(" Error, incorrect or missing input.")  
elif p_status == ap.ApAstrometry.NO_SOLUTION:  
    print(" Error, Astrometry.net failed to find a solution and/or timed out.")  
else:  
    print(" Error, unexpected error code returned by Astrometry.net")
```

```
2024-11-26 21:59:21,917 | AstroPhotography.core.ApAstrometry | INFO | Loading  
extension 0 of FITS file Calibrated-  
iTelescope-M101_Supernova_2023ixf-180s-Lum-1.fits  
2024-11-26 21:59:21,919 | AstroPhotography.core.ApAstrometry | DEBUG | 2-D  
BITPIX=-32 image with 3056 columns, 3056 rows  
2024-11-26 21:59:21,921 | AstroPhotography.core.ApAstrometry | INFO | Read 200  
source positions from srclist-M101_Supernova_2023ixf-180s-Lum-1.fits  
2024-11-26 21:59:21,921 | AstroPhotography.core.ApAstrometry | DEBUG | The  
source list was generated from this input image. Proceeding.  
2024-11-26 21:59:21,922 | AstroPhotography.core.ApAstrometry | DEBUG | Estimated  
center_ra=210.802, center_dec=54.349, radius=8.000 deg.  
2024-11-26 21:59:21,922 | AstroPhotography.core.ApAstrometry | WARNING | Could  
not generate scale hints.  
2024-11-26 21:59:21,922 | AstroPhotography.core.ApAstrometry | DEBUG |  
Astrometry.net hints dictionary: {'center_ra': 210.80166666666665, 'center_dec':  
54.34944444444444, 'radius': 8}  
2024-11-26 21:59:21,923 | AstroPhotography.core.ApAstrometry | DEBUG |  
Submitting astrometry.net solve from source list  
srclist-M101_Supernova_2023ixf-180s-Lum-1.fits with 200 positions (attempt #1)  
  
Solving...  
  
2024-11-26 21:59:35,838 | AstroPhotography.core.ApAstrometry | INFO | Obtained a  
WCS header  
2024-11-26 21:59:35,839 | AstroPhotography.core.ApAstrometry | DEBUG | Setting  
WCSAXES to (2, 'no comment')  
2024-11-26 21:59:35,839 | AstroPhotography.core.ApAstrometry | DEBUG | Setting  
CTYPE1 to ('RA---TAN', 'TAN (gnomic) projection')  
2024-11-26 21:59:35,839 | AstroPhotography.core.ApAstrometry | DEBUG | Setting  
CTYPE2 to ('DEC--TAN', 'TAN (gnomic) projection')  
2024-11-26 21:59:35,840 | AstroPhotography.core.ApAstrometry | DEBUG | Setting  
EQUINOX to (2000.0, 'Equatorial coordinates definition (yr)')  
2024-11-26 21:59:35,840 | AstroPhotography.core.ApAstrometry | DEBUG | Setting  
LONPOLE to (180.0, 'no comment')  
2024-11-26 21:59:35,840 | AstroPhotography.core.ApAstrometry | DEBUG | Setting  
LATPOLE to (0.0, 'no comment')  
2024-11-26 21:59:35,840 | AstroPhotography.core.ApAstrometry | DEBUG | Setting  
CRVAL1 to (210.754804653, 'RA of reference point')  
2024-11-26 21:59:35,840 | AstroPhotography.core.ApAstrometry | DEBUG | Setting  
CRVAL2 to (54.4174059435, 'DEC of reference point')  
2024-11-26 21:59:35,841 | AstroPhotography.core.ApAstrometry | DEBUG | Setting  
CRPIX1 to (1528.5, 'X reference pixel')  
2024-11-26 21:59:35,841 | AstroPhotography.core.ApAstrometry | DEBUG | Setting  
CRPIX2 to (1528.5, 'Y reference pixel')  
2024-11-26 21:59:35,841 | AstroPhotography.core.ApAstrometry | DEBUG | Setting  
CUNIT1 to ('deg', 'X pixel scale units')  
2024-11-26 21:59:35,842 | AstroPhotography.core.ApAstrometry | DEBUG | Setting  
CUNIT2 to ('deg', 'Y pixel scale units')  
2024-11-26 21:59:35,842 | AstroPhotography.core.ApAstrometry | DEBUG | Setting  
CD1_1 to (-4.64681630389e-06, 'Transformation matrix')
```

2024-11-26 21:59:35,842 | AstroPhotography.core.ApAstrometry | DEBUG | Setting CD1_2 to (0.000175223532872, 'no comment')
2024-11-26 21:59:35,842 | AstroPhotography.core.ApAstrometry | DEBUG | Setting CD2_1 to (-0.000175223532872, 'no comment')
2024-11-26 21:59:35,843 | AstroPhotography.core.ApAstrometry | DEBUG | Setting CD2_2 to (-4.64681630389e-06, 'no comment')
2024-11-26 21:59:35,843 | AstroPhotography.core.ApAstrometry | DEBUG | Setting IMAGEW to (3056, 'Image width, in pixels.')
2024-11-26 21:59:35,843 | AstroPhotography.core.ApAstrometry | DEBUG | Setting IMAGEH to (3056, 'Image height, in pixels.')
2024-11-26 21:59:35,868 | AstroPhotography.core.ApAstrometry | INFO | Wrote updated file with WCS to navigated-M101_Supernova_2023ixf-180s-Lum-1.fits
2024-11-26 21:59:35,868 | AstroPhotography.core.ApAstrometry | INFO | Updating photometry in srclist-M101_Supernova_2023ixf-180s-Lum-1.fits with WCS solution.

ApAstrometry return status: 0
Astrometric solution succeeded.

WARNING: VerifyWarning: Keyword name 'aperture_photometry_args' is greater than 8 characters or contains characters not allowed by the FITS standard; a HIERARCH card will be created. [astropy.io.fits.card]

Astrometry.net can plate solve an image (or a set of star coordinates drawn from an image, as in this case) without any other information. This can be time consuming, as it has to try solutions of a wider variety of scales and pointings (image covering the whole sky down to images covering a few arcminutes, centered anyway on the sky). The solution can fail to return an answer, if the process run time exceeds limits set by the Astrometry.net owners.

Providing it a hint at the approximate pointing and/or approximate angular size of the image can considerably speed up plate solution. The hint does not need to be particularly accurate, so even a rough guess is better than nothing. Note that in this example ApAstrometry picked up on the RA, DEC in the image header, but defaulted to a guess of 8 degrees for the image size. The value of 8 degrees is a reasonable median between camera images covering tens of degrees and amateur telescopes covering 30 arcminutes to a few degrees.

After the solution we can see that the image field of view is roughly 0.5 degrees on a side, so providing a hint of 8 degrees did not compromise the solution. You can also see that the approximate image center (210.80166666666665, 54.34944444444444), does differ from the astrometric solution (210.754804653, 54.4174059435) by approximately 0.05 degrees (3 arcminutes) in RA and Dec.

A brief aside about the CDi_j matrix The WCS returned by Astrometry.net (mostly) follows the official FITS standard. In particular a CD matrix is used instead of the older CDELT1, CDELT2, CROTA2. The older keywords are more informative when you want an idea of the traditional plate scale (arcseconds per pixel). The conversion back from a simple CDi_j matrix into CDELT and CROTA is given in <https://lweb.cfa.harvard.edu/~jzhao/SMA-FITS-CASA/docs/wcs88.pdf>. Note that the conversion assumes an idealized case of square pixels at the tangent point.

If you are only interested in the magnitude CDELT1, CDELT2, and not their signs or the rotation angle CROTA2, then the conversion is as simple as:

```
|CDELT1| = sqrt( CD1_1^2 + CD2_1^2 )      # Note: not CD1_1^2 and CD1_2^2  
  
|CDELT2| = sqrt( CD1_2^2 + CD2_2^2 )      # Note: not CD2_1^2 and CD2_2^2
```

3 Processing all the images

We'll now run though processing all the images, but still doing most of the coding work by hand.

```
[5]: p_dir = pathlib.Path(r"./")  
      flist = list(sorted(p_dir.glob(file_pattern)))  
      print(f'{len(flist)} files match pattern "{file_pattern}" in current directory.')  
      for fpath in flist:  
          print(f" {fpath.name}")
```

12 files match pattern "Calibrated-iTelescope*.fits" in current directory.
Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-1.fits
Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-2.fits
Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-3.fits
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Blue-1.fits
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Blue-2.fits
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Blue-3.fits
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Green-1.fits
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Green-2.fits
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Green-3.fits
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Red-1.fits
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Red-2.fits
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Red-3.fits

```
[7]: keys = ["naxis1", "naxis2", "imagetyp", "object", "filter", "exposure"]  
      ifc_cal = ImageFileCollection(  
          p_dir, keywords=keys, glob_include=file_pattern  
      ) # no glob_excludes at this time...  
      print(ifc_cal.summary)
```

file	naxis1	naxis2
imagetyp	object	filter
exposure		

```

-----
Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-1.fits 3056 3056
-- -- Lum --
Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-2.fits 3056 3056
-- -- Lum --
Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-3.fits 3056 3056
-- -- Lum --
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Blue-1.fits 3056 3056
-- -- Blue --
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Blue-2.fits 3056 3056
-- -- Blue --
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Blue-3.fits 3056 3056
-- -- Blue --
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Green-1.fits 3056 3056
-- -- Green --
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Green-2.fits 3056 3056
-- -- Green --
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Green-3.fits 3056 3056
-- -- Green --
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Red-1.fits 3056 3056
-- -- Red --
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Red-2.fits 3056 3056
-- -- Red --
Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Red-3.fits 3056 3056
-- -- Red --

```

Note: here we see one of the issues with the lack of a strict FITS standard. The common fits keyword **EXPOSURE** is not defined in these headers, and in fact the premium images use **EXPTIME**. In fact, both **EXPOSURE** and **EXPTIME** are valid.

For a list of commonly used (in professional settings) FITS keywords see https://heasarc.gsfc.nasa.gov/docs/fcg/common_dict.html. Note that the [official standard](#) is defined elsewhere. This [HEASARC webpage has a subset of the standard keywords](#) but note that it does not include the recommendations from the FITS papers.

This is not a huge issue for getting an Astrometric solution, where each image is processed separately. However, when later combining and mosaicing images we are likely to want to select on the exposure time of the observation in order to maximize the final signal to noise ratio.

```

8]: # define a structure to hold the processing status of the images
# Note: this is an astropy table, so it has that annoying string length must be specified
# if you must use 'object' limitation

arr_size = len(ifc_cal.summary)
arr_dtype = [
    ("filename", object),
    ("find_stars_status", object),
    ("find_stars_time", float),
    ("astrometry_status", object),
    ("astrometry_time", float),
]

processing_results = Table(data=np.empty(arr_size, dtype=arr_dtype))
processing_results["find_stars_time"].info.format = "7.3f"
processing_results["astrometry_time"].info.format = "7.3f"

# processing_results = Table(names=('filename', 'find_stars_status', 'find_stars_time', 'astrometry_status', 'astrometry_time'),
#                               dtype=(str, str, float, str, float))
print(processing_results)

```

```

filename find_stars_status find_stars_time astrometry_status astrometry_time
-----
None None 0.000 None 0.000
None None 0.000 None 0.000
None None 0.000 None 0.000
None None 0.000 None 0.000
None None 0.000 None 0.000
None None 0.000 None 0.000
None None 0.000 None 0.000
None None 0.000 None 0.000
None None 0.000 None 0.000
None None 0.000 None 0.000
None None 0.000 None 0.000
None None 0.000 None 0.000
None None 0.000 None 0.000
None None 0.000 None 0.000
None None 0.000 None 0.000

```

```

9]: # Assorted processing settings# parameters used with source detection
p_loglevel = "DEBUG" # Loglevel
p_fitsimg = str(flist[0]) # Input fits image
p_extnum = 0 # Extension number for data in input fits
p_search_fwhm = 3.0 # initial estimate at star fwhm in pixels

```

```

p_search_nsigma = 7.0 # min detection sigma above background
p_detector_bitdepth = (
    16 # detector bitdepth (16 for most CCDs, ?? for CMOS, ?? for camera)
)
p_sat_frac = 0.8 # Fraction of full well at which we assume star saturated
p_max_sources = 200 # Max number of sources to output or None
p_nosatmask = True # Keep possibly saturated stars.
p_quiet = False # Quiet mode suppresses STDOUT list printing

# parameters associated with astrometric solution, if not already specified above.
# Astrometry.Net API key will be dealt with elsewhere
p_inping = p_fitsimg
p_srclist = p_fitstbl
p_src_extname = "AP_XYPOS"
p_use_sip = False
p_user_scale = None
p_scale_err_ratio = None

```

```

# Settings for output file names and output file paths
# - This is difficult to fully automate without some assumptions about the input file names
# - I assume that all input file have the same file name prefix, e.g. Calibrated, or cal
# - I assume that all input files have the same file extension, e.g. fits or (sadness) fit
# - I prefer to place output files of a certain type in a subdirectories. If you don't want all
#   the diagnostic plots then it may just be simpler to not deal with subdirectories.
# - If dir is not None then the various outputs files will be written to directories with the specified
#   path relative to the **current** directory. The directory will be created if it not already
#   present.

```

```

name_conv_dict = {
    "srclist": {
        "replace": "Calibrated-iTelescope",
        "with": "srclist",
        "extension": None,
        "dir": "SourceLists",
    },
    "regfile": {
        "replace": "Calibrated-iTelescope",
        "with": "ds9",
        "extension": "reg",
        "dir": "SourceLists",
    },
    "plotfile": {
        "replace": "Calibrated-iTelescope",
        "with": "implot",
        "extension": "png",
        "dir": "SourceLists",
    },
    "fwhmfile": {
        "replace": "Calibrated-iTelescope",
        "with": "fwhmplot",
        "extension": "png",
        "dir": "SourceLists",
    },
    "qualfile": {
        "replace": "Calibrated-iTelescope",
        "with": "qual",
        "extension": "yaml",
        "dir": "MetaData",
    },
    "navfile": {
        "replace": "Calibrated-iTelescope",
        "with": "navigated",
        "extension": None,
        "dir": "NavigatedImages",
    },
}

```

```

# Settings for reprocessing.
# If p_clean == True then existing output files will be erased
# If p_clean == False then if a file with the expected sourcelist output file name exists
#   (for find stars) or navigated image output name exists (for astrometry) then that stage
#   of processing is assumed completed and will not be done again. This is a considerable
#   time saver when rerunning the pipeline to fix one or two files that failed previously.
p_clean = False

```

```

1]: def mk_output_file_path(
    infile,
    prefix_replace_this,
    prefix_with_this,
    new_file_extension=None,
    sub_dir=None,
    mkdir=False,
    verbose=True,
):
    """
    Given an input file name, modify the file prefix, optionally modifying the file
    extension and adding a subdirectory to the front of the returned path.

    :param infile: String containing the input file name we want to use as a
        basis for further file names, e.g. `Calibrated-iTelescope-M101-Supernova-2023iæf-180s-Lum-1.fits`
    :param prefix_replace_this: A part of the input file name that will be replaced with prefix_with_this,
        for example `Calibrated-iTelescope`. It is up to the user to ensure that this substring
        is present in the input file name.
    :param prefix_with_this: What to replace prefix_replace_this with. For example `srclist`.
    :param new_file_extension: If not None then replace the `fits` file extension with this.
        Do not include the `.`.
    :param sub_dir:
    :param mkdir:
    :param verbose:

    Returns a string representing a new file name, for use by other programs.
    """

    output_file_path = None
    if verbose:
        print(f"Input file name: {infile}")

    if sub_dir is not None:
        if not os.access(sub_dir, os.F_OK):
            if mkdir:
                try:
                    os.mkdir(sub_dir)
                except:
                    print(f"Error, mkdir failed trying to create {sub_dir}")
                    raise
            if verbose:
                print(f"Successfully created subdirectory {sub_dir}")
            else:
                raise RuntimeError(
                    f"Subdirectory {sub_dir} cannot be accessed, but mkdir=False"
                )
        else:
            if verbose:
                print(f"Subdirectory {sub_dir} already exists.")

    tmp_str = infile.replace(prefix_replace_this, prefix_with_this)
    if new_file_extension is not None:
        tmp_str = tmp_str.replace("fits", new_file_extension)
    if sub_dir is not None:
        output_file_path = f"{sub_dir}/{tmp_str}"
    else:
        output_file_path = tmp_str

    if verbose:
        print(f"Output file name: {output_file_path}")
    if infile == output_file_path:
        print(
            f"Warning: input file name ({infile}) matches output file name ({output_file_path})."
        )

    return output_file_path

def mk_all_file_names(infile, name_conv_dict, mkdir=True, verbose=False):
    """
    Given an input file name and a conversion dictionary, generate all the file names that might
    be used to run star finding and astrometry on that image.

    Returns f_srclist, f_regfile, f_plotfile, f_fwhmfile, f_qualfile, f_naufile
    """

```



```

oname = ["srclist", "regfile", "plotfile", "fwhmfile", "qualfile", "navfile"]
ofile = {}
for ftype in name_conv_dict.keys():
    ofile[ftype] = mk_output_file_path(
        inpfiler,
        name_conv_dict[ftype]["replace"],
        name_conv_dict[ftype]["with"],
        name_conv_dict[ftype]["extension"],
        name_conv_dict[ftype]["dir"],
        mkdir,
        verbose,
    )

return (
    ofile["srclist"],
    ofile["regfile"],
    ofile["plotfile"],
    ofile["fwhmfile"],
    ofile["qualfile"],
    ofile["navfile"],
)

```

```

2]: # test the function
mkdir = True
verbose = True
for ftype in name_conv_dict.keys():
    print(f"For file type {ftype} -----")
    ofile = mk_output_file_path(
        ifc_cal.summary["file"][0],
        name_conv_dict[ftype]["replace"],
        name_conv_dict[ftype]["with"],
        name_conv_dict[ftype]["extension"],
        name_conv_dict[ftype]["dir"],
        mkdir,
        verbose,
    )

print("\nAll the file names:")
f_srclist, f_regfile, f_plotfile, f_fwhmplot, f_qualfile, f_navfile = mk_all_file_names(
    ifc_cal.summary["file"][0], name_conv_dict, mkdir, False
)
print(
    f"  srclist={f_srclist}\n  regfile={f_regfile}\n  plotfile={f_plotfile}\n  fwhmplot={f_fwhmplot}\n  qualfile={f_qualfile}\n  ↵
    ↵navfile={f_navfile}"
)

```

```

For file type srclist -----
Input file name: Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-1.fits
Subdirectory SourceLists already exists.
Output file name: SourceLists/srclist-M101_Supernova_2023ixf-180s-Lum-1.fits
For file type regfile -----
Input file name: Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-1.fits
Subdirectory SourceLists already exists.
Output file name: SourceLists/ds9-M101_Supernova_2023ixf-180s-Lum-1.reg
For file type plotfile -----
Input file name: Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-1.fits
Subdirectory SourceLists already exists.
Output file name: SourceLists/implot-M101_Supernova_2023ixf-180s-Lum-1.png
For file type fwhmfile -----
Input file name: Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-1.fits
Subdirectory SourceLists already exists.
Output file name: SourceLists/fwhmplot-M101_Supernova_2023ixf-180s-Lum-1.png
For file type qualfile -----
Input file name: Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-1.fits
Subdirectory MetaData already exists.
Output file name: MetaData/qual-M101_Supernova_2023ixf-180s-Lum-1.yaml
For file type navfile -----
Input file name: Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-1.fits
Subdirectory NavigatedImages already exists.
Output file name:
NavigatedImages/navigated-M101_Supernova_2023ixf-180s-Lum-1.fits

```

```

All the file names:
  srclist=SourceLists/srclist-M101_Supernova_2023ixf-180s-Lum-1.fits
  regfile=SourceLists/ds9-M101_Supernova_2023ixf-180s-Lum-1.reg

```

```

plotfile=SourceLists/imshow-M101_Supernova_2023ixf-180s-Lum-1.png
fwhmplot=SourceLists/fwhmplot-M101_Supernova_2023ixf-180s-Lum-1.png
qualfile=MetaData/qual-M101_Supernova_2023ixf-180s-Lum-1.yaml
navfile=NavigatedImages/navigated-M101_Supernova_2023ixf-180s-Lum-1.fits

```

```

3]: def check_file_exists(filename):
    """Return true if the specified file path exists"""
    if not os.path.isfile(filename):
        return False
    return True

4]: # iterate over files in the image file collection
#
# TODO work out how to direct output or log to a file...
p_clean = False
p_loglevel = "INFO"
p_quiet = True
import time

idx = 0
proc_tstart = time.perf_counter()
for hdu, fname in ifc_cal.hdus(return_fname=True):
    print(80 * "-")
    print(f"Processing input file {fname}")

    (
        f_srclist,
        f_regfile,
        f_plotfile,
        f_fwhmplot,
        f_qualfile,
        f_navfile,
    ) = mk_all_file_names(fname, name_conv_dict, mkdir, False)

    # Determine whether to perform star detection, based on p_clean and presence of
    does_srclist_exist = check_file_exists(f_srclist)
    if p_clean or not does_srclist_exist:
        print(f"\n Finding stars, output source list: {f_srclist}")
        fs_tstart = time.perf_counter()
        status = "pass"
        try:
            find_stars_wrapper(
                p_loglevel,
                fname,
                f_srclist,
                p_extnum,
                p_search_fwhm,
                p_search_nsigma,
                p_detector_bitdepth,
                p_sat_frac,
                p_max_sources,
                p_nosatmask,
                f_plotfile,
                p_quiet,
                f_fwhmplot,
                f_qualfile,
                f_regfile,
            )
        except:
            print(f" Error, caught exception when processing {fname}")
            status = "fail"

        fs_tend = time.perf_counter()
        fs_telapsed = fs_tend - fs_tstart # seconds
        fs_status = status
    else:
        print(
            f"\n Skipping star detection because {f_srclist} exists and clean={p_clean}"
        )
        fs_status = "skipped_exists"
        fs_telapsed = 0

    # Determine whether to perform star detection, based on p_clean and presence of
    does_navfile_exist = check_file_exists(f_navfile)
    if does_navfile_exist:

```

```

if p_clean or not does_navfile_exist:
    ast_status = "pass"
    ast_tstart = time.perf_counter()
    print(f"\n Performing astrometry, output navigated image: {f_navfile}")
    try:
        ap_astrom = ap.ApAstrometry(
            fname,
            f_srclist,
            f_navfile,
            inp_img_extnum=p_extnum,
            srclist_extname=p_src_extname,
            astnet_key=p_astnetkey,
            use_sip=p_use_sip,
            user_scale=p_user_scale,
            scale_err_ratio=p_scale_err_ratio,
            loglevel=p_loglevel,
        )
        p_status = ap_astrom.status()
        print(f" ApAstrometry return status: {ast_status}")
        if p_status == ap.ApAstrometry.NOMINAL:
            ast_status = "pass"
        elif p_status == ap.ApAstrometry.INPUT_ERROR:
            ast_status = "input_error"
        else:
            ast_status = "fail"
    except:
        print(f" Error, caught exception when processing {fname}")
        ast_status = "exception"

    ast_tend = time.perf_counter()
    ast_telapsed = ast_tend - ast_tstart # seconds
    ast_status = status

else:
    print(
        f"\n Skipping astrometry because {f_navfile} exists and clean={p_clean}"
    )
    ast_status = "skipped_exists"
    ast_telapsed = 0

else:
    # No source list
    print(f"\n Skipping astrometry because {f_srclist} does not exist.")
    ast_status = "skipped_no_srclist"
    ast_telapsed = 0

# Fill in run info for this file
result_tuple = (fname, fs_status, fs_telapsed, ast_status, ast_telapsed)
processing_results[idx] = result_tuple

# update idx
idx += 1

proc_tend = time.perf_counter()
proc_telapsed = proc_tend - proc_tstart
print(f"Finished processing {idx+1} files in {proc_telapsed:.3f} seconds.")

```

```

-----
Processing input file Calibrated-
iTelescope-M101_Supernova_2023ixf-180s-Lum-1.fits

```

```

    Skipping star detection because
SourceLists/srclist-M101_Supernova_2023ixf-180s-Lum-1.fits exists and
clean=False

```

```

    Skipping astrometry because
NavigatedImages/navigated-M101_Supernova_2023ixf-180s-Lum-1.fits exists and
clean=False

```

```

-----
Processing input file Calibrated-
iTelescope-M101_Supernova_2023ixf-180s-Lum-2.fits

```

```

    Skipping star detection because
SourceLists/srclist-M101_Supernova_2023ixf-180s-Lum-2.fits exists and
clean=False

```

```

    Skipping astrometry because
NavigatedImages/navigated-M101_Supernova_2023ixf-180s-Lum-2.fits exists and

```

```

clean=False
-----
Processing input file Calibrated-
iTelescope-M101_Supernova_2023ixf-180s-Lum-3.fits

    Skipping star detection because
SourceLists/srcList-M101_Supernova_2023ixf-180s-Lum-3.fits exists and
clean=False

    Skipping astrometry because
NavigatedImages/navigated-M101_Supernova_2023ixf-180s-Lum-3.fits exists and
clean=False
-----
Processing input file Calibrated-
iTelescope-M101_Supernova_2023ixf-300s-Blue-1.fits

    Skipping star detection because
SourceLists/srcList-M101_Supernova_2023ixf-300s-Blue-1.fits exists and
clean=False

    Skipping astrometry because
NavigatedImages/navigated-M101_Supernova_2023ixf-300s-Blue-1.fits exists and
clean=False
-----
Processing input file Calibrated-
iTelescope-M101_Supernova_2023ixf-300s-Blue-2.fits

    Skipping star detection because
SourceLists/srcList-M101_Supernova_2023ixf-300s-Blue-2.fits exists and
clean=False

    Skipping astrometry because
NavigatedImages/navigated-M101_Supernova_2023ixf-300s-Blue-2.fits exists and
clean=False
-----
Processing input file Calibrated-
iTelescope-M101_Supernova_2023ixf-300s-Blue-3.fits

    Skipping star detection because
SourceLists/srcList-M101_Supernova_2023ixf-300s-Blue-3.fits exists and
clean=False

    Skipping astrometry because
NavigatedImages/navigated-M101_Supernova_2023ixf-300s-Blue-3.fits exists and
clean=False
-----
Processing input file Calibrated-
iTelescope-M101_Supernova_2023ixf-300s-Green-1.fits

    Skipping star detection because
SourceLists/srcList-M101_Supernova_2023ixf-300s-Green-1.fits exists and
clean=False

    Skipping astrometry because
NavigatedImages/navigated-M101_Supernova_2023ixf-300s-Green-1.fits exists and
clean=False
-----
Processing input file Calibrated-
iTelescope-M101_Supernova_2023ixf-300s-Green-2.fits

    Skipping star detection because
SourceLists/srcList-M101_Supernova_2023ixf-300s-Green-2.fits exists and
clean=False

    Skipping astrometry because
NavigatedImages/navigated-M101_Supernova_2023ixf-300s-Green-2.fits exists and
clean=False
-----
Processing input file Calibrated-
iTelescope-M101_Supernova_2023ixf-300s-Green-3.fits

    Skipping star detection because
SourceLists/srcList-M101_Supernova_2023ixf-300s-Green-3.fits exists and
clean=False

    Skipping astrometry because

```

```
NavigatedImages/navigated-M101_Supernova_2023ixf-300s-Green-3.fits exists and
clean=False
```

```
-----
Processing input file Calibrated-
iTelescope-M101_Supernova_2023ixf-300s-Red-1.fits
```

```
    Skipping star detection because
SourceLists/src1ist-M101_Supernova_2023ixf-300s-Red-1.fits exists and
clean=False
```

```
    Skipping astrometry because
NavigatedImages/navigated-M101_Supernova_2023ixf-300s-Red-1.fits exists and
clean=False
```

```
-----
Processing input file Calibrated-
iTelescope-M101_Supernova_2023ixf-300s-Red-2.fits
```

```
    Skipping star detection because
SourceLists/src1ist-M101_Supernova_2023ixf-300s-Red-2.fits exists and
clean=False
```

```
    Skipping astrometry because
NavigatedImages/navigated-M101_Supernova_2023ixf-300s-Red-2.fits exists and
clean=False
```

```
-----
Processing input file Calibrated-
iTelescope-M101_Supernova_2023ixf-300s-Red-3.fits
```

```
    Skipping star detection because
SourceLists/src1ist-M101_Supernova_2023ixf-300s-Red-3.fits exists and
clean=False
```

```
    Skipping astrometry because
NavigatedImages/navigated-M101_Supernova_2023ixf-300s-Red-3.fits exists and
clean=False
```

```
Finished processing 13 files in 0.081 seconds.
```

```
5]: # print(processing_results)
processing_results.pprint(max_lines=-1, max_width=-1)
```

	filename	find_stars_status	
find_stars_time	astrometry_status	astrometry_time	
-----	-----	-----	
	Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-1.fits	skipped_exists	
0.000	skipped_exists	0.000	
	Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-2.fits	skipped_exists	
0.000	skipped_exists	0.000	
	Calibrated-iTelescope-M101_Supernova_2023ixf-180s-Lum-3.fits	skipped_exists	
0.000	skipped_exists	0.000	
	Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Blue-1.fits	skipped_exists	
0.000	skipped_exists	0.000	
	Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Blue-2.fits	skipped_exists	
0.000	skipped_exists	0.000	
	Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Blue-3.fits	skipped_exists	
0.000	skipped_exists	0.000	
	Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Green-1.fits	skipped_exists	
0.000	skipped_exists	0.000	
	Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Green-2.fits	skipped_exists	
0.000	skipped_exists	0.000	
	Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Green-3.fits	skipped_exists	
0.000	skipped_exists	0.000	
	Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Red-1.fits	skipped_exists	
0.000	skipped_exists	0.000	
	Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Red-2.fits	skipped_exists	
0.000	skipped_exists	0.000	
	Calibrated-iTelescope-M101_Supernova_2023ixf-300s-Red-3.fits	skipped_exists	
0.000	skipped_exists	0.000	

3.1 Image Resampling and Mosaicing

Assuming that all the input images have been successfully navigated, i.e. have valid WCS solutions, we can move on to combining the images.

It is worth checking that the navigated images do have good solutions using SAOImage ds9. For example:

```
ds9 -align yes -asinh -zoom to fit -cmap viridis -blink interval 2 -blink yes -tile no -single \
    -scale mode 99.5 NavigatedImages/navigated*.fits
```

That should have loaded the images into separate frames, but only shown one at a time with the program blinking through them. However it seems to always tile them with multiple inputs. You need to perform the following actions in the ds9 GUI.

- resize the ds9 window to make it larger by dragging on the window edge
- Click through the menu items **Frame**, then **Single Frame**
- Click through the menu items **Frame**, then **Match**, then **Frame**, then **WCS**
- Click through the menu items **Frame**, then **Blink Frames**

The same stars/features in each separate image should appear to be in the same place on the screen, even as the brightness and noise levels change between frames.

```
61: # Make sure we're still in the original directory, then move into the directory with the navigated images
os.chdir(wdir)
ftype = "navfile"
fdir = name_conv_dict[ftype]["dir"]
fprefix = name_conv_dict[ftype]["with"]
fext = name_conv_dict[ftype]["extension"]
if fext is None:
    fext = ".fits"
try:
    os.chdir(fdir)
    print(f"Successfully changed directory to {os.getcwd()}")
except:
    print("Error, failed to change directory to expected navfile directory {fdir}")
    print(f" Current directory: {os.getcwd()}")
```

Successfully changed directory to
/old_lnx/home/dks/Downloads/iTelescopeScratch/Plan-20-M101SN/NavigatedImages

3.1.1 Caveats

Note: The following code assumes that the navigated images in the current directory correspond to a **single target**.

All images in the directory will be resampled to a single pointing and platescale. By default this will be selected based on the first navigated image in the image file collection.

We would not necessarily want to resample images of the same **filter** but differing **exposure** together as their noise characteristics might be quite distinct, and the resampling method used might not be able to correctly weight them. Similarly cases where the seeing is significantly poorer than average, as determined by the fits performed by **ApMeasureStars** and reported in the quality files, should be excluded from being combined.

Such ideals run afoul of cases where the fits files don't have those keywords or are using different keywords. For now I don't have a work around for such cases and for automatically incorporating the seeing/FWHM measurements.

```
71: nav_file_pattern = f"{fprefix}*{fext}"
p_dir = pathlib.Path(r"./")
keys = ["naxis1", "naxis2", "imagetyp", "object", "filter", "exposure"]
ifc_nav = ImageFileCollection(
    p_dir, keywords=keys, glob_include=nav_file_pattern
) # no glob_excludes at this time...
print(ifc_nav.summary)
```

file		naxis1	naxis2	imagetyp	object
filter	exposure				

	navigated-M101_Supernova_2023ixf-180s-Lum-1.fits	3056	3056	--	--
Lum	--				
	navigated-M101_Supernova_2023ixf-180s-Lum-2.fits	3056	3056	--	--
Lum	--				
	navigated-M101_Supernova_2023ixf-180s-Lum-3.fits	3056	3056	--	--
Lum	--				
	navigated-M101_Supernova_2023ixf-300s-Blue-1.fits	3056	3056	--	--
Blue	--				
	navigated-M101_Supernova_2023ixf-300s-Blue-2.fits	3056	3056	--	--
Blue	--				
	navigated-M101_Supernova_2023ixf-300s-Blue-3.fits	3056	3056	--	--
Blue	--				
	navigated-M101_Supernova_2023ixf-300s-Green-1.fits	3056	3056	--	--
Green	--				
	navigated-M101_Supernova_2023ixf-300s-Green-2.fits	3056	3056	--	--
Green	--				
	navigated-M101_Supernova_2023ixf-300s-Green-3.fits	3056	3056	--	--
Green	--				
	navigated-M101_Supernova_2023ixf-300s-Red-1.fits	3056	3056	--	--
Red	--				
	navigated-M101_Supernova_2023ixf-300s-Red-2.fits	3056	3056	--	--
Red	--				
	navigated-M101_Supernova_2023ixf-300s-Red-3.fits	3056	3056	--	--
Red	--				

```
8]: from astropy.table import unique

uniq_filter_exposure = unique(
    ifc_nav.summary, keys=["filter", "exposure"], keep="first", silent=True
)
uniq_filter_exposure.pprint(max_lines=-1, max_width=-1)
```

	file	naxis1	naxis2	imagetyp	object
filter exposure					
-----	-----	-----	-----	-----	-----
navigated-M101_Supernova_2023ixf-300s-Blue-1.fits	3056	3056	--	--	
Blue --					
navigated-M101_Supernova_2023ixf-300s-Green-1.fits	3056	3056	--	--	
Green --					
navigated-M101_Supernova_2023ixf-180s-Lum-1.fits	3056	3056	--	--	
Lum --					
navigated-M101_Supernova_2023ixf-300s-Red-1.fits	3056	3056	--	--	
Red --					

```
9]: filter_list = []
for filter in uniq_filter_exposure["filter"]:
    filter_list.append(filter)
print(
    f"There are {len(filter_list)} unique filters among {len(ifc_nav.summary)} files: {filter_list}"
)
```

There are 4 unique filters among 12 files: ['Blue', 'Green', 'Lum', 'Red']

3.1.2 What Pointing and Plate Scale Should We Resample To?

There are any number of possible choices we can make regarding the pointing and plate scale that all the images will be resampled to. A few possible choices are shown:

1. Resample all images to match the WCS of another image, for example the first image in the list of navigated images, irrespective of how uniform the platescale is or whether the image is North-up, East-left aligned.
2. Require the user to explicitly specify the RA and Dec of the image center, the X (RA) and Y (Dec) CDELTA* values, and the number of output image rows and columns.
3. Require the user to specify a single plate scale (arcsec/pixel) and a RA/Dec or target identifiers, and calculate the values needed for #2.

There are advantages and disadvantages to each of these approaches. Method #1 may be superior for a final pretty display image where alignment is unimportant, while #2 and #3 are likely better for scientific use of the combined images. Method #2 requires user knowledge of how to specify a WCS and the likely platescale of the images, and is really only good for experts.

For now AstroPhotography will concentrate on supporting methods #1 and #3, starting with method #1.

Note: In image resampling the question of whether the input is well sampled is important. We do not have the space to cover it here, except to note that in general it is better to have pixels in the final (resampled) image be square and as large or indeed slightly larger than the pixels in the input images.

Note: For technical reasons most resampling methods assume the input data pixels represent surface brightness. If the input image has a large field of view it is likely distorted, in particular with images from DLSR or Mirrorless cameras, and the sky area covered by each pixel is not uniform. **Astrometry.net** allows fitting of SIP distortion coefficients. However, correctly calculation surface brightness in such cases is difficult (this package does not currently support it, and I don't know of any that do). Furthermore many of the resampling methods available don't correctly handle the non-uniform spatial aspect of SIP distortions either.

Other methods: resample_all.sh shell script The bash script `resample_all.sh` does perform resampling using Astromatic's `swarp` resampler. Note that the script needs to be hard-wired for each set of images by specifying the RA, Dec of the image center, the plate scale, and the number of pixels along each axis.

```
cd /old_lnx/home/dks/Downloads/iTelescopeScratch/Plan-20-M101SN
~/git/AstroPhotography/AstroPhotography/scripts/resample_all.sh M101sn median clean verbose
... [lots of output omitted]...
```

Processing Lum filter images:

```
Using existing temporary file directory TmpSwarp
Found 3 navigated fits files.
File navigated-M101_Supernova_2023ixf-180s-Lum-3.fits EXPOSURE 180.000 FSCALE 0.005556
File navigated-M101_Supernova_2023ixf-180s-Lum-1.fits EXPOSURE 180.000 FSCALE 0.005556
File navigated-M101_Supernova_2023ixf-180s-Lum-2.fits EXPOSURE 180.000 FSCALE 0.005556
Flux scalings: 0.005556,0.005556,0.005556
Total exposure in Lum filter: 9 minutes.
```

```
Beginning swarp for filter Lum at Wed Feb 28 09:25:14 PM EST 2024
Running the following command: swarp ../NavigatedImages/navigated-M101_Supernova_2023ixf-180s-Lum-3.fits ../NavigatedImages/navigated-M101_Supernova_2023ixf-180s-Lum-1.fits
Generated resampled co-added image M101_Supernova_2023ixf_Lum_4096x4096_resamp_median.fits
Generated co-added weights image M101_Supernova_2023ixf_Lum_4096x4096_weights_median.fits
Swarp for filter Lum completed at Wed Feb 28 09:25:19 PM EST 2024, took 5.099 seconds.
```

Processing Luminance filter images:
 Using existing temporary file directory TmpSwarf
 Found 0 navigated fits files.
 Moving on to next filter...

 Run summary:

Filter	Resampled Output	Ninput	TexpMin	SwrptSec	Status
Red	M101_Supernova_2023ixf_Red_4096x4096_resamp_median.fits	3	15.00	5.20	0
Green	M101_Supernova_2023ixf_Green_4096x4096_resamp_median.fits	3	15.00	5.10	0
Blue	M101_Supernova_2023ixf_Blue_4096x4096_resamp_median.fits	3	15.00	5.09	0
Ha	None	0	0.00	0.00	0
OIII	None	0	0.00	0.00	0
SII	None	0	0.00	0.00	0
B	None	0	0.00	0.00	0
V	None	0	0.00	0.00	0
I	None	0	0.00	0.00	0
Clear	None	0	0.00	0.00	0
Lum	M101_Supernova_2023ixf_Lum_4096x4096_resamp_median.fits	3	9.00	5.10	0
Luminance	None	0	0.00	0.00	0

Master log file: /old_lnx/home/dks/Downloads/iTelescopeScratch/Plan-20-M101SN/tmp_resamplecd_all.log

 Temporary files occupy 1 MB in /old_lnx/home/dks/Downloads/iTelescopeScratch/Plan-20-M101SN/Resampled/TmpSwarf
 Commands were logged to /old_lnx/home/dks/Downloads/iTelescopeScratch/Plan-20-M101SN/tmp_resamplecd_all.log
 Resampled images were written to ./Resampled
 resample_all.sh finished at Sun Feb 25 08:33:13 AM EST 2024, run time 23.638 seconds.

Resampling To Match Another Image This is the simplest conceptual method, but does allow some flexibility in that the other image need not be one of the navigated images. For example we could resample to match the WCS of a Chandra X-ray image, which would be ideal for scientific multi-waveband analysis.

First let's specify the file we want to match...

```
0]: f_default_match = ifc_nav.summary["file"][0]
msg = f"Name of file to match WCS of in resampled images (hit return for default {f_default_match})"
f_to_match = input(msg).strip() or f_default_match
print(f"WCS of resampled files will match {f_to_match}")
```

Name of file to match WCS of in resampled images (hit return for default
 navigated-M101_Supernova_2023ixf-180s-Lum-1.fits)

WCS of resampled files will match
 navigated-M101_Supernova_2023ixf-180s-Lum-1.fits

```
1]: import math

from astropy import wcs

def load_wcs_from_file(filename, verbose=False):
    """
    Modified WCS example based on astropy documentation
    https://docs.astropy.org/en/stable/wcs/loading_from_fits.html

    Limitations: Assumes 2-dimensional image
    """
    w = None
    pix_scales = None # in deg
    pix_area = None # in deg^2
    im_scales = None # in deg

    # Load the FITS hdulist using astropy.io.fits
    with fits.open(filename) as hdulist:
        # Parse the WCS keywords in the primary HDU
        w = wcs.WCS(hdulist[0].header)

        # Print out the "name" of the WCS, as defined in the FITS header
        if verbose:
            print(w.wcs.name)

        # Print out all of the settings that were parsed from the header
        w.wcs.print_contents()

    # Pixel scale at the CRPIX pixel location. Note, ideal, ignores distortions
    pix_scales = wcs.utils.proj_plane_pixel_scales(w)
```



```

# Pixel area at the CRPIX pixel location. Again, ideal, ignores distortions
pix_area = wcs.utils.proj_plane_pixel_area(w.celestial)

if w.wcs.naxis != 2:
    print(
        f"WARNING, WCS from {filename} is {w.wcs.naxis}-dimensional, not 2-D as expected"
    )

if len(pix_scales) != len(w.array_shape):
    raise RuntimeError(
        f"Shape of pixel scales ({len(pix_scales)}) differs from shape of array ({len(w.array_shape)})"
    )
else:
    im_scales = pix_scales.copy()
    for idx in range(len(w.array_shape)):
        im_scales[idx] *= w.array_shape[idx]

if verbose:
    print(f"Pixel angular scale ({w.wcs.cunit[0]}): {pix_scales}")
    print(f"Pixel angular area ({w.wcs.cunit[0]}^2): {pix_area}")
    print(f"NAXIS1={w.array_shape[0]} NAXIS2={w.array_shape[1]}")
    print(f"Detector angular scale ({w.wcs.cunit[0]}): {im_scales}")

return w, pix_scales, pix_area, im_scales

```

```

2]: def summarize_wcs(w):
    """
    Given an astropy WCS object, summarise the properties of the WCS header,
    printing to STDOUT.

    This assumes that the images are two dimensional, and that angles are in degrees.

    :param w: astropy.wcs.WCS instance
    """

    ipwcs = w

    dothead_list = []

    # Numbr of dimensions
    val_str = f"{'NAXIS':8s} = {ipwcs.wcs.naxis}"
    dothead_list.append(val_str)

    keywords = ["NAXIS", "CTYPE", "CRVAL", "CRPIX"]
    values = [ipwcs.array_shape, ipwcs.wcs.ctype, ipwcs.wcs.crval, ipwcs.wcs.crpix]
    for keyword, value in zip(keywords, values):
        for idx in range(ipwcs.naxis):
            kw_str = f"{keyword}-{1+idx}"
            if "CTYPE" in keyword:
                # Wrap strings in quotes
                val_str = f"{kw_str:8s} = '{value[idx]}'"
            else:
                val_str = f"{kw_str:8s} = {value[idx]}"
            dothead_list.append(val_str)

    naxis = ipwcs.wcs.naxis
    naxis1 = ipwcs.array_shape[0]
    naxis2 = ipwcs.array_shape[1]
    if naxis == 3:
        naxis3 = ipwcs.array_shape[2]
        print(
            f"Warning: summarize_wcs() not written for 2-D images, {naxis}-dimensional data"
        )

    cdelt1 = None
    cdelt2 = None
    crot = None

    if hasattr(ipwcs.wcs, "cd"):
        for irow in range(ipwcs.naxis):
            for jcol in range(ipwcs.naxis):
                kw_str = f"CD{irow+1}_{jcol+1}"
                val_str = f"{kw_str:8s} = {ipwcs.wcs.cd[irow, jcol]}"
                dothead_list.append(val_str)

```

```

cd = ipwcs.wcs.cd
# From https://lweb.cfa.harvard.edu/~jzhao/SMA-FITS-CASA/docs/wcs88.pdf
cd11 = cd[0, 0]
cd12 = cd[0, 1]
cd21 = cd[1, 0]
cd22 = cd[1, 1]
cdelt1_mag = math.sqrt(cd12 * cd12 + cd22 * cd22)
cdelt2_mag = math.sqrt(cd12 * cd12 + cd22 * cd22)
# the sign of cdelt1.cdelt2 = sign of (cd11*cd22 - cd12*cd21)
# if the RHS is negative cdelt1 is negative by convention, so cdelt2 is always positive
tmpa = cd11 * cd22 - cd12 * cd21
cdelt1 = math.copysign(cdelt1_mag, tmpa)
cdelt2 = cdelt2_mag
sign = math.copysign(1, tmpa)
crot = math.degrees(math.atan2((sign * cd12), cd22))

elif hasattr(ipwcs.wcs, "pc"):
    for irow in range(ipwcs.naxis):
        for jcol in range(ipwcs.naxis):
            kw_str = f"PC{irow+1}_{jcol+1}"
            val_str = f"{kw_str:8s} = {ipwcs.wcs.pc[irow, jcol]}"
            dothead_list.append(val_str)
            kw_str = f"CDELTA{irow+1}"
            val_str = f"{kw_str:8s} = {ipwcs.wcs.cdelt[irow]}"
            dothead_list.append(val_str)
            # Think that CD1_* = CDELTA1 * PC1_* and CD2_* = CDELTA2 * PC2_*
            pc = ipwcs.wcs.pc
            cdelt_vec = ipwcs.wcs.cdelt
            cd = pc.copy()
            for irow in range(ipwcs.naxis):
                cd[irow] = cdelt_vec[irow] * pc[irow]
            # then as above
            raise RuntimeError(
                "summarize_wcs needs to be updated to handle cases with a PC_ matrix and no CD_ matrix"
            )
else:
    # Assume we have a simple CDELTA[12] case with CROTA
    for irow in range(ipwcs.naxis):
        kw_str = f"CDELTA{irow+1}"
        val_str = f"{kw_str:8s} = {ipwcs.wcs.cdelt[irow]}"
        dothead_list.append(val_str)
        kw_str = f"CROTA{irow+1}"
        val_str = f"{kw_str:8s} = {ipwcs.wcs.crota[irow]}"
        dothead_list.append(val_str)
    cdelt1 = ipwcs.wcs.cdelt[0]
    cdelt2 = ipwcs.wcs.cdelt[1]
    crot = ipwcs.wcs.crota[
        1
    ] # CROTA2 is used, CROTA1 not used. Assume crota[0] == crota[1]

cdelt1_as = cdelt1 * 3600.0 # arcseconds
cdelt2_as = cdelt2 * 3600.0 # arcseconds
ximgsz_am = cdelt1 * naxis1 * 60 # arcminutes
yimgsz_am = cdelt2 * naxis2 * 60 # arcminutes
dothead_list.append(
    f"Pixel size equivalent CDELTA1 = {cdelt1_as:.3f} arcseconds"
)
dothead_list.append(
    f"Pixel size equivalent CDELTA2 = {cdelt2_as:.3f} arcseconds"
)
dothead_list.append(
    f"Image X-axis angular size = {ximgsz_am:.3f} arcminutes"
)
dothead_list.append(
    f"Pixel Y-axis angular size = {yimgsz_am:.3f} arcminutes"
)
dothead_list.append(f"Image rotation equivalent CROTA2 = {crot:.3f} degrees")

dothead_list.append("END")
dothead_str = "\n".join(dothead_list)
print(dothead_str)
return

```

```

3]: # Test load_wcs_from_file
w, pix_scales, pix_area, im_scales = load_wcs_from_file(f_to_match, False)

```

```

pix_scales_as = pix_scales * 3600.0
root_area_as = math.sqrt(pix_area) * 3600.0
im_scales_am = im_scales * 60.0
print(f"Pixel scales (arcsec): {pix_scales_as}")
print(f"Image angular extent (arcmin): {im_scales_am}")
print(f"Square pixel equivalent size (arcsec): {root_area_as}")

summarize_wcs(w)

```

```

Pixel scales (arcsec): [0.63102649 0.63102649]
Image angular extent (arcmin): [32.14028278 32.14028278]
Square pixel equivalent size (arcsec): 0.6310264944404745
NAXIS = 2
NAXIS1 = 3056
NAXIS2 = 3056
CTYPE1 = 'RA---TAN'
CTYPE2 = 'DEC--TAN'
CRVAL1 = 210.754804653
CRVAL2 = 54.4174059435
CRPIX1 = 1528.5
CRPIX2 = 1528.5
CD1_1 = -4.64681630389e-06
CD1_2 = 0.000175223532872
CD2_1 = -0.000175223532872
CD2_2 = -4.64681630389e-06
Pixel size equivalent CDELT1 = 0.631 arcseconds
Pixel size equivalent CDELT2 = 0.631 arcseconds
Image X-axis angular size = 32.140 arcminutes
Pixel Y-axis angular size = 32.140 arcminutes
Image rotation equivalent CROTA2 = 91.519 degrees
END

```

```

4]: def make_dothedhead_from_file(
    input_fits_with_wcs, output_swarp_dothedhead, format="fits", verbose=False
):
    """
    Create the .head format file that swarp expects based on the WCS header of an input file.

    Internally this uses the method from https://docs.astropy.org/en/stable/\_modules/astropy/wcs/wcs.html#WCS.printwcs

    Limitations: Assumes the WCS information is in the primary header

    :param input_fits_with_wcs: Name of existing FITS file with WCS in primary HDU that we want to emulate.
    :param output_swarp_dothedhead:
    :param format: Either 'text' or 'fits'. If 'text' then an ASCII header will be created using the format
        specified in the Swarp manual. If 'fits' is specified, an empty FITS file consisting only of a primary
        HDU will be created.
    :param verbose: If True then diagnostic information will be written to stdout.
    """

    with fits.open(input_fits_with_wcs) as hdulist:
        # Parse the WCS keywords in the primary HDU
        ipwcs = wcs.WCS(hdulist[0].header)

        if verbose:
            print(f"WCS created from primary header of {input_fits_with_wcs}")
            print(ipwcs)
            print(80 * "-")
            # print(ipwcs.wcs)
            # print(80 * "-")

        if "text" in format:
            if verbose:
                print(
                    "Generating an ASCII header following the format specified in the swarp documentation."
                )

        dothead_list = []
        keywords = ["NAXIS", "CTYPE", "CRVAL", "CRPIX"]
        values = [
            ipwcs.array_shape,
            ipwcs.wcs.ctype,
            ipwcs.wcs.crval,
            ipwcs.wcs.crpix,
        ]
    ]

```

```

for keyword, value in zip(keywords, values):
    for idx in range(ipwcs.naxis):
        kw_str = f"{keyword}-{1+idx}"
        if "CTYPE" in keyword:
            # Wrap strings in quotes
            val_str = f"{kw_str:8s} = '{value[idx]}'"
        else:
            val_str = f"{kw_str:8s} = {value[idx]}"
        dothead_list.append(val_str)

if hasattr(ipwcs.wcs, "pc"):
    for irow in range(ipwcs.naxis):
        for jcol in range(ipwcs.naxis):
            kw_str = f"PC{irow+1}_{jcol+1}"
            val_str = f"{kw_str:8s} = {ipwcs.wcs.pc[irow, jcol]}"
            dothead_list.append(val_str)
            kw_str = f"CDELT{1+irow}"
            val_str = f"{kw_str:8s} = {ipwcs.wcs.cdelt[irow]}"
            dothead_list.append(val_str)
elif hasattr(ipwcs.wcs, "cd"):
    for irow in range(ipwcs.naxis):
        for jcol in range(ipwcs.naxis):
            kw_str = f"CD{irow+1}_{jcol+1}"
            val_str = f"{kw_str:8s} = {ipwcs.wcs.cd[irow, jcol]}"
            dothead_list.append(val_str)

dothead_list.append("END ")
dothead_str = "\n".join(dothead_list)

if verbose:
    print(f"--- dothead output from {input_fits_with_wcs} ---")
    print(dothead_str)
    print(f"--- about to write to {output_swarp_dothead} ---")

with open(output_swarp_dothead, "w") as ofile:
    ofile.write(dothead_str)
    if verbose:
        print(
            f"Wrote Swarp ASCII-format .head file to {output_swarp_dothead}"
        )
elif "fits" in format:
    if verbose:
        print(
            "Generating a FITS format header consisting of a PrimaryHDU only."
        )
    # based on https://docs.astropy.org/en/stable/wcs/example_create_imaging.html
    hdr = ipwcs.to_header()

    # NAXIS = x, NAXISx values set from data, not by manipulating header
    olddata = hdulist[0].data
    data = 0 * olddata.astype(int)

    hdu = fits.PrimaryHDU(header=hdr, data=data)
    hdu.writeto(output_swarp_dothead, overwrite=True, output_verify="ignore")
    if verbose:
        print(f"Wrote FITS header .head file to {output_swarp_dothead}")
else:
    raise RuntimeError(
        f'Error, format ({format}) is not one of the allowed options: "text" "fits"'
    )

return

```

```

# Test make_dothead_from_file
ofilename = "M101_Supernova_2023ixf_Lum_4096x4096_resamp_weighted.fits"
oheadname = ofilename.replace(".fits", ".head")
make_dothead_from_file(f_default_match, oheadname, "fits", True)

```

WCS created from primary header of
navigated-M101_Supernova_2023ixf-180s-Lum-1.fits
WCS Keywords

Number of WCS axes: 2
CTYPE : 'RA---TAN' 'DEC--TAN'
CRVAL : 210.754804653 54.4174059435
CRPIX : 1528.5 1528.5

```
CD1_1 CD1_2 : -4.64681630389e-06 0.000175223532872
CD2_1 CD2_2 : -0.000175223532872 -4.64681630389e-06
NAXIS : 3056 3056
```

Generating a FITS format header consisting of a PrimaryHDU only.
Wrote FITS header .head file to
M101_Supernova_2023ixf_Lum_4096x4096_resamp_weighted.head

Resampling Using Python We'll try to recreate the bash shell script resampling of the luminance images, although with the same header as an input image instead of a user-defined pointing. As a reminder the shell command the script used earlier was:

```
swarp ../../NavigatedImages/navigated-M101_Supernova_2023ixf-180s-Lum-3.fits ../../NavigatedImages/navigated-M101_Supernova_2023ixf-180s-Lum-1.fits
```

The `swarp` command line parameters we'll want to change are:

- `CELESTIAL_TYPE`: This needs to be set to native to copy the first input image.
- `VERBOSE_TYPE`: By default `NORMAL`, options are `QUIET`, `LOG`, `NORMAL`, `FULL`.
- `COMBINE_TYPE`: Median is good for a first look but will increase the variance. If possible use `WEIGHTED`
- `PIXEL_SCALE_TYPE`: Various options, see `swarp manual`. Maybe set to `MAX`
 - Note this is ignored if a `.head` file is supplied
- `CENTER_TYPE`: Instead of `MANUAL` we want `MOST` or `ALL`. The latter options also generate output images that are aligned North up, East to the left.
 - Hopefully this is ignored if we specify a `.head`?
- `CENTER` and `PIXEL_SCALE` wont be used if we change `PIXEL_SCALE_TYPE` and `CENTER_TYPE`
- `IMAGE_SIZE`: Use 0 for automatic sizing based on other parameter values
- `SUBTRACT_BACK`: Using `N` is appropriate if we have a lot of nebular emission, crowded fields, large galaxies, or have already performed background subtraction. You should really try with both 'Y' and 'N' to determine the effect of `swarp`'s in-built background subtraction.
- `RESAMPLE_DIR`: Directory where temporary files are written. This must exist.

In the case of the M101 SN premium images `SUBTRACT_BACK` should be `N`. The premium dataset is not a great test of the different center type options because the three input images very closely overlap.

```
6]: # ***You may want to disable this, as it uses assumed file names and directory paths***
# It also assumes a Unix like operating system!!!!
do_simple_test = False
do_better_test = True
do_better_test_center_type = (
    "FIRST" # MANUAL, MOST, ALL, FIRST (MANUAL and ALL are swarp types)
)
dohead_format = "fits" # 'text' or 'fits'

ofilename = "M101_Supernova_2023ixf_Lum_4096x4096_resamp_weighted.fits"
pathlib.Path(ofilename).unlink(missing_ok=True) # remove old file
oheadname = ofilename.replace(".fits", ".head")

if do_simple_test:
    # Make sure to delete any .head file
    pathlib.Path(oheadname).unlink(missing_ok=True)

    # test command, long form, based of the previous example from the resample_all.sh script
    test_cmd = "swarp navigated-M101_Supernova_2023ixf-180s-Lum-1.fits navigated-M101_Supernova_2023ixf-180s-Lum-2.fits
↪navigated-M101_Supernova_2023ixf-180s-Lum-3.fits -VERBOSE_TYPE FULL -SUBTRACT_BACK N -COMBINE_TYPE WEIGHTED -FSCALASTRO_TYPE
↪VARIABLE -VERBOSE_TYPE FULL -GAIN_DEFAULT 1.0 -GAIN_KEYWORD EGAIN -FSSCALE_DEFAULT 0.005556,0.005556,0.005556 -PIXELSCALE_TYPE
↪MANUAL -PIXEL_SCALE 0.62 -CENTER_TYPE MANUAL -CENTER 210.8022671,54.3489500 -RESAMPLING_TYPE LANCZOS3 -OVERSAMPLING 4 -IMAGE_SIZE
↪4096,4096 -PROJECTION_TYPE TAN -IMAGEOUT_NAME M101_Supernova_2023ixf_Lum_4096x4096_resamp_weighted.fits -WRITE_FILEINFO Y -WRITE_XML
↪N -DELETE_TMPFILES Y -RESAMPLE_DIR ./ -WEIGHTOUT_NAME M101_Supernova_2023ixf_Lum_4096x4096_weights_weighted.fits"
    test_cmd_list = shlex.split(test_cmd)
    print(f"Command string to be passed to subprocess.run is {test_cmd_list}")
    fs_tstart = time.perf_counter()
    result = subprocess.run(
        test_cmd_list, check=True, stdout=subprocess.PIPE, stderr=subprocess.PIPE
    )
    fs_tend = time.perf_counter()
    fs_telapsed = fs_tend - fs_tstart # seconds
    print(f" Process took {fs_telapsed:.3f} seconds, return code {result.returncode}")
    print(f" Input args: {result.args}")
    print(f" Stdout: {result.stdout}")
    print(f" Stderr: {result.stderr}")

if do_better_test:
    # test command, split into related sub-sections
```

```

test_cmd1 = "swarp navigated-M101_Supernova_2023ixf-180s-Lum-1.fits navigated-M101_Supernova_2023ixf-180s-Lum-2.fits
→navigated-M101_Supernova_2023ixf-180s-Lum-3.fits -FSCALASTRO_TYPE VARIABLE -FSCALE_DEFAULT 0.005556,0.005556,0.005556"
test_cmd2 = "-VERBOSE_TYPE FULL -SUBTRACT_BACK N -COMBINE_TYPE WEIGHTED -GAIN_DEFAULT 1.0 -GAIN_KEYWORD EGAIN -RESAMPLING_TYPE
→LANCZOS3 -OVERSAMPLING 4 -PROJECTION_TYPE TAN"
if "MANUAL" in do_better_test_center_type:
    pathlib.Path(ohheadname).unlink(missing_ok=True)
    test_cmd3 = "-PIXELSCALE_TYPE MANUAL -PIXEL_SCALE 0.62 -CENTER_TYPE MANUAL -CENTER 210.8022671,54.3489500 -IMAGE_SIZE
→4096,4096"
elif "ALL" in do_better_test_center_type:
    test_cmd3 = "-PIXELSCALE_TYPE MAX -CENTER_TYPE ALL"
    pathlib.Path(ohheadname).unlink(missing_ok=True)
elif "MOST" in do_better_test_center_type:
    test_cmd3 = "-PIXELSCALE_TYPE MAX -CENTER_TYPE MOST"
    pathlib.Path(ohheadname).unlink(missing_ok=True)
elif "FIRST" in do_better_test_center_type:
    # Need to generate a header based on the first file, with a name based on the output file name
    # The pixel scale and centertype should be set from the .head file...
    make_dothhead_from_file(f_default_match, ohheadname, dothead_format, True)
    test_cmd3 = ""
else:
    raise RuntimeError(
        f"Error, center type {do_better_test_center_type} has not yet been implemented."
    )

test_cmd4 = f"-IMAGEOUT_NAME {ofilename} -WRITE_FILEINFO Y -WRITE_XML N -DELETE_TMPFILES Y -RESAMPLE_DIR ./ -WEIGHTOUT_NAME
→M101_Supernova_2023ixf_Lum_4096x4096_weights_weighted.fits"
test_cmd_list = (
    shlex.split(test_cmd1)
    + shlex.split(test_cmd2)
    + shlex.split(test_cmd3)
    + shlex.split(test_cmd4)
)
print(f"Command string to be passed to subprocess.run is {test_cmd_list}")
fs_tstart = time.perf_counter()
try:
    result = subprocess.run(
        test_cmd_list, check=True, stdout=subprocess.PIPE, stderr=subprocess.PIPE
    )
    fs_tend = time.perf_counter()
    fs_telapsed = fs_tend - fs_tstart # seconds
    print(
        f" Success: process took {fs_telapsed:.3f} seconds, return code {result.returncode}"
    )
    # print(f' Input args: {result.args}')
    # print(f' Stdout: {result.stdout}')
    # print(f' Stderr: {result.stderr}')
except subprocess.CalledProcessError as err:
    fs_tend = time.perf_counter()
    fs_telapsed = fs_tend - fs_tstart # seconds
    print(
        f" Error, process took {fs_telapsed:.3f} seconds, return code {err.returncode}"
    )
    print(f" Input args: {err.cmd}\n")
    print(f" Stdout: {err.output}\n")
    print(f" Stderr: {err.stderr}\n")
    print(f" Command line equivalent command: { " ".join(err.cmd)}")

```

WCS created from primary header of
navigated-M101_Supernova_2023ixf-180s-Lum-1.fits
WCS Keywords

Number of WCS axes: 2
CTYPE : 'RA---TAN' 'DEC--TAN'
CRVAL : 210.754804653 54.4174059435
CRPIX : 1528.5 1528.5
CD1_1 CD1_2 : -4.64681630389e-06 0.000175223532872
CD2_1 CD2_2 : -0.000175223532872 -4.64681630389e-06
NAXIS : 3056 3056

Generating a FITS format header consisting of a PrimaryHDU only.
Wrote FITS header .head file to
M101_Supernova_2023ixf_Lum_4096x4096_resamp_weighted.head
Command string to be passed to subprocess.run is ['swarp',
'navigated-M101_Supernova_2023ixf-180s-Lum-1.fits',

```
'navigated-M101_Supernova_2023ixf-180s-Lum-2.fits',
'navigated-M101_Supernova_2023ixf-180s-Lum-3.fits', '-FSCALASTRO_TYPE',
'VARIABLE', '-FSCALE_DEFAULT', '0.005556,0.005556,0.005556', '-VERBOSE_TYPE',
'FULL', '-SUBTRACT_BACK', 'N', '-COMBINE_TYPE', 'WEIGHTED', '-GAIN_DEFAULT',
'1.0', '-GAIN_KEYWORD', 'EGAIN', '-RESAMPLING_TYPE', 'LANCZOS3',
'-OVERSAMPLING', '4', '-PROJECTION_TYPE', 'TAN', '-IMAGEOUT_NAME',
'M101_Supernova_2023ixf_Lum_4096x4096_resamp_weighted.fits', '-WRITE_FILEINFO',
'Y', '-WRITE_XML', 'N', '-DELETE_TMPFILES', 'Y', '-RESAMPLE_DIR', './',
'-WEIGHTOUT_NAME', 'M101_Supernova_2023ixf_Lum_4096x4096_weights_weighted.fits']
    Success: process took 4.685 seconds, return code 0
```

Issues with swarp's ASCII .head file format Information retention: Using swarp's .head format with the END does not seem to work. The following .head should work

```
CTYPE1 = RA---TAN
CTYPE2 = DEC--TAN
CRVAL1 = 210.754804653
CRVAL2 = 54.4174059435
CRPIX1 = 1528.5
CRPIX2 = 1528.5
CD1_1 = -4.64681630389e-06
CD1_2 = 0.000175223532872
CD2_1 = -0.000175223532872
CD2_2 = -4.64681630389e-06
END
```

but we get the following error:

```
----- SWarp 2.38.0 started on 2024-03-13 at 09:04:47 with 16 threads
```

```
Examining input data ...
Looking for navigated-M101_Supernova_2023ixf-180s-Lum-1.fits ...
Looking for navigated-M101_Supernova_2023ixf-180s-Lum-2.fits ...
Looking for navigated-M101_Supernova_2023ixf-180s-Lum-3.fits ...
Creating NEW output image ...
```

```
> *FATAL ERROR*: Unknown FITS type in fitswrite()
```

This is emitted by the fitswrite keyword writing code in src/fits/fitsutil.c.

Trying with an actual FITS header Added an option to make_dothhead_from_file to generate FITS format header. **This works.** Note that swarp can eat inwards from the edge...

Resampling All The Filters Now that we have the method for resampling to match a given image we'll move onto to resampling all the bands (filters) we have.

The new challenge is to compute the appropriate scale factors and which header keywords need to be updated post-resampling (e.g. exposure related keywords, history, and so on). We'll also want some form of summary table.

```
7]: def get_exposure_time(hdr, verbose=False):
    """Return the first of EXPTIME, EXPOSURE, ONTIME, or LIVETIME from a FITS header"""
    keywords = ["EXPTIME", "EXPOSURE", "ONTIME", "LIVETIME"]
    exposure_time = None
    for key in keywords:
        if hdr.count(key) > 0:
            exposure_time = float(hdr[key])
            if verbose:
                print(f"Keyword {key} found, setting exposure_time to {exposure_time}")
            break
    else:
        if verbose:
            print(f"Keyword {key} not found, continuing search...")
    return exposure_time
```

```
8]: do_better_test_center_type = (
    "FIRST" # MANUAL, MOST, ALL, FIRST (MANUAL and ALL are swarp types)
)
dothead_format = "fits" # 'text' or 'fits'
swarp_verbose = False # Echo swarp input string if True, echo .head for FIRST case
final_images = []

for filter in filter_list:
    print(f"Processing images from filter {filter}")

    # Names for output resampled image, net weights, and .head files
    ofilename = f"M101_Supernova_2023ixf_{filter}_resamp_weighted.fits"
    owgtsname = f"M101_Supernova_2023ixf_{filter}_weights_weighted.fits"
```

```

pathlib.Path(ofilename).unlink(missing_ok=True) # remove old file
oheadname = ofilename.replace(".fits", ".head")
pathlib.Path(oheadname).unlink(missing_ok=True) # remove old file

filtered_netexp = 0
filtered_numimg = 0
inp_weights = []
inp_files = []

for hdu, fname in ifc_nav.hdus(return_fname=True, filter=filter):
    inp_files.append(fname)
    texp = get_exposure_time(hdu.header)
    if texp is None:
        raise RuntimeError(f"Error, could not get exposure time from {fname}")
    fscale = 1.0 / texp
    inp_weights.append(f"{fscale:.7f}")
    filtered_netexp += texp
    filtered_numimg += 1

if len(inp_files) == 0:
    print(f" No files found for filter {filter}")
    continue
else:
    fs_tstart = time.perf_counter()
    print(f" {filter} {filtered_numimg} {filtered_netexp}")
    print(f" inp_files: {inp_files}")
    print(f" inp_weights: {inp_weights}")

    file_str = " ".join(inp_files)
    fscale_str = " ".join(inp_weights)
    test_cmd1 = (
        f"swarp {file_str} -FSCALASTRO_TYPE VARIABLE -FSCALE_DEFAULT {fscale_str}"
    )
    test_cmd2 = "-VERBOSE_TYPE FULL -SUBTRACT_BACK N -COMBINE_TYPE WEIGHTED -GAIN_DEFAULT 1.0 -GAIN_KEYWORD EGAIN -RESAMPLING_TYPE_
↳LANCZOS3 -OVERSAMPLING 4 -PROJECTION_TYPE TAN"
    if "MANUAL" in do_better_test_center_type:
        pathlib.Path(oheadname).unlink(missing_ok=True)
        test_cmd3 = "-PIXELSCALE_TYPE MANUAL -PIXEL_SCALE 0.62 -CENTER_TYPE MANUAL -CENTER 210.8022671,54.3489500 -IMAGE_SIZE_
↳4096,4096"
    elif "ALL" in do_better_test_center_type:
        test_cmd3 = "-PIXELSCALE_TYPE MAX -CENTER_TYPE ALL"
        pathlib.Path(oheadname).unlink(missing_ok=True)
    elif "MOST" in do_better_test_center_type:
        test_cmd3 = "-PIXELSCALE_TYPE MAX -CENTER_TYPE MOST"
        pathlib.Path(oheadname).unlink(missing_ok=True)
    elif "FIRST" in do_better_test_center_type:
        # Need to generate a header based on the first file, with a name based on the output file name
        # The pixel scale and centertype should be set from the .head file...
        make_dothhead_from_file(
            f_default_match, oheadname, dothead_format, swarp_verbose
        )

        # swarp does honor the imagesize if set in a FITS format .head file, but not if test format.
        test_cmd3 = ""
    else:
        raise RuntimeError(
            f"Error, center type {do_better_test_center_type} has not yet been implemented."
        )

    test_cmd4 = f"-IMAGEOUT_NAME {ofilename} -WRITE_FILEINFO Y -WRITE_XML N -DELETE_TMPFILES Y -RESAMPLE_DIR ./ -WEIGHTOUT_NAME_
↳{owgtsname}"
    test_cmd_list = (
        shlex.split(test_cmd1)
        + shlex.split(test_cmd2)
        + shlex.split(test_cmd3)
        + shlex.split(test_cmd4)
    )
    if swarp_verbose:
        print(f"Command string to be passed to subprocess.run is {test_cmd_list}")

    # Run swarp
    fs_tstart = time.perf_counter()
    try:
        result = subprocess.run(
            test_cmd_list,

```



```

        check=True,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
    )
    fs_tend = time.perf_counter()
    fs_telapsed = fs_tend - fs_tstart # seconds
    print(
        f" Success: process took {fs_telapsed:.3f} seconds, return code {result.returncode}"
    )
    final_images.append(ofilename)

except subprocess.CalledProcessError as err:
    fs_tend = time.perf_counter()
    fs_telapsed = fs_tend - fs_tstart # seconds
    print(
        f" Error, process took {fs_telapsed:.3f} seconds, return code {err.returncode}"
    )
    print(f" Input args: {err.cmd}\n")
    print(f" Stdout: {err.output}\n")
    print(f" Stderr: {err.stderr}\n")
    print(f' Command line equivalent command: {" ".join(err.cmd)}')

# Generate a summary
print(f"Finished. Generated {len(final_images)} resampled images:")
show_full_wcs = True
for img in final_images:
    extnum = 0
    hdu = fits.open(img)[extnum]
    w = wcs.WCS(hdu.header)
    naxis1 = hdu.header["NAXIS1"]
    naxis2 = hdu.header["NAXIS2"]
    print(f" {img} naxis1={naxis1} naxis2={naxis1}")
    if show_full_wcs:
        print(w)
        print(80 * "-")

```

```

Processing images from filter Blue
Blue 3 900.0
    inp_files: ['navigated-M101_Supernova_2023ixf-300s-Blue-1.fits',
'navigated-M101_Supernova_2023ixf-300s-Blue-2.fits',
'navigated-M101_Supernova_2023ixf-300s-Blue-3.fits']
    inp_weights: ['0.0033333', '0.0033333', '0.0033333']
    Success: process took 4.606 seconds, return code 0
Processing images from filter Green
Green 3 900.0
    inp_files: ['navigated-M101_Supernova_2023ixf-300s-Green-1.fits',
'navigated-M101_Supernova_2023ixf-300s-Green-2.fits',
'navigated-M101_Supernova_2023ixf-300s-Green-3.fits']
    inp_weights: ['0.0033333', '0.0033333', '0.0033333']
    Success: process took 4.614 seconds, return code 0
Processing images from filter Lum
Lum 3 540.0
    inp_files: ['navigated-M101_Supernova_2023ixf-180s-Lum-1.fits',
'navigated-M101_Supernova_2023ixf-180s-Lum-2.fits',
'navigated-M101_Supernova_2023ixf-180s-Lum-3.fits']
    inp_weights: ['0.0055556', '0.0055556', '0.0055556']
    Success: process took 4.662 seconds, return code 0
Processing images from filter Red
Red 3 900.0
    inp_files: ['navigated-M101_Supernova_2023ixf-300s-Red-1.fits',
'navigated-M101_Supernova_2023ixf-300s-Red-2.fits',
'navigated-M101_Supernova_2023ixf-300s-Red-3.fits']
    inp_weights: ['0.0033333', '0.0033333', '0.0033333']
    Success: process took 4.627 seconds, return code 0
Finished. Generated 4 resampled images:
    M101_Supernova_2023ixf_Blue_resamp_weighted.fits naxis1=3056 naxis2=3056
WCS Keywords

```

```

Number of WCS axes: 2
CTYPE : 'RA---TAN' 'DEC--TAN'
CRVAL : 210.754804653 54.4174059435
CRPIX : 1528.5 1528.5
PC1_1 PC1_2 : -4.64681630389e-06 0.000175223532872
PC2_1 PC2_2 : -0.000175223532872 -4.64681630389e-06
CDELT : 1.0 1.0
NAXIS : 3056 3056

```

```
-----
M101_Supernova_2023ixf_Green_resamp_weighted.fits  naxis1=3056  naxis2=3056
WCS Keywords

Number of WCS axes: 2
CTYPE : 'RA---TAN' 'DEC--TAN'
CRVAL : 210.754804653 54.4174059435
CRPIX : 1528.5 1528.5
PC1_1 PC1_2 : -4.64681630389e-06 0.000175223532872
PC2_1 PC2_2 : -0.000175223532872 -4.64681630389e-06
CDELT : 1.0 1.0
NAXIS : 3056 3056
-----
```

```
-----
M101_Supernova_2023ixf_Lum_resamp_weighted.fits  naxis1=3056  naxis2=3056
WCS Keywords

Number of WCS axes: 2
CTYPE : 'RA---TAN' 'DEC--TAN'
CRVAL : 210.754804653 54.4174059435
CRPIX : 1528.5 1528.5
PC1_1 PC1_2 : -4.64681630389e-06 0.000175223532872
PC2_1 PC2_2 : -0.000175223532872 -4.64681630389e-06
CDELT : 1.0 1.0
NAXIS : 3056 3056
-----
```

```
-----
M101_Supernova_2023ixf_Red_resamp_weighted.fits  naxis1=3056  naxis2=3056
WCS Keywords

Number of WCS axes: 2
CTYPE : 'RA---TAN' 'DEC--TAN'
CRVAL : 210.754804653 54.4174059435
CRPIX : 1528.5 1528.5
PC1_1 PC1_2 : -4.64681630389e-06 0.000175223532872
PC2_1 PC2_2 : -0.000175223532872 -4.64681630389e-06
CDELT : 1.0 1.0
NAXIS : 3056 3056
-----
```

3.1.3 Inspecting The Resampled Images

We can load one or more of the resampled images and plot them... Here we'll just pick the first one in the `final_images` list.

```
9] from astropy import units
from astropy.visualization import AsinhStretch, ImageNormalize, ManualInterval

def load_image_and_plot(
    fname,
    extnum=0,
    usewcs=True,
    vmin=None,
    vmax=None,
    xaxlim=None,
    yaxlim=None,
    verbose=True,
    angle_tick_spacing_am=2.0,
    swap_radec_axis=False,
):
    """
    Quick and dirty FITS image plot.

    By default `asinh` scaling will be used between a minimum and maximum data value. If vmin
    and/or vmmax are not specified then the 0.5th and 99.5th perciles will be used, as appropriate.
    A viridis color map is used by default, arag with a color bar.

    Note
    ----
    Axis limits (xaxlim and yaxlim) are currently defined in image x-axis and y-axis
    pixels.

    Astropy WCSaxes transforms do not reorient the image to be North up, East left, in the same
    way SAOImage ds9 does when the WCS is applied. It still plots the original data rows and
    column as a 2-D XY grid. Consequently, for images not already in NE alignment, the RA may
    change most rapidly along Y and Dec most rapidly along X, in contrast to normal expectation.
    To avoid highly confusing plots this function plots RA/Dec grids when `usewcs=True`. Line
    of constant RA are red, lines of constant Declination are blue. In cases where your data
```

is aligned closer to 90 degrees or 270 degrees away from North up, East left, specifying ``swap_radec_axis=True`` will reduce confusion by showing RA tickmarks and values along the Y axes and Declination tickmarks and values along the X axis (contrary to the normal convention).

If your ``usewcs=True`` plot appears without axis tick values and labels then it is likely you need to alter ``swap_radec_axis`` or ``angle_tick_spacing_am``

TODO: Find out how to specify them in RA and Dec.

Parameters

```
-----
:param fname: Name of existing FITS file with WCS in HDU number extnum that we want to plot.
:param extnum: Extension number (zero-based) for data and WCS header
:param usewcs: If True then plot using WCS information
:param vmin: If not None then vmin is the minimum value in the normalization interval.
:param vmax: If not None then vmax is the maximum value in the normalization interval.
:param xaxlim: 2-element list or tuple of the minimum to maximum x-axis coordinates to
    plot. If None then the default axis limits will be used. To see those limits run
    with verbose=True.
:param yaxlim: 2-element list or tuple of the minimum to maximum y-axis coordinates to
    plot. If None then the default axis limits will be used. To see those limits run
    with verbose=True.
:param verbose: If True then diagnostic information will be written to stdout.
:param angle_tick_spacing_am: If usewcs is True, specify the spacing between RA/Dec axis
    tick values in units of arcminutes. Astropy won't label the RA/Dec axis without
    specifying a value for this.
:param swap_radec_axis: If True then plot RA tickmarks and values along the Y
    axes and Declination tickmarks and values along the X axis (contrary to the normal
    convention). This does not alter how the image data itself is plotted.
"""
```

```
hdu = fits.open(fname)[extnum]
w = wcs.WCS(hdu.header)

# Compute vmin and vmax if necessary
# percentiles
ipctls = [0.5, 99.5]
opctls = np.nanpercentile(hdu.data, ipctls)
ourmin = opctls[0]
ourmax = opctls[1]
if verbose:
    print(
        f"Input data 0.5th percentile = {ourmin:.4f}, 99.5th percentile = {ourmax:.4f}"
    )

if vmin is not None:
    ourmin = vmin
if vmax is not None:
    ourmax = vmax
if verbose:
    print(f"Applying asinh stretch between {ourmin:.4f} and {ourmax:.4f}")

# Create an ImageNormalize object
norm = ImageNormalize(
    hdu.data, interval=ManualInterval(ourmin, ourmax), stretch=AsinhStretch()
)

# Display the image
fig = plt.figure()
if usewcs:
    if verbose:
        print("Using WCS information for axis projection")
        print(w)
    ax = fig.add_subplot(1, 1, 1, projection=w)
else:
    ax = fig.add_subplot(1, 1, 1)

im = ax.imshow(hdu.data, origin="lower", norm=norm)
cbar = fig.colorbar(
    im,
    ax=ax,
    extend="neither",
    spacing="proportional",
    orientation="vertical",
    shrink=0.85,
```

```

)
cbar.set_label(r"Units TBA")
cbar.ax.tick_params(labelsize=8)
title_str = fname # .replace("_", "\_")
ax.set_title(f"{title_str}", fontsize=8)

# Display default axis limits
xlim_used = ax.get_xbound()
ylim_used = ax.get_ybound()
if verbose:
    print(f"Default X-axis limits: {xlim_used}")
    print(f"Default Y-axis limits: {ylim_used}")

if usewcs:
    # Adapted from @astrofrog at https://github.com/astropy/astropy/issues/13458#issuecomment-1242640539
    ra = ax.coords[0]
    dec = ax.coords[1]
    ra.set_major_formatter("dd:mm:ss.ss")
    dec.set_major_formatter("dd:mm:ss.ss")
    ra.set_ticks(
        spacing=angle_tick_spacing_am * units.arcmin, color="red"
    ) # Ticks must be defined for axis tickvals to appear
    dec.set_ticks(spacing=angle_tick_spacing_am * units.arcmin, color="blue")
    ra.set_ticklabel(color="red", fontsize=8)
    dec.set_ticklabel(color="blue", fontsize=8)
    ra.grid(color="red", linestyle="--", alpha=0.6)
    dec.grid(color="blue", linestyle="--", alpha=0.6)
    ra.set_xlabel("Right Ascension (dms)", fontsize=8, color="red")
    dec.set_xlabel("Declination (dms)", fontsize=8, color="blue")

    if swap_radec_axis:
        # Images where ra changes fastest on Y, not X
        dec.set_ticks_position("b")
        dec.set_ticklabel_position("b")
        dec.set_xlabel_position("b")
        ra.set_ticks_position("l")
        ra.set_ticklabel_position("l")
        ra.set_xlabel_position("l")

    # ax.set_xlabel('Right Ascension (deg)', fontsize=8)
    # ax.set_ylabel('Declination (deg)', fontsize=8)
else:
    ax.set_xlabel("X-axis pixel number", fontsize=8)
    ax.set_ylabel("Y-axis pixel number", fontsize=8)

# Modify the axis limits?
if xaxlim is not None:
    loval = np.min(xaxlim)
    hival = np.max(xaxlim)
    ax.set_xbound(lower=loval, upper=hival)
    if verbose:
        print(f"Setting X-axis limits to [{loval}, {hival}]")
if yaxlim is not None:
    loval = np.min(yaxlim)
    hival = np.max(yaxlim)
    ax.set_ybound(lower=loval, upper=hival)
    if verbose:
        print(f"Setting Y-axis limits to [{loval}, {hival}]")
return

```

0] : # 2024-08-23 Something weird with matplotlib plots
 # Basically they don't rotate the image to N up E left the way ds9 does,
 # so the coord labelling is odd
 # see <https://github.com/astropy/astropy/issues/13458>

WARNING

Astropy's WCS axis plotting projection code does *NOT* work the way you might expect based on other FITS viewers.

The image itself is not projected to align with the Right Ascension/Declination coordinate system (*North up, East left), e.g. the way `ds9`'s Align mode works. Instead, the axis tickmark labeling and any over-plotted grid is aware of the actual image WCS, and can correctly label and mark the RA and Dec values. This isn't apparent in most cases because many images are provided with the data already in North up and East left mode.

See:

- Astropy issue discussing this: <https://github.com/astropy/astropy/issues/13458> This also discussed getting the axis labels and tickmarks right
- Astropy request for it to do the right thing: <https://github.com/astropy/astropy/issues/8423>

- Work-around using montage to resample the data to NE-aligned: <https://stackoverflow.com/questions/51952436/plotting-sdss-images-with-python/51967697#51967697>

```
1]: print(f"Final images: {final_images}")
output_plot = "delme.png"
ap.util.load_image_and_plot(
    final_images[0],
    0,
    output_plot,
    True,
    angle_tick_spacing_am=10,
    swap_ra_dec_axis=True,
)
```

Final images: ['M101_Supernova_2023ixf_Blue_resamp_weighted.fits',
'M101_Supernova_2023ixf_Green_resamp_weighted.fits',
'M101_Supernova_2023ixf_Lum_resamp_weighted.fits',
'M101_Supernova_2023ixf_Red_resamp_weighted.fits']

Input data 0.5th percentile = 0.5249, 99.5th percentile = 0.8013

Applying asinh stretch between 0.5249 and 0.8013

Using WCS information for axis projection

WCS Keywords

Number of WCS axes: 2

CTYPE : 'RA---TAN' 'DEC--TAN'

CRVAL : 210.754804653 54.4174059435

CRPIX : 1528.5 1528.5

PC1_1 PC1_2 : -4.64681630389e-06 0.000175223532872

PC2_1 PC2_2 : -0.000175223532872 -4.64681630389e-06

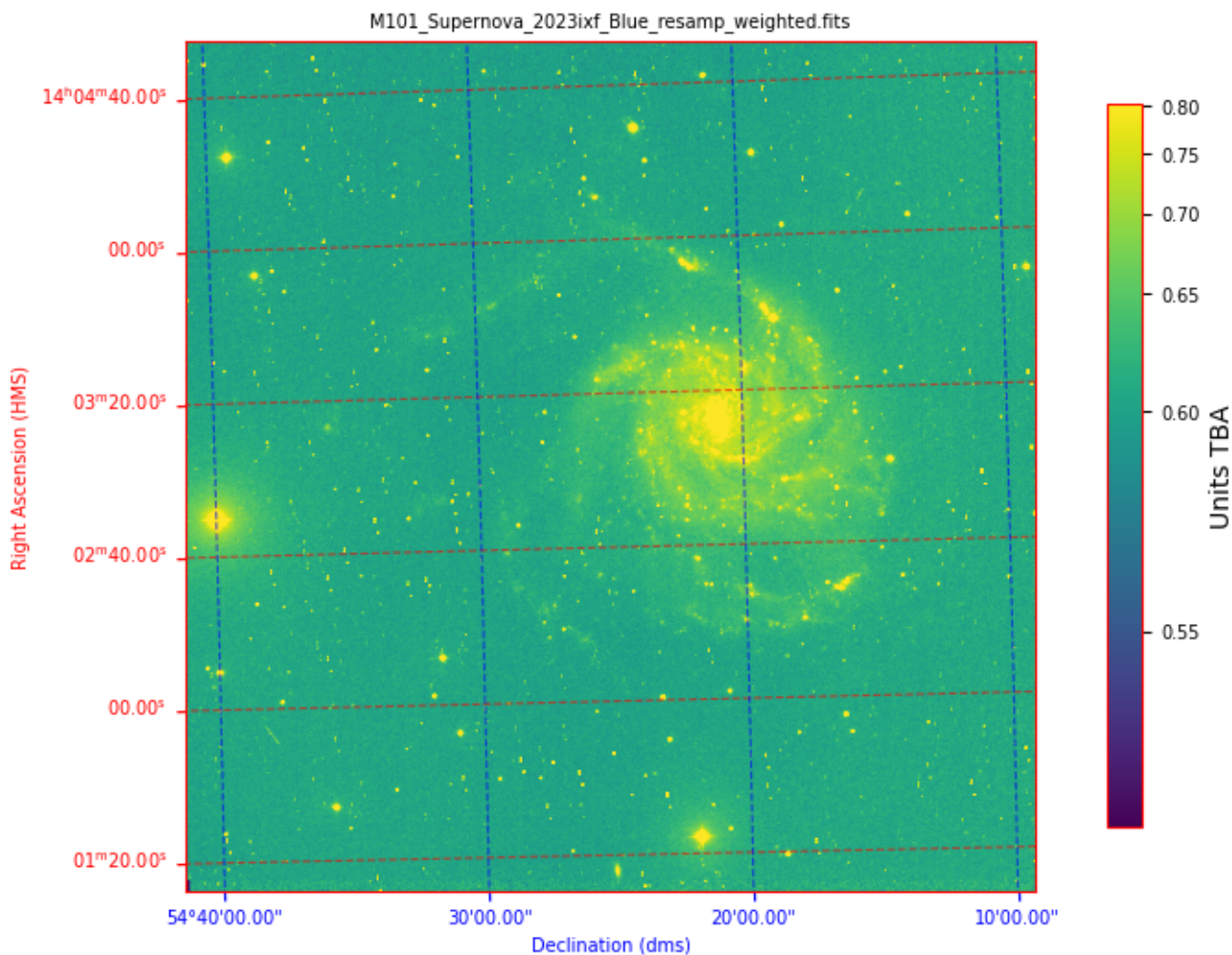
CDELT : 1.0 1.0

NAXIS : 3056 3056

Default X-axis limits: (-0.5, 3055.5)

Default Y-axis limits: (-0.5, 3055.5)

Wrote plot to delme.png

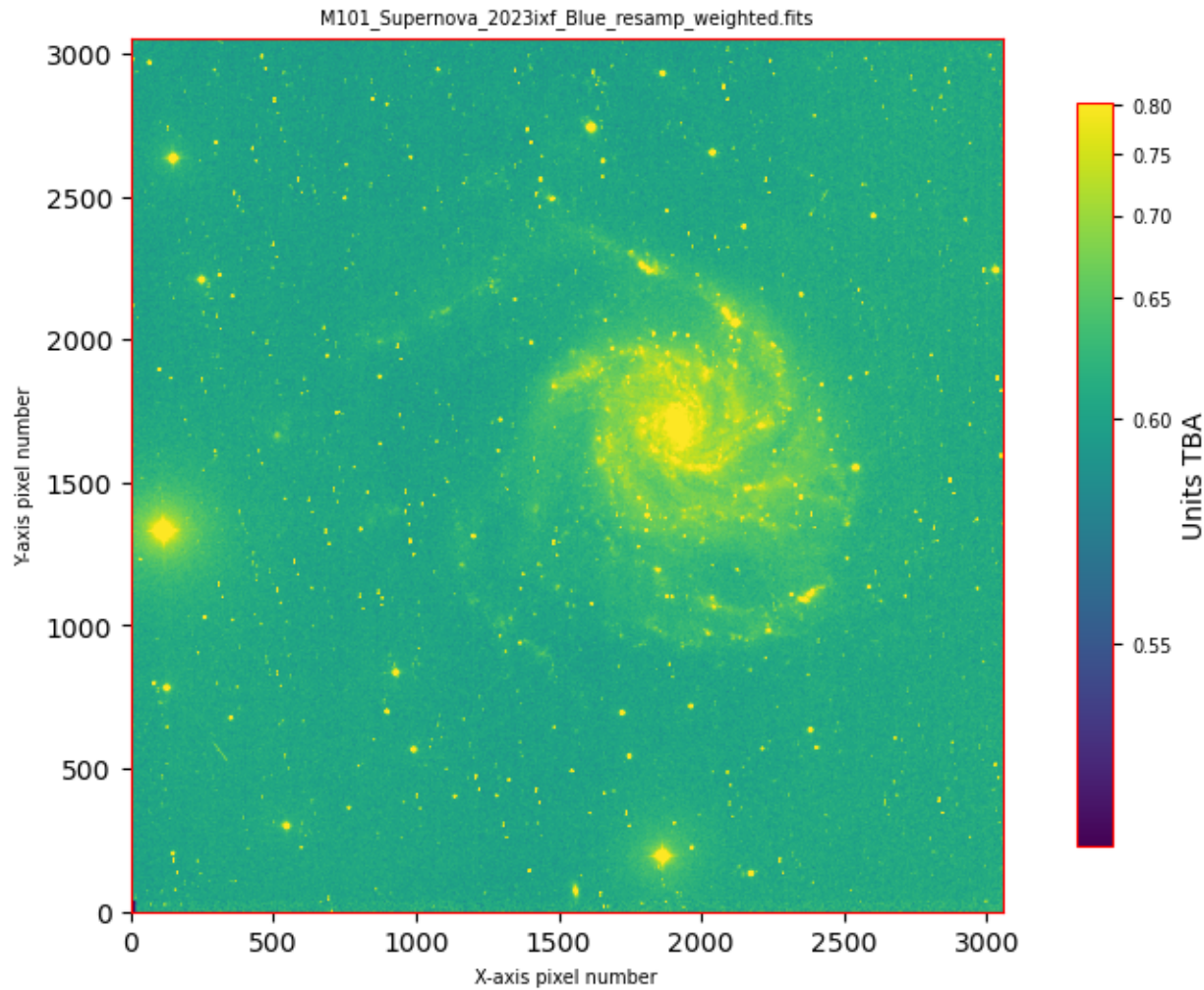


Axis limits: Note that the axis limits are in terms of pixel coordinates, not astronomical RA or Dec.

For now, xaxlim and yaxlim passed to load_image_and_plot must be in pixels.

```
2]: # We can plot without the WCS axes, just raw pixels..
output_plot = None
ap.util.load_image_and_plot(final_images[0], 0, output=output_plot, usewcs=False)
```

Input data 0.5th percentile = 0.5249, 99.5th percentile = 0.8013
Applying asinh stretch between 0.5249 and 0.8013
Default X-axis limits: (-0.5, 3055.5)
Default Y-axis limits: (-0.5, 3055.5)



So what do we see here?

Problem 1: Despite specifying a WCS header with a non-zero rotation it appears that **swarp** will always force A North-up, East to the left alignment. Looking at the header of any of the resampled files we can see

```
SIMPLE = T / This is a FITS file
BITPIX = -32 /
NAXIS = 2 /
NAXIS1 = 3326 / NUMBER OF ELEMENTS ALONG THIS AXIS
NAXIS2 = 3301 / NUMBER OF ELEMENTS ALONG THIS AXIS
EXTEND = T / This file may contain FITS extensions
EQUINOX = 2000.00000000 / Mean equinox
RADESYS = 'ICRS' / Astrometric system
CTYPE1 = 'RA---TAN' / WCS projection type for this axis
CUNIT1 = 'deg' / Axis unit
CRVAL1 = 2.107644916490E+02 / World coordinate on this axis
CRPIX1 = 1.663500000000E+03 / Reference pixel on this axis
CD1_1 = -1.752763491822E-04 / Linear projection matrix
```

```

CD1_2 = 0.000000000000E+00 / Linear projection matrix
CTYPE2 = 'DEC--TAN' / WCS projection type for this axis
CUNIT2 = 'deg' / Axis unit
CRVAL2 = 5.441565675623E+01 / World coordinate on this axis
CRPIX2 = 1.651000000000E+03 / Reference pixel on this axis
CD2_1 = 0.000000000000E+00 / Linear projection matrix
CD2_2 = 1.752763491822E-04 / Linear projection matrix

```

which differs from the header we specified in the .head file, e.g. in M101_Supernova_2023ixf_Green_resamp_weighted.head

```

fitsheader M101_Supernova_2023ixf_Green_resamp_weighted.head
# HDU 0 in M101_Supernova_2023ixf_Green_resamp_weighted.head:
SIMPLE = T / conforms to FITS standard
BITPIX = 64 / array data type
NAXIS = 2 / number of array dimensions
NAXIS1 = 3056
NAXIS2 = 3056
WCSAXES = 2 / Number of coordinate axes
CRPIX1 = 1528.5 / Pixel coordinate of reference point
CRPIX2 = 1528.5 / Pixel coordinate of reference point
PC1_1 = -4.64681630389E-06 / Coordinate transformation matrix element
PC1_2 = 0.000175223532872 / Coordinate transformation matrix element
PC2_1 = -0.000175223532872 / Coordinate transformation matrix element
PC2_2 = -4.64681630389E-06 / Coordinate transformation matrix element
CDELT1 = 1.0 / [deg] Coordinate increment at reference point
CDELT2 = 1.0 / [deg] Coordinate increment at reference point
CUNIT1 = 'deg' / Units of coordinate increment and value
CUNIT2 = 'deg' / Units of coordinate increment and value
CTYPE1 = 'RA---TAN' / Right ascension, gnomonic projection
CTYPE2 = 'DEC--TAN' / Declination, gnomonic projection
CRVAL1 = 210.754804653 / [deg] Coordinate value at reference point
CRVAL2 = 54.4174059435 / [deg] Coordinate value at reference point
LONPOLE = 180.0 / [deg] Native longitude of celestial pole
LATPOLE = 54.4174059435 / [deg] Native latitude of celestial pole
MJDREF = 0.0 / [d] MJD of fiducial time
RADESYS = 'FK5' / Equatorial coordinate system
EQUINOX = 2000.0 / [yr] Equinox of equatorial coordinates

```

Problem 2: The sky background in the image is really bright (approx 0.6 per pixel) compared to the maximum data values from the stars and galaxy (about 0.78 per pixel at the 99.5th percentile). We can plot using a hand-chosen minimum, e.g.

```

3]: ap.util.load_image_and_plot(
    final_images[0],
    0,
    output_plot,
    True,
    vmin=0.6,
    vmax=None,
    angle_tick_spacing_am=10,
    swap_radec_axis=True,
)

```

Input data 0.5th percentile = 0.5249, 99.5th percentile = 0.8013

Applying asinh stretch between 0.6000 and 0.8013

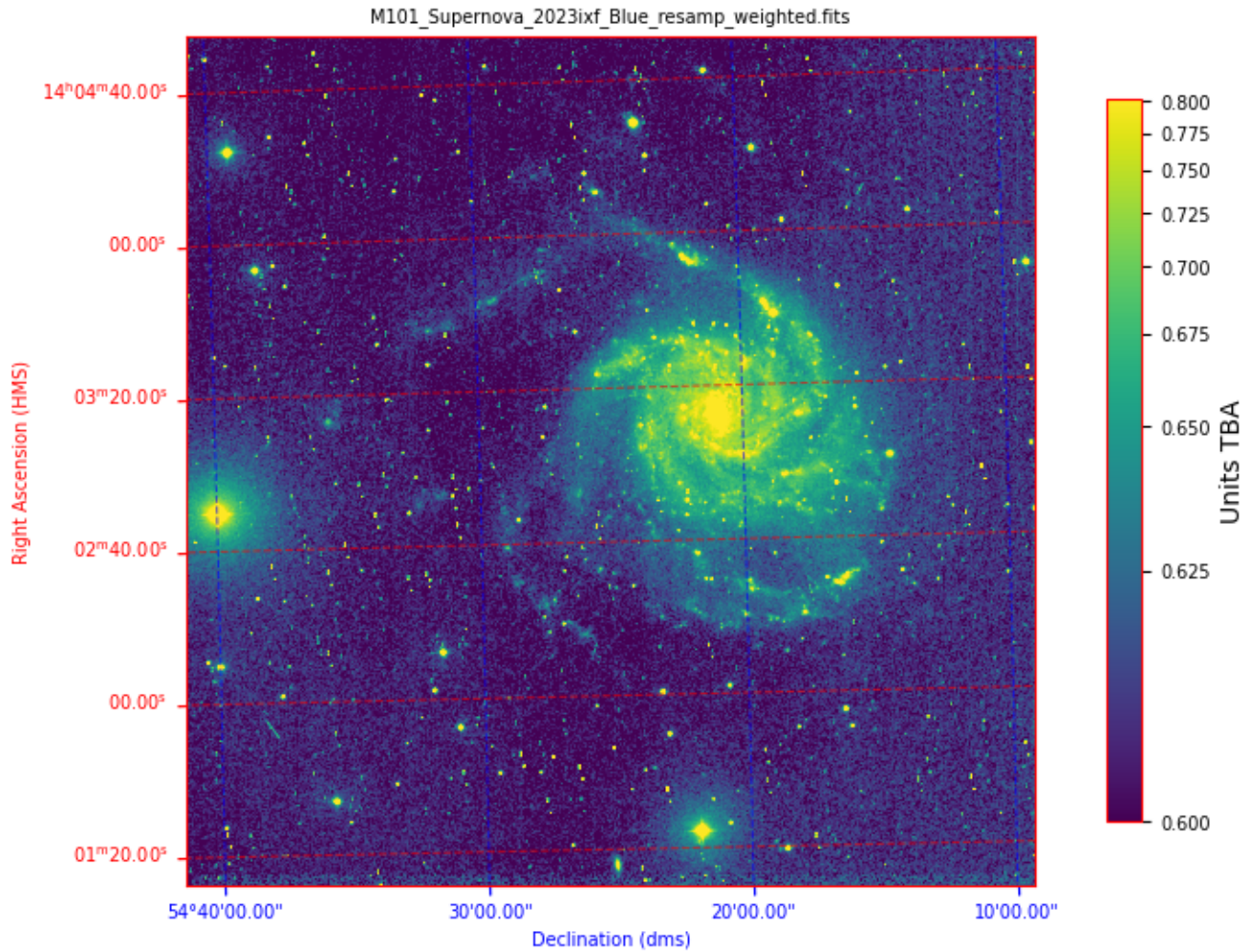
Using WCS information for axis projection

WCS Keywords

```

Number of WCS axes: 2
CTYPE : 'RA---TAN' 'DEC--TAN'
CRVAL : 210.754804653 54.4174059435
CRPIX : 1528.5 1528.5
PC1_1 PC1_2 : -4.64681630389e-06 0.000175223532872
PC2_1 PC2_2 : -0.000175223532872 -4.64681630389e-06
CDELT : 1.0 1.0
NAXIS : 3056 3056
Default X-axis limits: (-0.5, 3055.5)
Default Y-axis limits: (-0.5, 3055.5)

```



...at which point we can start to see signs of the third problem.

Zooming in on a smaller region we see a repeated pattern of two bright points, separated by approximately the maximum offset between the raw frames used to make the composite image. The points are also more compact than real stars in the image.

```
4]: ap.util.load_image_and_plot(
    final_images[0],
    0,
    output_plot,
    True,
    vmin=0.6,
    vmax=1.0,
    xaxlim=[1100, 1500],
    yaxlim=[350, 750],
    verbose=True,
    angle_tick_spacing_am=2,
    swap_radec_axis=True,
)
```

Input data 0.5th percentile = 0.5249, 99.5th percentile = 0.8013

Applying asinh stretch between 0.6000 and 1.0000

Using WCS information for axis projection

WCS Keywords

Number of WCS axes: 2

CTYPE : 'RA---TAN' 'DEC--TAN'

CRVAL : 210.754804653 54.4174059435

CRPIX : 1528.5 1528.5

PC1_1 PC1_2 : -4.64681630389e-06 0.000175223532872

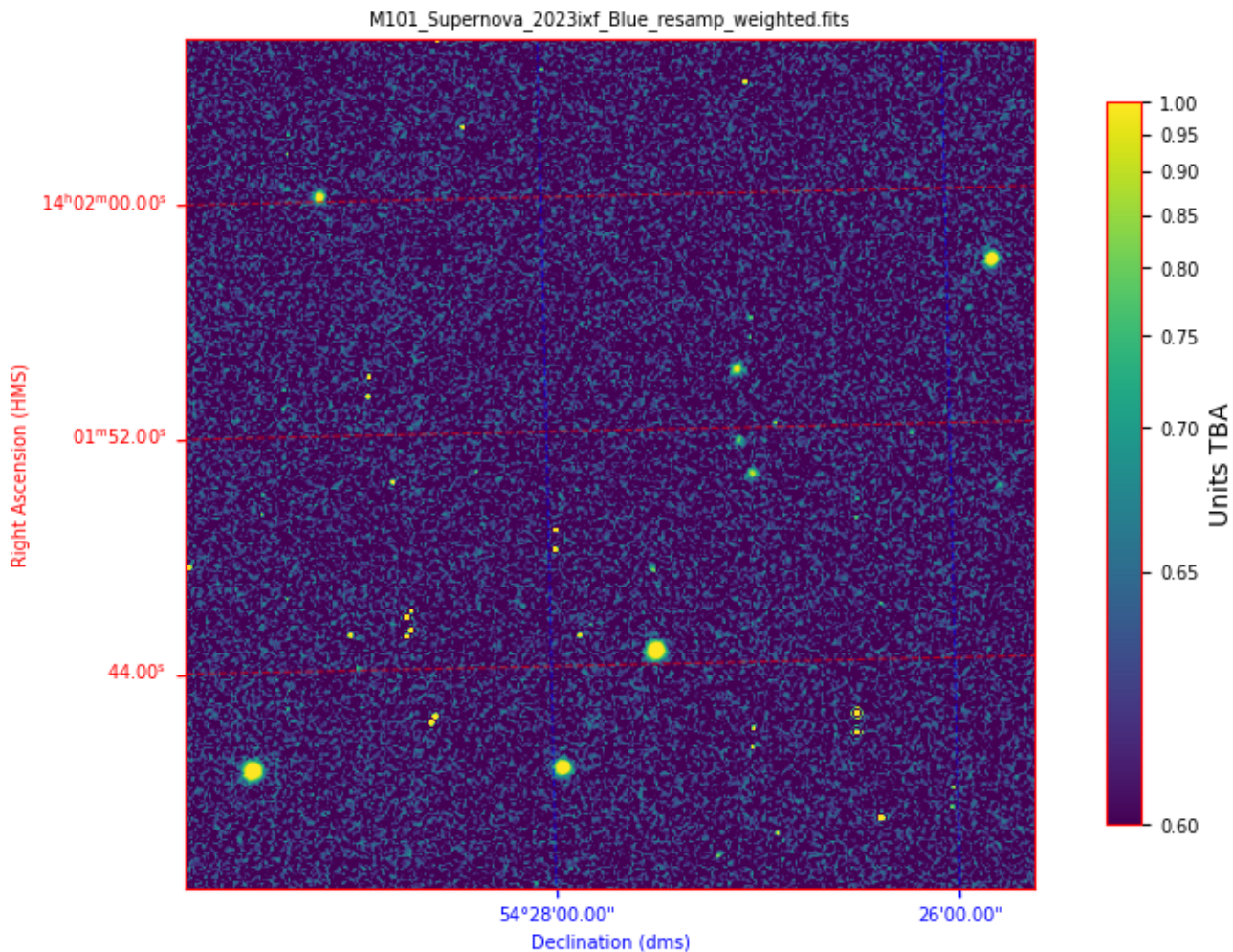
PC2_1 PC2_2 : -0.000175223532872 -4.64681630389e-06

CDELT : 1.0 1.0

NAXIS : 3056 3056

Default X-axis limits: (-0.5, 3055.5)

Default Y-axis limits: (-0.5, 3055.5)
 Setting X-axis limits to [1100, 1500]
 Setting Y-axis limits to [350, 750]

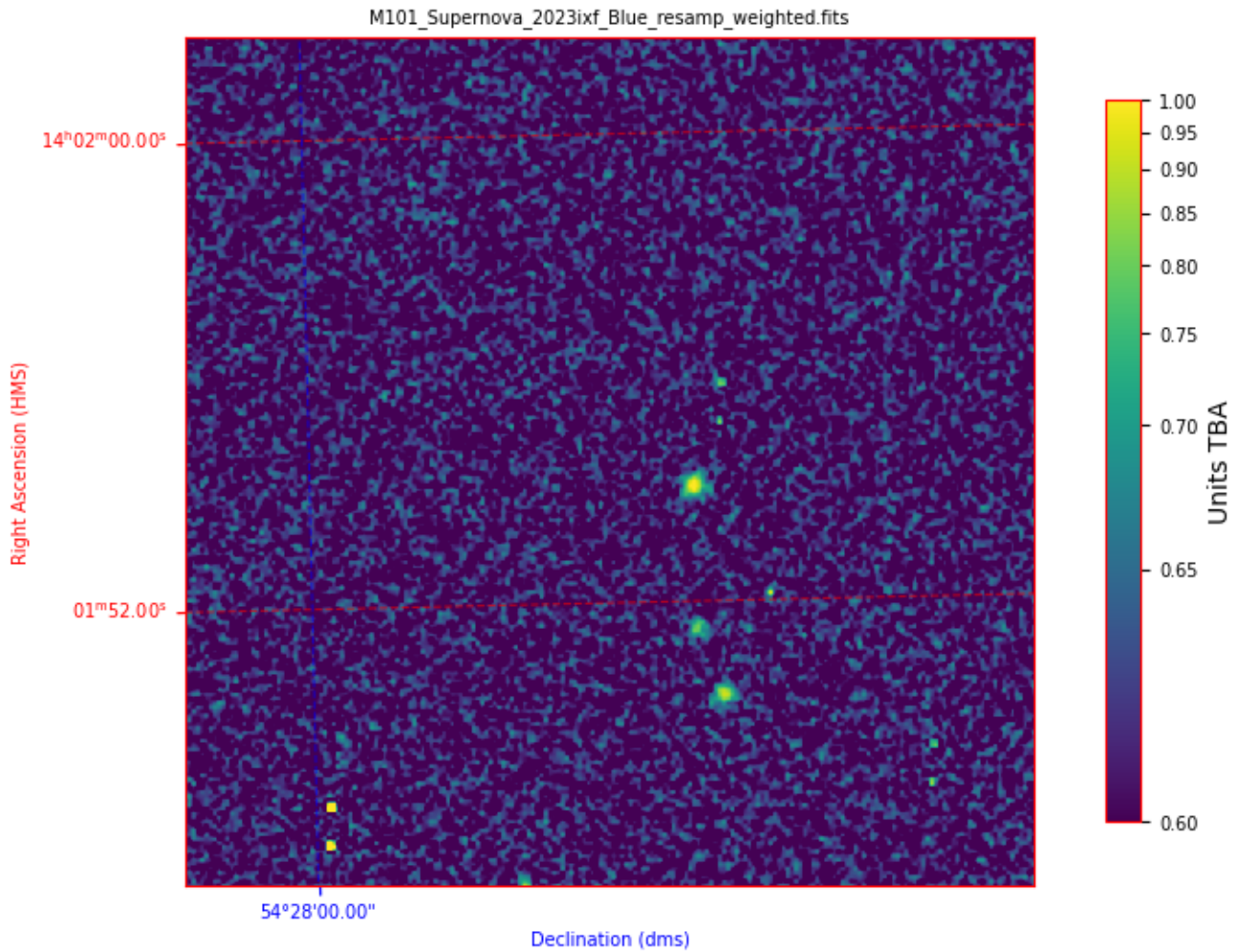


```
5]: # Zoom in even further
ap.util.load_image_and_plot(
    final_images[0],
    0,
    output_plot,
    True,
    vmin=0.6,
    vmax=1.0,
    xaxlim=[1240, 1440],
    yaxlim=[500, 700],
    verbose=True,
    angle_tick_spacing_am=2,
    swap_radec_axis=True,
)
```

Input data 0.5th percentile = 0.5249, 99.5th percentile = 0.8013
 Applying asinh stretch between 0.6000 and 1.0000
 Using WCS information for axis projection
 WCS Keywords

Number of WCS axes: 2
 CTYPE : 'RA---TAN' 'DEC--TAN'
 CRVAL : 210.754804653 54.4174059435
 CRPIX : 1528.5 1528.5
 PC1_1 PC1_2 : -4.64681630389e-06 0.000175223532872
 PC2_1 PC2_2 : -0.000175223532872 -4.64681630389e-06
 CDELT : 1.0 1.0
 NAXIS : 3056 3056

Default X-axis limits: (-0.5, 3055.5)
 Default Y-axis limits: (-0.5, 3055.5)
 Setting X-axis limits to [1240, 1440]
 Setting Y-axis limits to [500, 700]



So we have uncorrected hot pixels!

For now we'll ignore that, and move on.

3.2 Three color composite images

There are two ways we can generate 3-color composite images from the navigated resampled and combined images.

One option is to use Astropy's native `make_lupton_rgb` function. This is based on [Lupton et al 2004](#). The advantages to this algorithm are it uses the `asinh` stretch function, arguably the most versatile stretch and the color is unique and not brightness dependent. The downside to this is the lack of exact control over the scaling options, in particular per-channel minimum and maximum values, and also the lack of a 16-bit color depth option. It also has a builtin luminance function $L = (R + G + B)/3$, which is neither human eye equivalent nor adjustable. In practice Q is typically around 10 for good-looking images, and stretch is typically less than 1 (e.g. 0.5, but 0.02 in some cases).

The second option is to install Astromatic `stiff` and call it via the subprocess module in the same way we used `swarp`. This gives us more options.

```
5]: # First, lets just plot each resampled image separately
def plot_threecolor(
    redfile,
    greenfile,
    bluefile,
    extnum=0,
    usewcs=True,
    vmin=None,
    vmax=None,
    xaxlim=None,
    yaxlim=None,
    verbose=True,
):
```

```

"""
Quick and dirty FITS image plot.

By default `asinh` scaling will be used between a minimum and maximum data value. If vmin
and/or vmax are not specified then the 0.5th and 99.5th perciles will be used, as appropriate.
A viridis color map is used by default, along with a color bar.

Note that axis limits (xaxlim and yaxlim) are currently defined in image x-axis and y-axis
pixels.

TODO: Find out how to specify them in RA and Dec.

:param redfile: Name of existing FITS file with WCS in HDU number extnum that we want to
    appear as red in the RGB color composite.
:param greenfile: Name of existing FITS file with WCS in HDU number extnum that we want to
    appear as green in the RGB color composite.
:param bluefile: Name of existing FITS file with WCS in HDU number extnum that we want to
    appear as blue in the RGB color composite.
:param extnum: Extension number (zero-based) for data and WCS header
:param usewcs: If True then plot using WCS information
:param vmin: If not None then vmin is the minimum value in the normalaztion interval.
:param vmax: If not None then vmax is the maximum value in the normalaztion interval.
:param yaxlim: 2-element list or tuple of the minimum to maximum x-axis coordinates to
    plot. If None then the default axis limits will be used. To see those limits run
    with verbose=True.
:param yaxlim: 2-element list or tuple of the minimum to maximum x-axis coordinates to
    plot. If None then the default axis limits will be used. To see those limits run
    with verbose=True.
:param verbose: If True then diagnostic information will be written to stdout.
"""

hdur = fits.open(redfile)[extnum]
hdug = fits.open(greenfile)[extnum]
hdub = fits.open(bluefile)[extnum]

# Compute vmin and vmax if necessary. NOTE this only uses the red channel
# percentiles
ipctls = [0.5, 99.5]
opctls = np.nanpercentile(hdur.data, ipctls)
ourmin = opctls[0]
ourmax = opctls[1]
if verbose:
    print(
        f"Input red channel data 0.5th percentile = {ourmin:.4f}, 99.5th percentile = {ourmax:.4f}"
    )

if vmin is not None:
    ourmin = vmin
if vmax is not None:
    ourmax = vmax
if verbose:
    print(f"Applying asinh stretch between {ourmin:.4f} and {ourmax:.4f}")

# Display the image
fig = plt.figure()

for idx in range(3):
    if idx == 0:
        hdu = hdur
        title_str = redfile
        w = wcs.WCS(hdu.header)
        print(80 * "-")
        print(f"WCS for {title_str}:")
        print(w)
    elif idx == 1:
        hdu = hdug
        title_str = greenfile
        w = wcs.WCS(hdu.header)
        print(80 * "-")
        print(f"WCS for {title_str}:")
        print(w)
    elif idx == 2:
        hdu = hdub
        title_str = bluefile
        w = wcs.WCS(hdu.header)

```

```

        print(80 * "-")
        print(f"WCS for {title_str}:")
        print(w)
    else:
        raise RuntimeError(
            f"Error, index {idx} should be in range 0..2 for 3-image plotting."
        )

    if usewcs:
        ax = fig.add_subplot(2, 2, idx + 1, projection=w)
    else:
        ax = fig.add_subplot(2, 2, idx + 1)

    # Create an ImageNormalize object
    norm = ImageNormalize(
        hdu.data, interval=ManualInterval(ourmin, ourmax), stretch=AsinhStretch()
    )

    im = ax.imshow(hdu.data, origin="lower", norm=norm)
    fig.colorbar(im, ax=ax)

    # Display default axis limits
    xlim_used = ax.get_xbound()
    ylim_used = ax.get_ybound()
    if verbose:
        print(f"Default X-axis limits: {xlim_used}")
        print(f"Default Y-axis limits: {ylim_used}")

    ax.set_title(f"{title_str}", fontsize=8)
    if usewcs:
        ax.set_xlabel("Right Ascension (deg)", fontsize=8)
        ax.set_ylabel("Declination (deg)", fontsize=8)
    else:
        ax.set_xlabel("X-axis pixel number", fontsize=8)
        ax.set_ylabel("Y-axis pixel number", fontsize=8)

    # Modify the axis limits?
    if xaxlim is not None:
        loval = np.min(xaxlim)
        hival = np.max(xaxlim)
        ax.set_xbound(lower=loval, upper=hival)
        if verbose:
            print(f"Setting X-axis limits to [{loval}, {hival}]")
    if yaxlim is not None:
        loval = np.min(yaxlim)
        hival = np.max(yaxlim)
        ax.set_ybound(lower=loval, upper=hival)
        if verbose:
            print(f"Setting Y-axis limits to [{loval}, {hival}]")

    return

```

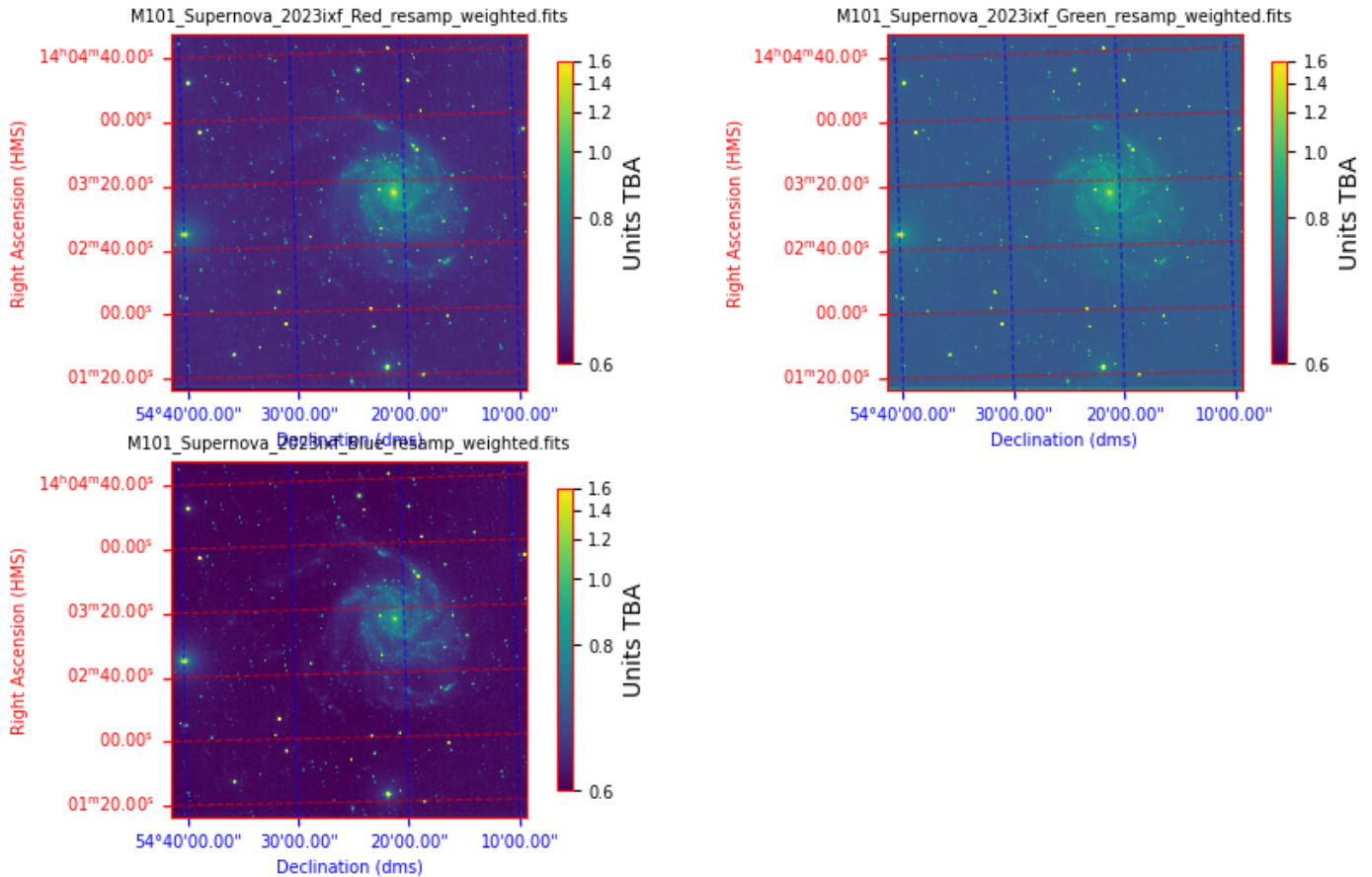
```

7]: bluef = "M101_Supernova_2023ixf_Blue_resamp_weighted.fits"
    greenf = "M101_Supernova_2023ixf_Green_resamp_weighted.fits"
    lumf = "M101_Supernova_2023ixf_Lum_resamp_weighted.fits"
    redf = "M101_Supernova_2023ixf_Red_resamp_weighted.fits"
    valmin = None
    valmax = None
    ap.util.load_imlist_and_plot(
        [redf, greenf, bluef],
        extnum=0,
        output=output_plot,
        usewcs=True,
        vmin=0.6,
        vmax=1.6,
        xaxlim=None,
        yaxlim=None,
        verbose=True,
        angle_tick_spacing_am=10,
        swap_radec_axis=True,
    )

```

The are 3 FITS images in the input imlist
 With 3 files the plot layout is 2 rows x 2 columns.
 Attempting to plot M101_Supernova_2023ixf_Red_resamp_weighted.fits
 Input image data 0.5th percentile = 0.5399, 99.5th percentile = 0.8944

Applying asinh stretch between 0.6000 and 1.6000
Default X-axis limits: (-0.5, 3055.5)
Default Y-axis limits: (-0.5, 3055.5)
Attempting to plot M101_Supernova_2023ixf_Green_resamp_weighted.fits
Input image data 0.5th percentile = 0.6154, 99.5th percentile = 0.9078
Applying asinh stretch between 0.6000 and 1.6000
Default X-axis limits: (-0.5, 3055.5)
Default Y-axis limits: (-0.5, 3055.5)
Attempting to plot M101_Supernova_2023ixf_Blue_resamp_weighted.fits
Input image data 0.5th percentile = 0.5249, 99.5th percentile = 0.8013
Applying asinh stretch between 0.6000 and 1.6000
Default X-axis limits: (-0.5, 3055.5)
Default Y-axis limits: (-0.5, 3055.5)



3.2.1 Astropy make_lupton_rgb

```
3]: from astropy.visualization import make_lupton_rgb

def plot_lupton_threecolor(
    redfile,
    greenfile,
    bluefile,
    outpngfile,
    extnum=0,
    usewcs=True,
    vmin=None,
    xaxlim=None,
    yaxlim=None,
    qval=10,
    stretchval=0.5,
    verbose=True,
):
    """
    Quick and dirty 3-color composite using make_lupton_rgb
```

Note:

- All images must share the same size, WCS orientation, and pixel size.
- Axis limits (`xa xlim` and `ya xlim`) are currently defined in image x-axis and y-axis pixels.

TODO: Find out how to specify them in RA and Dec.

```
:param redfile: Name of existing FITS file with WCS in HDU number extnum that we want to
  appear as red in the RGB color composite.
:param greenfile: Name of existing FITS file with WCS in HDU number extnum that we want to
  appear as green in the RGB color composite.
:param bluefile: Name of existing FITS file with WCS in HDU number extnum that we want to
  appear as blue in the RGB color composite.
:param outpngfile: Name of PNG version of composite the generate.
:param extnum: Extension number (zero-based) for data and WCS header
:param usewcs: If True then plot using WCS information
:param vmin: If not None then vmin is the minimum value the image, i.e. that will correspond to black
:param xa xlim: 2-element list or tuple of the minimum to maximum x-axis coordinates to
  plot. If None then the default axis limits will be used. To see those limits run
  with verbose=True.
:param ya xlim: 2-element list or tuple of the minimum to maximum x-axis coordinates to
  plot. If None then the default axis limits will be used. To see those limits run
  with verbose=True.
:param qual: Value of make_lupton_rgb Q parameter (Q in Lupton et al 2004)
:param stretchval: Value of make_lupton_rgb stretch parameter (alpha in Lupton et al 2004)
:param verbose: If True then diagnostic information will be written to stdout.
"""
```

```
hdur = fits.open(redfile)[extnum]
hdug = fits.open(greenfile)[extnum]
hdub = fits.open(bluefile)[extnum]
w = wcs.WCS(hdur.header)

# Compute vmin and vmax if necessary. NOTE this only uses the red channel
# percentiles
ipctls = [0.5, 99.5]
opctls = np.nanpercentile(hdur.data, ipctls)
ourmin = opctls[0]
ourmax = opctls[1]
if verbose:
    print(
        f"Input red channel data 0.5th percentile = {ourmin:.4f}, 99.5th percentile = {ourmax:.4f}"
    )

vmax = None
if vmin is not None:
    ourmin = vmin
if vmax is not None:
    ourmax = vmax
if verbose:
    print(f"Using an image minimum of {ourmin} for all channels.")

rgb_array = make_lupton_rgb(
    hdur.data,
    hdug.data,
    hdub.data,
    minimum=ourmin,
    stretch=stretchval,
    Q=qval,
    filename=outpngfile,
)

# Display the image
fig = plt.figure()
if usewcs:
    ax = fig.add_subplot(1, 1, 1, projection=w)
else:
    ax = fig.add_subplot(1, 1, 1)

im = ax.imshow(rgb_array, origin="lower")
fig.colorbar(im, ax=ax)

# Display default axis limits
xlim_used = ax.get_xbound()
ylim_used = ax.get_ybound()
```

```

if verbose:
    print(f"Default X-axis limits: {xlim_used}")
    print(f"Default Y-axis limits: {ylim_used}")

title_str = f"RGB composite image stretch={stretchval}, Q={qval}\nRed = {redfile}\nGreen = {greenfile}\nBlue = {bluefile}\n"
ax.set_title(f"{title_str}", fontsize=8)
if usewcs:
    ax.set_xlabel("Right Ascension (deg)", fontsize=8)
    ax.set_ylabel("Declination (deg)", fontsize=8)
else:
    ax.set_xlabel("X-axis pixel number", fontsize=8)
    ax.set_ylabel("Y-axis pixel number", fontsize=8)

# Modify the axis limits?
if xaxlim is not None:
    loval = np.min(xaxlim)
    hival = np.max(xaxlim)
    ax.set_xbound(lower=loval, upper=hival)
    if verbose:
        print(f"Setting X-axis limits to [{loval}, {hival}]")
if yaxlim is not None:
    loval = np.min(yaxlim)
    hival = np.max(yaxlim)
    ax.set_ybound(lower=loval, upper=hival)
    if verbose:
        print(f"Setting Y-axis limits to [{loval}, {hival}]")
return rgb_array

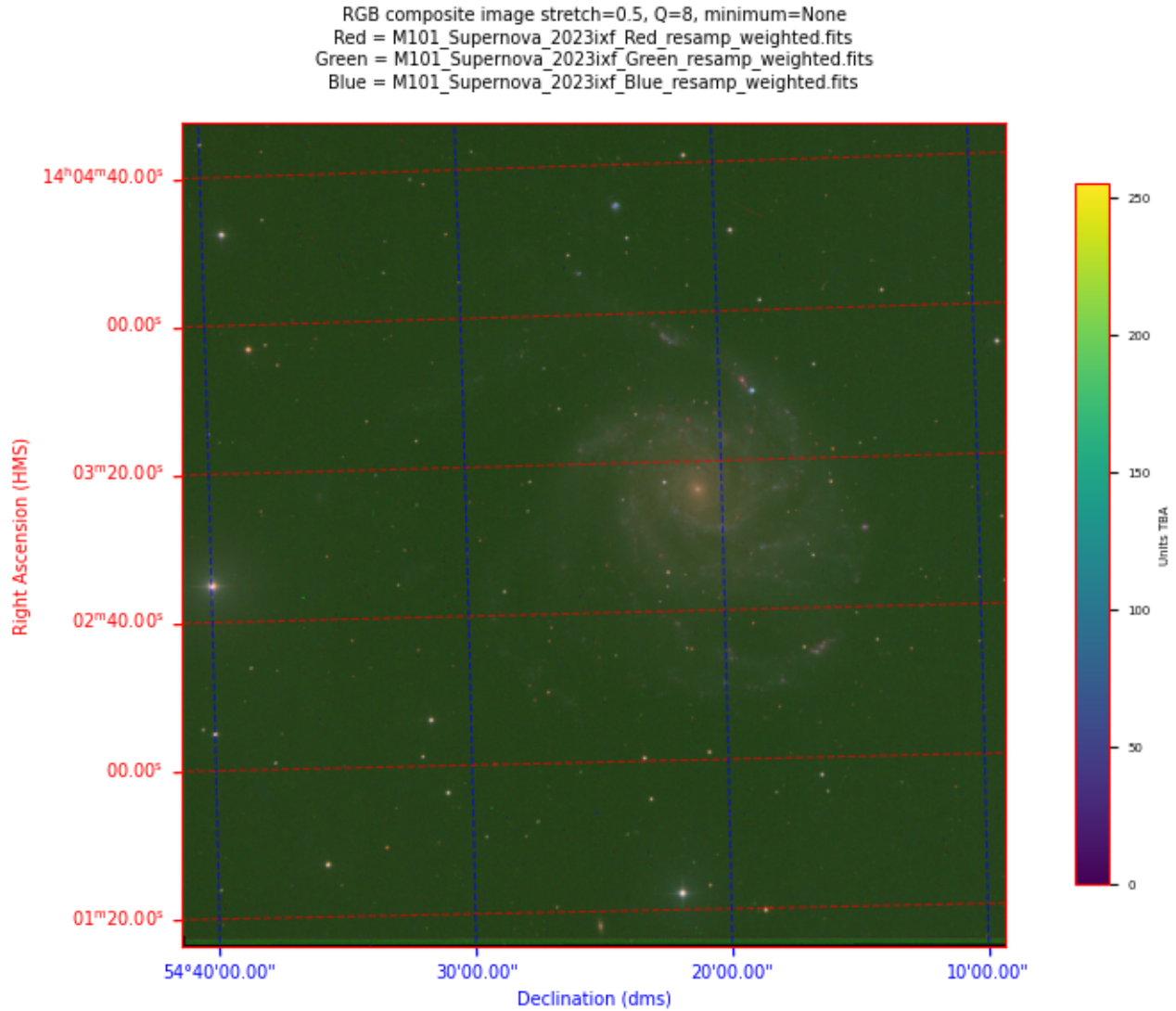
```

```

9]: bluef = "M101_Supernova_2023ixf_Blue_resamp_weighted.fits"
greenf = "M101_Supernova_2023ixf_Green_resamp_weighted.fits"
lumf = "M101_Supernova_2023ixf_Lum_resamp_weighted.fits"
redf = "M101_Supernova_2023ixf_Red_resamp_weighted.fits"
qval = 8
stretch = 0.5
valmin = None
outf = "M101_lupton_RedGreenBlue.png"
rgb = ap.util.plot_lupton_threecolor(
    redf,
    greenf,
    bluef,
    outf,
    extnum=0,
    usewcs=True,
    vmin=valmin,
    xaxlim=None,
    yaxlim=None,
    qval=qval,
    stretchval=stretch,
    verbose=True,
    angle_tick_spacing_am=10,
    swap_radec_axis=True,
)

```

Input red channel data 0.5th percentile = 0.5399, 99.5th percentile = 0.8944
 Using an image minimum of 0.5398805156350136 for all channels.
 Default X-axis limits: (-0.5, 3055.5)
 Default Y-axis limits: (-0.5, 3055.5)
 Wrote plot to M101_lupton_RedGreenBlue.png



Analysis: That looks reasonably decent from a professional astronomer's point of view, but isn't quite good enough for the more display-oriented astrophotographer's point of view: - The cast if the background is too green, given the higher background in the green channel compared to the other images. - The galaxy is too red. - The lack of control over the image maximum value means the galaxy is too faint (reducing Q from the default would help). The feature of the Lupton method not saturating the stars is counterproductive in this case. - My python wrapper has a bug with the axis tickvals and labels, and the color bar is unnecessary.

Of course we can wrap the method in some more pre-processing to fix some of those issues, but showing the default is informative.

3.2.2 Astromatic STIFF

If you have stiff installed then we can use the python subprocess module to run it. Invoking the commands by hand on the command line this would look as follows:

```
stiff M101_Supernova_2023ixf_Red_resamp_weighted.fits M101_Supernova_2023ixf_Green_resamp_weighted.fits M101_Supernova_2023ixf_Blue_resamp_weighted.fits
  -OUTFILE_NAME M101_Supernova_2023ixf_RedGreenBlue_resamp_weighted_gf10_cs10_b8.tiff -GAMMA_TYPE POWER-LAW -GAMMA 2.2 -GAMMA_FAC 1.0 \
  -COLOUR_SAT 1.0 -MAX_TYPE =QUANTILE,QUANTILE,QUANTILE -MAX_LEVEL 0.999,0.999,0.999 -MIN_TYPE =QUANTILE,QUANTILE,QUANTILE -MIN_LEVEL 0.60, \
  -BITS_PER_CHANNEL 8 -VERBOSE_TYPE FULL -DESCRIPTION M101_Supernova_2023ixf -COPYRIGHT dks -WRITE_XML N
```

```
> WARNING: stiff.conf not found, using internal defaults
```

```
----- STIFF 2.7.1 started on 2024-04-04 at 14:33:23 with 16 threads
```

```
----- Inputs:
```

```
M101_Supernova_2023ixf_Red_resamp_weighted.fits: "no ident" 3056x3056 32 bits (floats)
Background level: 0.634998 Min level: 0.618508 Max level: 3.07458
M101_Supernova_2023ixf_Green_resamp_weighted.fits: "no ident" 3056x3056 32 bits (floats)
Background level: 0.700548 Min level: 0.683503 Max level: 2.88235
M101_Supernova_2023ixf_Blue_resamp_weighted.fits: "no ident" 3056x3056 32 bits (floats)
Background level: 0.607307 Min level: 0.590825 Max level: 2.50276
```


----- Output:

M101_Supernova_2023ixf_RedGreenBlue_resamp_weighted_gf10_cs10_b8.tiff: 3056x3056 3x8 bits (integers) gamma: x1.00 compression: LZ

Generated M101_Supernova_2023ixf_RedGreenBlue_resamp_weighted_gf10_cs10_b8.tiff
Stiff took 0.713 seconds.

```
0]: # where are we?
cwd = pathlib.Path(".")
print(f"Current directory: {cwd.resolve()}")
tiff_file = (
    cwd / "M101_Supernova_2023ixf_RedGreenBlue_resamp_weighted_gf10_cs10_b8.tiff"
)
if tiff_file.exists():
    fig, (ax1, ax2) = plt.subplots(1, 2)
    print(f"Loading {tiff_file.name}")
    tiff_img = plt.imread(tiff_file)
    print(f"Plotting full image {tiff_file}")
    ax1.imshow(tiff_img)

    # Zoom in
    print(f"Plotting zoomed-in part of {tiff_file}")
    ax2.imshow(tiff_img[600:1000, 1600:2000])
else:
    print(f"File not found: {tiff_file.name}")
    print(" Skipping display of STIFF generated file...")
```

Current directory:

/old_lnx/home/dks/Downloads/iTelescopeScratch/Plan-20-M101SN/NavigatedImages

Loading M101_Supernova_2023ixf_RedGreenBlue_resamp_weighted_gf10_cs10_b8.tiff

Plotting full image

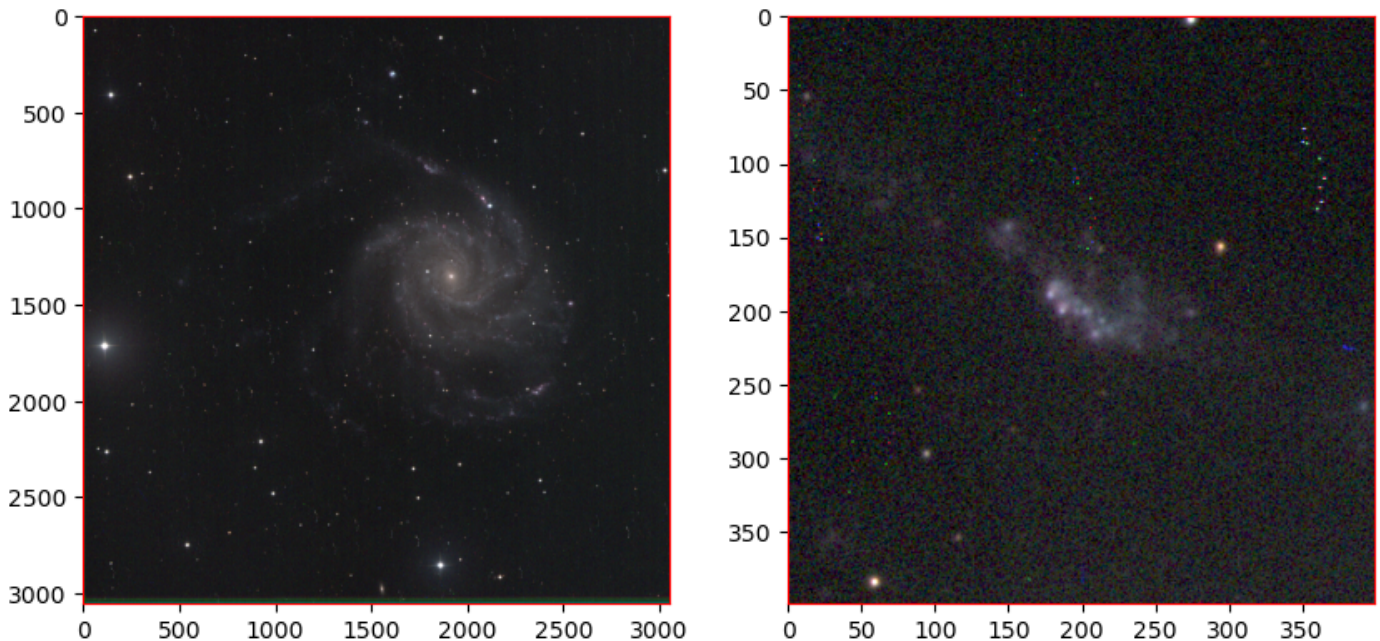
M101_Supernova_2023ixf_RedGreenBlue_resamp_weighted_gf10_cs10_b8.tiff

0]: <matplotlib.image.AxesImage at 0x7f66cd5d1630>

Plotting zoomed-in part of

M101_Supernova_2023ixf_RedGreenBlue_resamp_weighted_gf10_cs10_b8.tiff

0]: <matplotlib.image.AxesImage at 0x7f66b993bf40>



The full image looks a little better, although note that the setting for the image minimum (black-level) are quite different from the make_lupton_rgb case. Zooming in again shows the bad pixels.

Anyway, on to running STIFF from python...

```
1]: stiff_verbose = True # Echo stiff input string if True

# Hard-coded, same as with the Lupton case, sorry...
```

```

bluef = "M101_Supernova_2023ixf_Blue_resamp_weighted.fits"
greenf = "M101_Supernova_2023ixf_Green_resamp_weighted.fits"
redf = "M101_Supernova_2023ixf_Red_resamp_weighted.fits"
description = "M101_Supernova_2023ixf"
user = os.getenv("USER")

# Name for output TIFF image
ofilename = f"M101_Supernova_2023ixf_RedGreenBlue_resamp_weighted_gf10_cs10_b8.tiff"

# Quantiles Red channel green channel blue channel
# Note these are fraction in the range [0:1], not percentages
quantiles_lo = [0.600, 0.600, 0.600]
quantiles_hi = [0.999, 0.999, 0.999]

# See STIFF manual at https://vizier.cfa.harvard.edu/vizier/catstd/man/stiff.pdf
gamma = 2.2
gamma_factor = 1.0
color_sat = 1.0
bitsperpix = 8

fs_tstart = time.perf_counter()
test_cmd1 = f"stiff {redf} {greenf} {bluef} -OUTFILE_NAME {ofilename}"
test_cmd2 = f"-GAMMA_TYPE POWER-LAW -GAMMA {gamma} -GAMMA_FAC {gamma_factor} -COLOUR_SAT {color_sat}"
test_cmd3 = f"-MAX_TYPE =QUANTILE,QUANTILE,QUANTILE -MAX_LEVEL {quantiles_hi[0]:.4f},{quantiles_hi[1]:.4f},{quantiles_hi[2]:.4f}"
test_cmd4 = f"-MIN_TYPE =QUANTILE,QUANTILE,QUANTILE -MIN_LEVEL {quantiles_lo[0]:.4f},{quantiles_lo[1]:.4f},{quantiles_lo[2]:.4f}"
test_cmd5 = f"-BITS_PER_CHANNEL {bitsperpix} -VERBOSE_TYPE FULL -DESCRIPTION {description} -COPYRIGHT {user} -WRITE_XML N"
test_cmd_list = (
    shlex.split(test_cmd1)
    + shlex.split(test_cmd2)
    + shlex.split(test_cmd3)
    + shlex.split(test_cmd4)
    + shlex.split(test_cmd5)
)
if stiff_verbose:
    print(f"Command string to be passed to subprocess.run is {test_cmd_list}")

# Run stiff
fs_tstart = time.perf_counter()
try:
    result = subprocess.run(
        test_cmd_list,
        check=True,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
    )
    fs_tend = time.perf_counter()
    fs_telapsed = fs_tend - fs_tstart # seconds
    print(
        f" Success: process took {fs_telapsed:.3f} seconds, return code {result.returncode}"
    )
    final_images.append(ofilename)
except subprocess.CalledProcessError as err:
    fs_tend = time.perf_counter()
    fs_telapsed = fs_tend - fs_tstart # seconds
    print(
        f" Error, process took {fs_telapsed:.3f} seconds, return code {err.returncode}"
    )
    print(f" Input args: {err.cmd}\n")
    print(f" Stdout: {err.output}\n")
    print(f" Stderr: {err.stderr}\n")
    print(f" Command line equivalent command: {" ".join(err.cmd)}')

# Generate a summary
tiff_file = cwd / ofilename
if tiff_file.exists():
    fig, (ax1, ax2) = plt.subplots(1, 2)
    print(f"Loading {tiff_file.name}")
    tiff_img = plt.imread(tiff_file)
    print(f"Plotting full image {tiff_file}")
    ax1.imshow(tiff_img)

# Zoom in
print(f"Plotting zoomed-in part of {tiff_file}")
ax2.imshow(tiff_img[600:1000, 1600:2000])

```

```

else:
    print(f"Error, file not found: {tiff_file.name}")
    print(" STIFF generated file missing")

```

```

Command string to be passed to subprocess.run is ['stiff',
'M101_Supernova_2023ixf_Red_resamp_weighted.fits',
'M101_Supernova_2023ixf_Green_resamp_weighted.fits',
'M101_Supernova_2023ixf_Blue_resamp_weighted.fits', '-OUTFILE_NAME',
'M101_Supernova_2023ixf_RedGreenBlue_resamp_weighted_gf10_cs10_b8.tiff',
'-GAMMA_TYPE', 'POWER-LAW', '-GAMMA', '2.2', '-GAMMA_FAC', '1.0', '-COLOUR_SAT',
'1.0', '-MAX_TYPE', '=QUANTILE,QUANTILE,QUANTILE', '-MAX_LEVEL',
'0.9990,0.9990,0.9990', '-MIN_TYPE', '=QUANTILE,QUANTILE,QUANTILE',
'-MIN_LEVEL', '0.6000,0.6000,0.6000', '-BITS_PER_CHANNEL', '8', '-VERBOSE_TYPE',
'FULL', '-DESCRIPTION', 'M101_Supernova_2023ixf', '-COPYRIGHT', 'dks',
'-WRITE_XML', 'N']
Success: process took 0.648 seconds, return code 0
Loading M101_Supernova_2023ixf_RedGreenBlue_resamp_weighted_gf10_cs10_b8.tiff
Plotting full image
M101_Supernova_2023ixf_RedGreenBlue_resamp_weighted_gf10_cs10_b8.tiff

```

```

In [1]: <matplotlib.image.AxesImage at 0x7f66bb0df490>

```

```

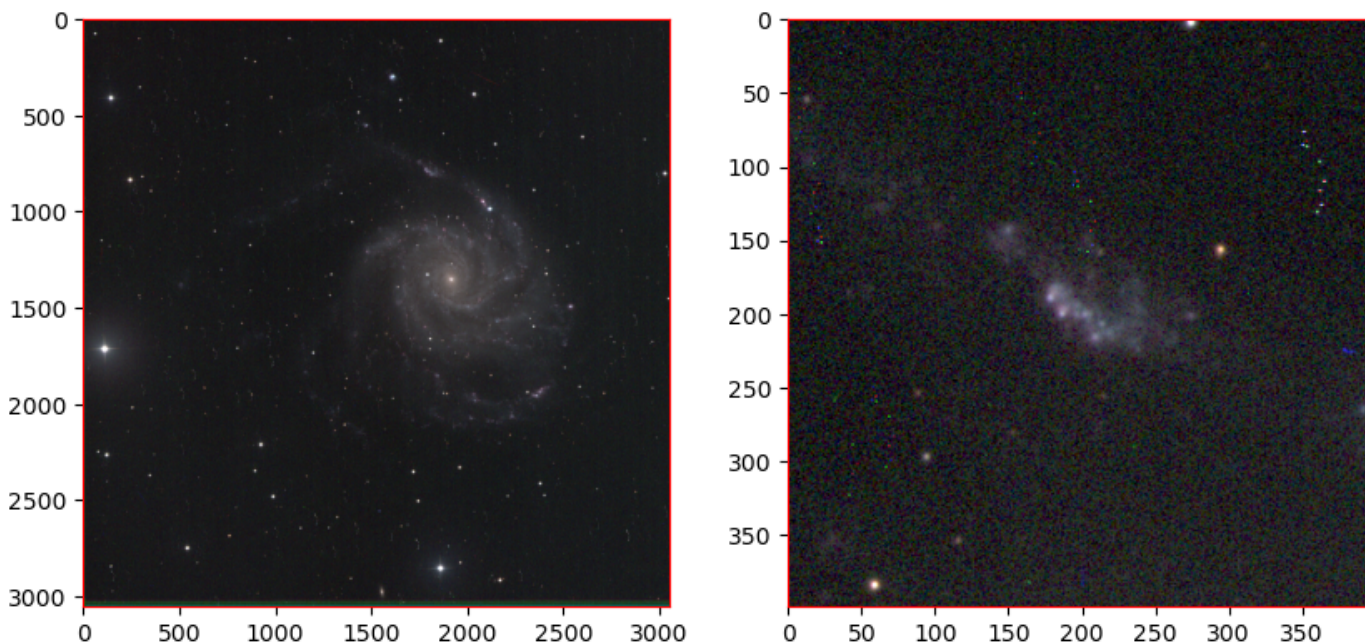
Plotting zoomed-in part of
M101_Supernova_2023ixf_RedGreenBlue_resamp_weighted_gf10_cs10_b8.tiff

```

```

In [1]: <matplotlib.image.AxesImage at 0x7f66bb081f90>

```



Other methods: composite_all.sh shell script The bash script `composite_all.sh` uses Astromatic's `stiff` to combine FITS images from multiple filters into RGB tiff files. It iterates through multiple combinations of output image bit-depth, color saturation and gamma factor. The user must then choose the best version to work on further using an image manipulation program such as the `gimp`.

```

cd /old_lnx/home/dks/Downloads/iTelescopeScratch/Plan-20-M101SN/Resampled/
~/git/AstroPhotography/AstroPhotography/scripts/composite_all.sh M101_Supernova_2023ixf 4096x4096_resamp_median.fits rgb
/home/dks/git/AstroPhotography/AstroPhotography/scripts/composite_all.sh started at Sun Feb 25 09:05:32 AM EST 2024
Running from /old_lnx/home/dks/Downloads/iTelescopeScratch/Plan-20-M101SN/Resampled
Logging file: tmp_composite_all.log
File prefix: M101_Supernova_2023ixf
Fill suffix: 4096x4096_resamp_median.fits
Color selections: rgb
Using stiff from /usr/local/bin/stiff

```

```

-----
Processing 3-color combination rgb
Filters in red, green, blue order: Red Green Blue
Checking that input FITS files exist:
Found M101_Supernova_2023ixf_Red_4096x4096_resamp_median.fits
Found M101_Supernova_2023ixf_Green_4096x4096_resamp_median.fits

```

```
Found M101_Supernova_2023ixf_Blue_4096x4096_resamp_median.fits
Bits-per-pixel=8, gamma-fac=1.0, color_sat=1.0
Running stiff, generating M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf10_cs10_b8.tiff at Sun Feb 25 09:05:32 AM EST 2024
Generated M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf10_cs10_b8.tiff
Stiff took 0.797 seconds.

Bits-per-pixel=8, gamma-fac=1.0, color_sat=1.5
Running stiff, generating M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf10_cs15_b8.tiff at Sun Feb 25 09:05:33 AM EST 2024
Generated M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf10_cs15_b8.tiff
Stiff took 0.731 seconds.

Bits-per-pixel=8, gamma-fac=1.0, color_sat=2.0
Running stiff, generating M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf10_cs20_b8.tiff at Sun Feb 25 09:05:33 AM EST 2024
Generated M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf10_cs20_b8.tiff
Stiff took 0.729 seconds.

Bits-per-pixel=8, gamma-fac=1.2, color_sat=1.0
Running stiff, generating M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf12_cs10_b8.tiff at Sun Feb 25 09:05:34 AM EST 2024
Generated M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf12_cs10_b8.tiff
Stiff took 0.734 seconds.

Bits-per-pixel=8, gamma-fac=1.2, color_sat=1.5
Running stiff, generating M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf12_cs15_b8.tiff at Sun Feb 25 09:05:35 AM EST 2024
Generated M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf12_cs15_b8.tiff
Stiff took 0.716 seconds.

Bits-per-pixel=8, gamma-fac=1.2, color_sat=2.0
Running stiff, generating M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf12_cs20_b8.tiff at Sun Feb 25 09:05:36 AM EST 2024
Generated M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf12_cs20_b8.tiff
Stiff took 0.721 seconds.

Bits-per-pixel=8, gamma-fac=1.4, color_sat=1.0
Running stiff, generating M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf14_cs10_b8.tiff at Sun Feb 25 09:05:36 AM EST 2024
Generated M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf14_cs10_b8.tiff
Stiff took 0.732 seconds.

Bits-per-pixel=8, gamma-fac=1.4, color_sat=1.5
Running stiff, generating M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf14_cs15_b8.tiff at Sun Feb 25 09:05:37 AM EST 2024
Generated M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf14_cs15_b8.tiff
Stiff took 0.714 seconds.

Bits-per-pixel=8, gamma-fac=1.4, color_sat=2.0
Running stiff, generating M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf14_cs20_b8.tiff at Sun Feb 25 09:05:38 AM EST 2024
Generated M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf14_cs20_b8.tiff
Stiff took 0.726 seconds.
```

```
-----
Run summary:
Img  Resampled Output                                     Time Status
0    M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf10_cs10_b8.tiff 0.797      0
1    M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf10_cs15_b8.tiff 0.731      0
2    M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf10_cs20_b8.tiff 0.729      0
3    M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf12_cs10_b8.tiff 0.734      0
4    M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf12_cs15_b8.tiff 0.716      0
5    M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf12_cs20_b8.tiff 0.721      0
6    M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf14_cs10_b8.tiff 0.732      0
7    M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf14_cs15_b8.tiff 0.714      0
8    M101_Supernova_2023ixf_RedGreenBlue_4096x4096_resamp_median_gf14_cs20_b8.tiff 0.726      0
Master log file: tmp_composite_all.log
/home/dks/git/AstroPhotography/AstroPhotography/scripts/composite_all.sh finished at Sun Feb 25 09:05:39 AM EST 2024, run time 6.716 seconds.
```

4 Summary and Conclusions

We’ve processed a (small) set of iTelescope Premium pre-calibrated images using a lot of hand-written python wrapping around individual **AstroPhotography** package functions: star detection, astrometric solutions, stacking, and processing color composite images from the stacked images in each band. The results are OK, but still leave a lot to be desired.

- This particular set of premium images, although calibrated, still include a lot of hot/bad pixels.
 - Those would be fixed if we were processing from the raw files using our own calibration functions.
 - We’ll need a way of fixing those in any eventual calibrated images pipeline, along with recognizing whether bad pixel correction has been already applied at calibration.
- The “sky” background in the images is high. This might be a pedestal, or just the ambient conditions.
 - Again, we’ll need a way of fixing that in any eventual calibrated images pipeline, along with recognizing whether sky background correction (and/or pedestal removal) has been already applied at calibration.

4.1 Versions and Changes

Version	Date	Description
0.5.2-alpha	2024-04-26	Partial version includes by-hand processing, but not first version of pipeline script
0.6.0	2024-11-26	Final version worked under issue-002

```
2]: # Close all the open figures in memory  
plt.close("all")
```